

ON-DEVICE TRAINING

RigL — Complete

Dynamic sparse training for the HAR classifier on the Nucleo-F746ZG

Concept, then survey, then code.

Every listing here compiles clean under `-Wall -Wextra -Wconversion -Wdouble-promotion -Wvla`, and the algorithm is verified by the harness in Part VIII. Where something is not implemented it is marked. Nothing in this document contradicts anything else in it.

Part	Covers
I	What RigL is, what it costs, and what it can honestly claim on this model
II	The codebase as it stands, and where each new piece belongs
III	The tensor-level primitives: mask, popcount, histogram, thresholds
IV	Establishing the initial topology — and why ERK, not uniform
V	The algorithm itself
VI	Integration: three insertions, and two traps that fail silently
VII	Persistence: the four resume gaps
VIII	Verification: assertions, harness, build order
App. A	Complete source, file by file

How to read this document

Box conventions, and a note on what changed

IN PLAIN ENGLISH — the gold box restates things without jargon

Whenever a passage gets technical, a gold box says the same thing the way you would explain it to a colleague. If a paragraph loses you, read ahead to the next gold box, then come back knowing where the details are heading.

DEFINITION — the teal box pins down one word

Terms like "mask", "density", "order statistic" and "invariant" all have narrow technical meanings here. Each gets a teal box the first time it appears.

TRACED — the purple box works one example by hand

Every number in a traced box was either computed and checked, or read from the real source and dataset. None are illustrative unless the box says so.

Box	Gives you
IN PLAIN ENGLISH	the idea without jargon
DEFINITION	one term, pinned down
TRACED	one example, followed by hand
CONCEPT	the computer-science background
ALGORITHM	the numbered recipe
ON THE STM32	what changes on the Nucleo-F746ZG
DEFECT	a real bug in the current source, and its fix
WARNING / TIP	a trap, or a practice worth adopting

NOTE — what changed from the earlier edition

The previous version grew by accretion: a main body, then two appendices that *corrected* it. A reader met the wrong `rigLStep` signature on page 60 and the right one on page 150.

This edition is resequenced so that each thing is stated once, correctly, in the place it belongs. Three substantive corrections are now integrated rather than appended: the `scratch` parameter is gone (Part III.6), `rigLStep` takes an optimizer (Part V.4), and the code is split across two files to avoid a build cycle (Part II.5).

PART I

The Problem and the Idea

What sparse training is, how RigL differs from pruning, what it costs on a 320 KB device, and — the part most easily got wrong — what it can honestly claim on a model this size.

I.1 Sparse training, and why it is not pruning

IN PLAIN ENGLISH — the idea in one paragraph

Most of a trained network's weights contribute almost nothing. If you could identify the useful ten percent in advance you could train only those, and save the memory and the arithmetic of the other ninety.

You cannot identify them in advance. That is the whole difficulty, and the different techniques in this area are different answers to it.

Approach	When the network is sparse	Needs dense memory?	Can it fix a bad choice?
Post-training pruning	only at the end	yes, throughout	n/a
Static sparse	from step 1, fixed topology	no	no
RigL	from step 1, topology evolves	no	yes

DEFINITION — sparse-to-sparse training

The network is sparse from the very first step and stays at the same sparsity throughout. This is what RigL does. It matters on a microcontroller because the alternative — train dense, then prune — requires enough memory for the dense model, and therefore does not help at all if the dense model never fit.

DEFINITION — mask

One boolean per weight saying whether it is active. Packed eight to a byte. The mask *is* the topology: which connections exist. Everything RigL does is a change to the mask.

DEFINITION — density and sparsity

Two names for the same thing from opposite ends. **Density** is the fraction of weights that are active; **sparsity** is the fraction that are not. 90% sparsity is 10% density. The code uses sparsity at the interface and density internally in the ERK calculation, so both appear.

I.2 RigL in one page

Drop the smallest, grow the largest gradients

IN PLAIN ENGLISH — the whole algorithm

The network starts sparse. Every deltaT steps — typically 100 — RigL rewires: it switches **off** the active weights with the smallest magnitude, and switches **on** an equal number of inactive weights with the largest gradient.

The two criteria are different on purpose. A small weight is barely contributing, so losing it costs little. A large gradient on a *dormant* weight means "if this connection existed, it would move a lot" — the network asking for it. Same number out as in, so the sparsity never drifts.

ALGORITHM — one topology update, per layer

1. $\alpha = \alpha_{\text{Init}} \times 0.5 \times (1 + \cos(\pi \times \text{progress}))$
2. $\text{numActive} = \text{popcount of the mask}$
3. $k = \alpha \times \text{numActive}$; if 0, skip this layer
4. Find the magnitude threshold below which k active weights lie; clear their bits; count how many *actually* went as $n\text{Dropped}$
5. Find the gradient threshold above which $n\text{Dropped}$ inactive weights lie; set their bits, stopping at exactly $n\text{Dropped}$
6. Write real 0.0 into everything now inactive



A 10%-dense mask: 160 weights, 16 active. RigL never changes how many cells are blue, only which ones.

CONCEPT — why the count is conserved, and why that is structural

Step 5 grows $n\text{Dropped}$, not k . That looks like a detail and is the single most important line in the algorithm.

Under int8 quantisation a weight can hold only 256 distinct values, so in a layer of any size many weights share each magnitude. Asking to drop the 649 smallest will usually drop 652 or 700, because the threshold lands on a value that many weights hold exactly. Ties are the normal case here, not an edge case.

Grow back k and the layer loses a few weights on every update, drifting away from its target sparsity over hundreds of updates with nothing reporting it. Grow back $n\text{Dropped}$ and the count is identical before and after, by construction. Part V.3 works through the numbers.

I.3 The cosine schedule

Explore early, settle late

```
float progress = (float)step / (float)totalSteps;  
float alpha    = alphaInit * 0.5f * (1.0f + cosf(3.14159265f * progress));
```

IN PLAIN ENGLISH — why the rewiring rate decays

Early in training the network knows nothing, so rewiring aggressively costs little and may find far better connections. Late in training the topology is good and the weights have been tuned to it — rewiring now throws away real progress, because a freshly grown weight starts at zero and must learn from scratch.

Cosine decay encodes exactly that. It starts at a `alphaInit`, falls smoothly, and reaches exactly zero at the end, so there is no step at which the network is disrupted sharply and no rewiring at all in the final phase.



Smooth decay to exactly zero. The final topology is stable while the weights make their last refinements.

progress	cos(pi x progress)	alpha at alphaInit = 0.3	What is happening
0.00	+1.0	0.300	30% of active weights rewired — aggressive exploration
0.25	+0.707	0.256	still exploring freely
0.50	0.0	0.150	halfway; half the initial rate
0.75	-0.707	0.044	mostly settled
1.00	-1.0	0.000	no rewiring; the topology is final

WARNING — alpha depends only on the step count, and nothing saves it

If a run is interrupted at step 15,000 of 20,000 and resumed from a checkpoint that did not store `globalStep`, then `step` is 0, `progress` is 0, and `alpha` snaps back to the full 0.3.

The next `RigL` call then rewires 30% of a converged topology. Nothing errors, nothing warns; the loss jumps and the run never recovers. Part VII closes this and three related gaps at one seam.

I.4 What this actually costs on the HAR classifier

The honest numbers

IN PLAIN ENGLISH — the model is smaller than you might assume

The HAR classifier is not a large network. It is three Conv1d blocks and a small linear head: **10,214 parameters**, 40 KB as float32 and 10 KB as int8.

That matters because the usual motivation for sparsity — "the model does not fit" — is simply not available here. It fits comfortably. The real argument is different and better, and it is worth getting right before writing a thesis around it.

Layer	Params	% of params	MACs / sample	% of MACs
conv1 9->16 K7	1,024	10.0%	129,024	26.3%
conv2 16->32 K5	2,592	25.4%	163,840	33.4%
conv3 32->64 K3	6,208	60.8%	196,608	40.1%
linear 64->6	390	3.8%	384	0.1%
total	10,214	100%	489,856	100%

Component	float32	Note
weights	39.9 KB	10,214 parameters × 4 bytes
gradients	39.9 KB	one per weight, same shape
SGD momentum	39.9 KB	one per weight again
activations retained for backward	61.0 KB	one sample, micro-batch 1
one input window	4.5 KB	9 × 128 × 4
training total	185.2 KB	58% of the 320 KB available
<i>inference only</i>	<i>47.9 KB</i>	<i>weights plus peak activation</i>



Training needs 3.9x what inference needs. It fits — but with 135 KB left for stack, buffers and the runtime, not comfortably.

CONCEPT — so what is the real argument for quantisation here?

Not "the model does not fit". Three better ones, in order of strength.

1. Training costs 3.9× inference. A device that can comfortably *run* this model cannot necessarily *train* it. Closing that gap is the whole point of on-device training, and it is the honest framing of the problem.

2. 58% of SRAM is tight, not safe. 135 KB remains for everything else, on a part with no MMU, in a codebase with documented unbounded stack arrays. Headroom is a safety property.

3. Activations outweigh weights. 61.0 KB against 39.9 KB, because convolutions have few weights and large outputs. Quantising activations saves more than quantising weights — and RigL, which sparsifies weights, does not touch the larger number at all.

I.5 What RigL can honestly claim here

Worth settling before the experiments

Claim	Supportable?	Why
"RigL makes the model fit"	no	It already fits — 40 KB of weights on a 320 KB device
"RigL reduces training memory"	weakly	Saves at most 36 KB of 185 KB, and not the 61 KB of activations
"RigL reduces MACs"	not as implemented	A dense matmul over zeroed weights costs the same; see VI.4
"RigL works under FQT"	yes — this is the contribution	Genuinely open, and this is a clean fast testbed
"FQT reduces training memory 3.7x"	yes	185.2 KB to 49.8 KB, from real shapes

CONCEPT — the honest framing, and why it is a better thesis

At 90% sparsity this model retains 1,021 active weights. Conv1 keeps 102 of its 1,024. That is small, and the memory argument for sparsity is correspondingly weak.

The defensible claim is not an engineering one but a **scientific** one: *does RigL-style dynamic sparse training still work when every tensor in the training loop is int8?* Nobody has established that. RigL was developed and validated in float32; FQT was developed on dense networks. The interaction is open, and there is a specific reason to think it is hard — the drop criterion is weight magnitude, and under int8 only 256 distinct magnitudes exist, so ties dominate.

Framed that way the small model is an **advantage**: fast to run, easy to see failure modes in. You are not claiming RigL makes HAR fit on a microcontroller. You are asking whether the two techniques compose.

I.6 The research question

The question as posed

Can RigL-style dynamic sparse training be made to work under fully quantized training (int8 forward + backward + gradient) on a Cortex-M7 microcontroller, for the HAR dataset?

IN PLAIN ENGLISH — what is right about it

It names the constraint precisely — int8 through the *backward* pass, not merely quantised inference, which is the harder and less-studied case. It names the hardware and the task. It is specific enough that someone could disagree with the answer.

WARNING — and the one phrase that needs pinning down

"Be made to work" is load-bearing and undefined. What would **no** look like? Poor accuracy compared to what?

Without a criterion, a negative result is uninterpretable — low accuracy could be the sparsity, the quantisation, the learning rate, the data split, or a bug. All five are plausible and the question cannot distinguish them.

CONCEPT — four things could cause a "no", and they need separating

The mechanism — does drop-and-grow still select meaningfully when the signals are int8? *Scientific*.

The hardware — does it fit in 320 KB and run in tolerable time? *Engineering*.

The task — does HAR have enough signal at 10% density? *Task-dependent*.

The implementation — is the code correct? *Confound*.

The fourth is what Part VIII's harness and assertions are for: they remove it as an explanation before the experiments begin. The other three need separate measurements.

ALGORITHM — the decomposition

Q1 — mechanism. Does drop-and-grow still select meaningfully under int8? Measured by the stillborn fraction (Part IX.3). Needs no accuracy number at all.

Q2 — outcome. Does RigL under FQT beat *static sparse* under FQT on HAR test accuracy, across seeds? The headline result.

Q3 — feasibility. Does it run on the F746ZG within memory and in tolerable wall-clock? Reported as peak SRAM and seconds per epoch.

NOTE — a "no" on Q3 is still a thesis

If Q1 and Q2 come out yes and Q3 comes out no, you have shown the technique works and the hardware needs to be larger — with the exact number quantifying how much larger. That is a useful result, and it is only available because the three were measured separately.

TIP — a phrasing that carries the whole thing

Can RigL-style dynamic sparse training be made to work under fully quantized training (int8 forward, backward and gradient) on a Cortex-M7, for HAR — and if so, what mechanism makes the rewiring survive quantisation?

The clause after the dash turns a yes/no into a contribution. Part VI.3 and VI.4 give the candidate answer: stochastic rounding, because grown weights start at exactly zero and must escape the dead zone. If that is right, the finding generalises beyond HAR and beyond this board.

I.7 The three baselines

What "works" is measured against

IN PLAIN ENGLISH — the dense model is the wrong comparison

Comparing RigL+FQT against a dense float32 model tells you the combined cost of being sparse *and* being quantised, which is two effects at once and neither of them is the question.

The comparison that isolates rewiring is **static sparse at the same sparsity, under the same quantisation**. Everything is held fixed except whether the mask ever changes.

Run	Sparsity	Rewiring	Quantisation	What it tells you
1. Dense FQT	0%	—	int8	the ceiling — what sparsity costs at all
2. Static sparse FQT	90% ERK	none	int8	what RigL must beat
3. RigL FQT	90% ERK	every deltaT	int8	the treatment
4. Dense float32	0%	—	float32	optional: what quantisation costs

CONCEPT — what each outcome would mean

Run 3 lands on run 2. The rewiring machinery ran perfectly and contributed nothing. That is a sharp **no**, and Part IX.3 tells you whether the cause was stillborn weights.

Run 3 sits between 2 and 1. Rewiring recovers some of the dense-to-sparse gap. Report the fraction recovered — that is the headline number.

Run 3 reaches run 1. Sparsity costs nothing at this density. Worth checking the sparsity is actually being maintained before believing it.

Run 3 falls below run 2. Rewiring is actively harmful — which would be the most interesting outcome of all, and points straight at the thrashing cycle in Part VI.3.

WARNING — run 2 must use the same ERK distribution as run 3

If run 2 uses uniform 90% and run 3 uses ERK, you are comparing two things at once and any difference could be the distribution rather than the rewiring. Part IV.4 shows uniform leaves the classifier with 38 weights against ERK's 278 — a large enough gap to dominate the result.

Same ERK budget, same seed for the initial mask, same everything. Only the rewiring differs.

NOTE — and report across seeds, not once

The initial mask is random. A single run of each configuration compares one random topology against another, and at 90% sparsity the variance between topologies is substantial. Five seeds minimum, and report the spread rather than the best.

PART II

The Codebase, and Where the New Code Goes

Twelve of the twenty-two functions in the call sequence already exist and work. Two exist but stop the build. Eight must be written — and they do not all belong in the same file.

II.1 What already exists

More than you might expect

 exists and works  exists but defective  not written yet  control flow, not a call

Primitive	File	Status	What RigL uses it for
tensorBoolGet / Set	Tensor.c	works	Test and flip one packed bit
tensorMaskCountActive	Tensor.c	works	numActive, from which k is derived
calcNumberOfBytesForData	Tensor.c	works	Size the packed buffer; BOOL is handled correctly
readBytesAsFloat	DTypes.c	works	Aliasing-safe read of a weight or gradient
getParamFromParameter	Tensor.c	works	Reach the values
getGradFromParameter	Tensor.c	works	Reach the gradients
rngNextFloat	RNG.c	works	Random initial topology
rngGetSeed / SetSeed	RNG.c	works, never called	Would checkpoint the RNG — see VII
serializeModel	Serialize.c	works	The frame the mask record slots into
optimizerStep	Sgd.c, AdamW.c	works	Applies updates — including to pruned weights
tensorMaskInitDense	Tensor.c	will not compile	All-ones mask; III.4
tensorApplyMaskZero	Tensor.c	will not compile	The invariant; III.5

IN PLAIN ENGLISH — the mask machinery is already there

Bit packing, population count with correct tail handling, byte-accurate sizing, aliasing-safe reads — all present and working. What is missing is the algorithm that drives them, plus one hook on the optimizer.

That makes the job more contained than a list of eight missing functions suggests.

II.2 What is defective

DEFECT — three single-token typos stop Tensor.c compiling

1. Line 18: `size_t nBytes = TENSOR_MAX_BYTES;` — no such identifier exists anywhere in the project.

2. and 3. Lines 50 and 68: `values->quantization->type` where the parameter is named `value`.

All three are caught by simply running the build. They are fixed in III.4 and III.5 respectively, and they must be fixed first — until then any error you see is from these rather than from your new code.

WARNING — the first one is the dangerous kind

A misspelled *variable* is caught immediately. But "maximum tensor bytes" is a plausible thing for someone to define later — and the day anyone does, the file compiles and memsets a fixed size over a variable-size buffer. Silent heap corruption rather than a build error.

II.3 What is missing

Function	Goes in	Why it is needed
rigLKthSmallestActive	src/tensor/RigL.c	The drop threshold
rigLKthLargestInactive	src/tensor/RigL.c	The grow threshold
rigLInitSparsity	src/tensor/RigL.c	Drop to target sparsity before step 1
rigLInitSparsityERK	src/tensor/RigL.c	Distribute one budget across layers
rigLApplyToParameter	src/tensor/RigL.c	Enforce the invariant
rigLLayerSparseParts	src/layer/RigLStep.c	Ask whether a layer participates
rigLStep	src/layer/RigLStep.c	The algorithm
optimizerClearElementState	src/optimizer/	Zero momentum for pruned weights

RigL GAP — both RigL files are currently zero bytes

src/tensor/RigL.c and src/tensor/include/RigL.h exist in the tree and are empty. sparsityConfig in Tensor.h is an empty struct. serializeSparsity() is an empty function taking no parameters.

Four places where the extension point was scaffolded and left unfilled. The positioning is correct in every case, which is the good news.

II.4 The call sequence

Four phases, three of which repeat



Phase 2 is the one most often left out: it runs every step, not every deltaT, and without it the layer silently stops being sparse. Part VI.2.

II.5 Where the new code goes

Two files, because one would close a build cycle

IN PLAIN ENGLISH — the obvious layout does not work

The instinct is to put all of `RigL` in `src/tensor/RigL.c`, next to the mask primitives it uses. That is wrong, and the build system is what says so.

`rigLStep` takes a `layer_t **` and an `optimizer_t *`, so its header must include `Layer.h` and `Optimizer.h`. But `Layer` already links `Tensor`.

The existing dependency

```
# src/layer/CMakeLists.txt, as it stands today
add_library(Layer Layer.c)
target_link_libraries(Layer PRIVATE
    DTypes
    Tensor      # <-- layer already depends on tensor
    Arithmetic
    ...
)
```

WARNING — putting `rigLStep` in the `Tensor` library closes a cycle

You get `Tensor -> Layer -> Tensor`. CMake tolerates cycles among static libraries by repeating them on the link line — `Linear` and `Layer` already link each other — but it is not a thing to add to deliberately.

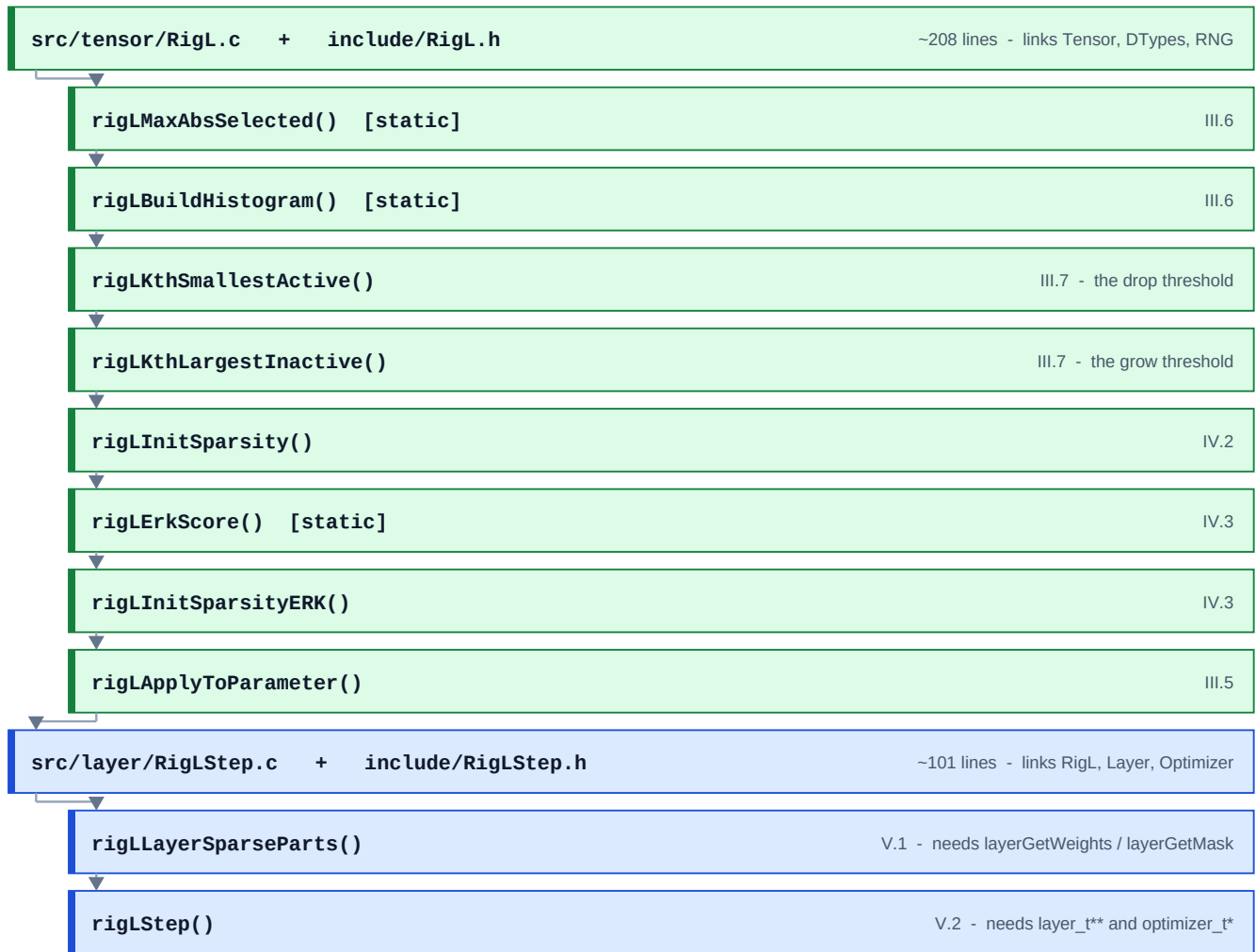
The practical cost is that the `Tensor` library stops being usable on its own. Any unit test for the threshold searches would have to drag in every layer type and every optimizer.

CONCEPT — the split falls out naturally

The threshold searches, the histogram, the sparsity initialisation and the invariant helper need **only** `Tensor.h`. They operate on a tensor and a mask and know nothing about layers.

Only two functions need the layer world: `rigLLayerSparseParts`, which asks a layer for its weights and mask, and `rigLStep`, which walks the model and clears optimizer state.

So the boundary the headers already wanted is exactly the boundary the build system needs. Both halves compile clean, and `RigL.c` contains no reference to `layer_t`, `optimizer_t`, `layerGet*` or `optimizerClear*`.



The two translation units. Nothing in the upper group references the lower, so the dependency runs one way only.

CMakeLists entries

src/tensor/CMakeLists.txt

```
# src/tensor/CMakeLists.txt -- append

add_library(RigL
    RigL.c
)
target_include_directories(RigL PUBLIC include)
target_link_libraries(RigL PRIVATE
    Tensor
    DTypes
    RNG
    Common
    m
)
```

src/layer/CMakeLists.txt

```
# src/layer/CMakeLists.txt -- append

add_library(RigLStep
    RigLStep.c
)
target_include_directories(RigLStep PUBLIC include)
target_link_libraries(RigLStep PRIVATE
    RigL          # the tensor-level half
    Layer
    Optimizer
    Tensor
    Common
    m
)
```

NOTE — the dependency direction is one-way

RigLStep links RigL. RigL links neither RigLStep nor Layer. So the tensor library stays independently usable, and the test harness in Part VIII needs only RigL, Tensor and DTypes — which is what makes it practical to put in Cl.

TIP — build the tensor half first

It has no dependency on the layer half, so it compiles and can be unit-tested on its own. Get RigL.c passing the Part VIII harness before writing a line of RigLStep.c. Any error you then hit in the second file is genuinely about the second file.

II.6 The C idioms used throughout

A short primer, so the line-by-line commentary reads without stopping

IN PLAIN ENGLISH — why this section exists

The listings from Part III onward are explained line by line, and those explanations lean on a handful of C constructs that are unremarkable to an experienced reader and opaque to everyone else.

They are collected here so you meet each one once, with a reason, rather than tripping over it mid-algorithm. Skip this if you write C daily.

DEFINITION — static, at file scope

On a variable or function declared outside any function body, `static` means "visible only inside this `.c` file". The thing still exists for the whole run — only the *name* is hidden. It is C's private.

Used here for `g_hist` (the 1 KB histogram, which nothing outside `RigL.c` should touch) and for the two helper functions that support the threshold searches.

DEFINITION — const, on a pointer parameter

`tensor_t const *mask` promises the function will not modify what `mask` points at. It is enforced by the compiler: an accidental write becomes a build error rather than a bug.

Look at which parameters carry it. `tensorBoolGet` takes a `const tensor` and `tensorBoolSet` does not — the signatures say which one writes, before you read a line of the body.

DEFINITION — the ternary operator

`a ? b : c` means "if `a` is true use `b`, otherwise use `c`". It is an *expression*, so it can appear inside a larger one — which is why `rigLStep` can write `(totalSteps > 0u ? totalSteps : 1u)` directly inside a division rather than needing a separate `if`.

DEFINITION — unsigned arithmetic does not go negative

`size_t` is unsigned. Subtract 1 from 0 and you do not get `-1`; you get 4,294,967,295 on the M7. This is defined behaviour, not a bug in C, and it is why several guards in these listings exist — `if (nBytes == 0u) return;` before indexing `nBytes - 1`, for instance.

Idiom	Means	Where it appears here
<code>a ^= b</code>	<code>a = a ^ b</code>	compound assignment, throughout
<code>i / 8, i % 8</code>	byte index, bit index	every packed-mask access
<code>(n + 7u) / 8u</code>	divide by 8, rounded up	sizing the packed buffer
<code>(1u << t) - 1u</code>	a run of <code>t</code> ones	the tail mask
<code>for (size_t b = N; b-- > 0;)</code>	count down without wrapping	the grow-phase bucket walk
<code>memcpy(&x, p, sizeof x)</code>	reinterpret bytes safely	reading a float from a byte buffer
Type <code>**out</code>	an out-parameter	<code>rigLLayerSparseParts</code>
<code>#ifndef NDEBUG</code>	debug-build only	the two assertions in <code>rigLStep</code>

CONCEPT — why memcpy and not a pointer cast

To read four bytes as a float, the tempting code is `*(float *)bytes`. It compiles, usually works, and is undefined behaviour twice over.

Strict aliasing: C assumes pointers of different types never refer to the same memory, which lets the optimiser cache values in registers. Break that and at `-O2` the compiler may reorder or discard your reads — so the failure appears when you turn on optimisation for the release build.

Alignment: a byte buffer has no alignment guarantee. On Cortex-M7 an unaligned float load can raise a `UsageFault` and halt the device. `memcpy` has neither problem, and for a four-byte copy the compiler emits the same single load instruction the cast would have. You pay nothing for correctness.

DEFINITION — the out-parameter pattern

A C function returns one value. When you need several — a success flag plus two pointers, say — the convention is to return the flag and take **out-parameters**: pointers to variables the caller owns, which the function fills in.

Hence `parameter_t **weightsOut`. The caller passes `&weights`; the function writes through it with `*weightsOut = w`. The double star is not complexity for its own sake — it is the only way to change what the caller's pointer points at.

TIP — a convention worth noticing as you read

Every function in these listings writes its out-parameters **only** on the success path, after all checks have passed. A caller that ignores the return value and uses the outputs anyway gets whatever it initialised them to — `NULL`, if it followed the pattern — rather than a half-filled result.

That is what makes a header comment like "on failure both are left untouched" a guarantee rather than a hope.

II.7 The running example

One 16-element tensor, followed through every function

IN PLAIN ENGLISH — why a toy tensor rather than a real layer

Conv3 has 6,144 weights. You cannot check 6,144 numbers by hand, so a trace of it would be a trace you have to take on faith — which is the opposite of the point.

So every worked example from here on uses the same **16-element tensor**. It is small enough to print in full and large enough to exhibit every behaviour that matters: ties at the threshold, a weight dropped and immediately grown back, and the conservation property. Every number was produced by running the real code, not by hand.

Setting	Value
elements	16
initially active	the 8 even indices — mask bytes 0x55 0x55
k for the traced update	3
what we follow	weights, gradients, mask bytes, thresholds, and the active count

idx	active	w	w	g	idx	active	w	w	g
0	1	+0.42	0.42	0.011	8	1	−0.28	0.28	0.150
1	0	−0.03	0.03	0.240	9	0	+0.02	0.02	0.280
2	1	+0.18	0.18	0.008	10	1	+0.47	0.47	0.007
3	0	−0.51	0.51	0.130	11	0	−0.09	0.09	0.220
4	1	+0.07	0.07	0.310	12	1	+0.21	0.21	0.009
5	0	+0.33	0.33	0.004	13	0	−0.38	0.38	0.170
6	1	−0.11	0.11	0.190	14	1	+0.15	0.15	0.006
7	0	+0.64	0.64	0.002	15	0	−0.05	0.05	0.260

NOTE — watch element 4

Its weight is 0.07 — the second smallest among the active ones, so RigL will prune it. Its gradient is 0.310 — the largest in the whole tensor, so RigL will immediately grow it back.

That is not a flaw in the example. It is the algorithm working: "this connection is barely contributing right now, but the loss surface says it would move a lot if it were switched on". Element 4 is the clearest single illustration of why the drop and grow criteria are different.

	b0	b1	b2	b3	b4	b5	b6	b7
byte 0	1	0	1	0	1	0	1	0
holds	0	1	2	3	4	5	6	7

Mask byte 0 = 0x55. Bit *i* of the byte holds element *i*, so the even elements are set. Byte 1 is identical, holding elements 8-15.

PART III

The Tensor-Level Primitives

The packed mask, the population count, the invariant, and the two threshold searches. Everything in this Part lives in `src/tensor/` and needs nothing above it.

III.1 The packed mask

Eight booleans in one byte

IN PLAIN ENGLISH — the waste being eliminated

A C `bool` occupies a whole byte to store one yes-or-no. Seven eighths is wasted. Usually nobody cares; here we do — the mask needs one boolean per weight, and it has to sit alongside the weights, the gradients and the momentum state in 320 KB.

For the HAR classifier the mask is 10,214 bits. Packed that is 1.3 KB; unpacked it would be 10 KB, as much as the entire `int8` model.

DEFINITION — bit packing

Boolean number i lives in bit $i \% 8$ of byte $i / 8$. Because 8 is a power of two the compiler emits $i >> 3$ and $i \& 7$, both single-cycle on the M7.

	b0	b1	b2	b3	b4	b5	b6	b7
byte 0	0	1	2	3	4	5	6	7
byte 1	8	9	10	11	12	13	14	15

Flat indices 0-15 in two bytes. Labels are the element number, not the bit value. Index 15 highlighted.

DEFINITION — padding bits

When the element count is not a multiple of 8, the last byte has bits corresponding to no real element. They must be ignored by anything that scans the whole array — and `tensorMaskCountActive` does exactly that.

TRACED — the tail mask, for $n = 11$

$\text{tail} = 11 \% 8 = 3$. We want the lowest 3 bits on.

$1u \ll 3 = 0b00001000 = 8$.

$8 - 1 = 0b00000111 = 7$.

Subtracting one from a lone set bit turns on every bit below it.

WARNING — the idiom breaks when tail is 0

If n is an exact multiple of 8 then $(1u \ll 0) - 1$ is 0, and the mask wipes out the whole last byte — eight real elements discarded. Every use must be guarded with `if (tail != 0)`. Both uses in `Tensor.c` are correctly guarded.

III.2 tensorBoolGet and tensorBoolSet

Reading and flipping one bit

src/tensor/Tensor.c — already present and correct

```
bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex) {
    if (tensor->quantization->type != BOOL) {
        PRINT_ERROR("tensorBoolGet called on non-BOOL tensor");
        exit(1);
    }
    size_t byteIndex = flatIndex / 8;
    size_t bitIndex = flatIndex % 8;
    return ((tensor->data[byteIndex] >> bitIndex) & 1u) != 0;
}
```

Variable	Type	What it refers to	Example
tensor	tensor_t const *	The mask being queried. const promises this function only reads	the layer mask
flatIndex	size_t	Which weight we are asking about, counted flat from 0	1200
byteIndex	size_t	Which byte of the packed array holds that bit	1200/8 = 150
bitIndex	size_t	Which bit inside that byte	1200%8 = 0

Line	Code	What it does, and why
170	bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex)	const on the pointer is a promise to the compiler and the reader that this function only reads. Any accidental write becomes a compile error rather than a bug.
171	if (tensor->quantization ->type != BOOL)	A type guard. Packed-bit indexing on a float tensor would silently read the wrong memory, so the function refuses rather than guessing.
175	size_t byteIndex = flatIndex / 8;	Integer division. Because 8 is a power of two and flatIndex is unsigned, the compiler emits a single right shift by 3 rather than a division.
176	size_t bitIndex = flatIndex % 8;	The remainder. Compiles to & 7, one instruction.
177	return ((tensor->data[byteIndex] >> bitIndex) & 1u) != 0;	Shift the wanted bit down to position 0, mask off everything else, compare against zero. Three single-cycle operations.
		<i>Why != 0 and not a bare return:</i> the AND yields 0 or 1 here, so it happens to be safe. But writing the comparison explicitly means the function keeps working if the return type ever changes, and costs nothing.

WORKED EXAMPLE — tensorBoolGet(mask, 6) on the running example

Mask byte 0 is 0x55 = 0b01010101. We want element 6.

```
byteIndex = 6 / 8 = 0
bitIndex  = 6 % 8 = 6
data[0] >> 6 = 0b01010101 >> 6 = 0b00000001
... & 1u = 1, and 1 != 0, so it returns true.
```

Check it against the table in II.7: element 6 is active. Now try element 7 — 0b01010101 >> 7 is 0, so false. Correct: the odd elements are inactive.

src/tensor/Tensor.c — already present and correct

```
void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value) {
    /* ... same type guard and index arithmetic ... */
    if (value) {
        tensor->data[byteIndex] |= (uint8_t)(1u << bitIndex);
    } else {
        tensor->data[byteIndex] &= (uint8_t)~(1u << bitIndex);
    }
}
```

Line	Code	What it does, and why
180	<pre>void tensorBoolSet(tensor_t *tensor, ..., bool value)</pre>	No const here — this one writes. Returns nothing; the whole effect is on the tensor's buffer.
187	<pre>tensor->data[byteIndex] = (uint8_t)(1u << bitIndex);</pre>	Turn the bit on. OR sets the one bit and leaves the other seven exactly as they were.
189	<pre>tensor->data[byteIndex] &= (uint8_t)~(1u << bitIndex);</pre>	Turn it off. ~(1u << bitIndex) is ones everywhere except the target, so ANDING forces that bit to 0 and preserves the rest.
		<i>The casts are not decoration:</i> 1u << bitIndex is an unsigned int, and ~ of it sets all 32 bits. The (uint8_t) makes the narrowing explicit rather than implicit, which is what -Wconversion wants to see.

Wanted	Correct operation	A wrong one, and its effect
turn bit on	byte = mask	byte = mask — wipes seven other weights
turn bit off	byte &= ~mask	byte &= mask — clears seven others, keeps this one
flip bit	byte ^= mask	byte = ~byte — flips all eight

WORKED EXAMPLE — tensorBoolSet(mask, 4, false) — pruning element 4

This is the call RigL makes when it prunes element 4 in the traced update. Byte 0 starts at 0x55.

	b0	b1	b2	b3	b4	b5	b6	b7
data[0]	1	0	1	0	1	0	1	0
1u<<4	0	0	0	0	1	0	0	0
~(1u<<4)	1	1	1	1	0	1	1	1
result	1	0	1	0	0	0	1	0

0x55 becomes 0x45. Bit 4 cleared (red); bits 0, 2 and 6 — elements 0, 2 and 6 — are untouched. That preservation is what the AND-with-complement buys.

IN PLAIN ENGLISH — read-modify-write, and why it is unavoidable

You cannot write one bit to memory. The smallest unit the hardware stores is a byte. So changing one bit means reading the whole byte, changing your copy, and writing it back.

The seven neighbouring bits ride through untouched, and the |= and &= ~ are what guarantee it. Use = instead and you silently corrupt seven other weights' mask entries.

ON THE STM32 — cost, and why it dictates where the mask may be used

About 6-8 cycles per call: a shift, a load, a shift, an AND, a store. RigL calls it once per weight in each of the drop and grow phases — for the whole HAR model roughly 160,000 cycles per RigL step, about 0.7 ms.

RigL runs every deltaT steps rather than every step, so at deltaT = 100 that is 7 microseconds amortised per training step. Entirely acceptable — and it is precisely why the mask must stay out of the matmul, where the same 6-8 cycles would be paid on every multiply-accumulate. Part VI.4.

III.3 tensorMaskCountActive

Population count, with the tail handled

src/tensor/Tensor.c — already present and correct

```
size_t tensorMaskCountActive(tensor_t const *mask) {
    /* ... NULL and BOOL type guards ... */
    size_t n = calcNumberOfElementsByShape(mask->shape);
    size_t whole = n / 8u;
    unsigned tail = (unsigned)(n % 8u);
    size_t total = 0;
    for (size_t i = 0; i < whole; i++) {
        total += (size_t)__builtin_popcount(mask->data[i]);
    }
    if (tail != 0u) {
        uint8_t m = (uint8_t)((1u << tail) - 1u);
        total += (size_t)__builtin_popcount(mask->data[whole] & m);
    }
    return total;
}
```

Variable	Type	What it refers to	Example
n	size_t	Total weights this mask covers	6,144 for conv3
whole	size_t	How many complete bytes there are	768
tail	unsigned	How many real bits are in the final partial byte	0 here
total	size_t	Running count of active weights	394
m	uint8_t	Tail mask: ones for real bits, zeros for padding	0b00000111

Line	Code	What it does, and why
27	if (mask == NULL mask->quantization->type != B00L)	Two checks in one. short-circuits, so if mask is NULL the second test never runs — which is essential, because evaluating it would dereference NULL. The order is load-bearing.
31	size_t n = calcNumberOf ElementsByShape(mask->shape);	How many weights the mask covers. Note it asks the <i>shape</i> , not the byte buffer — the buffer is eight times smaller and would give the wrong answer.
32	size_t whole = n / 8u;	How many bytes are completely full of real bits.
33	unsigned tail = (unsigned)(n % 8u);	How many real bits are left over. Zero when n divides evenly, which it does for every HAR layer.
35-37	for (...) total += popcount(data[i]);	Count set bits in every complete byte. No masking needed — every bit in these bytes is a real weight.
38	if (tail != 0u) {	The guard that matters. Without it the next line computes $(1u \ll 0) - 1 = 0$ and masks away the entire final byte, discarding eight real elements.
39	uint8_t m = (uint8_t) ((1u << tail) - 1u);	Ones for the real bits, zeros for the padding. Subtracting one from a lone set bit turns on every bit below it.
40	total += popcount(mask->data[whole] & m);	Count only the real bits of the partial byte. The AND is what excludes padding bits that happen to be 1.

WORKED EXAMPLE — `tensorMaskCountActive` on the running example

`n = 16`, so `whole = 2` and `tail = 0`. The tail branch is skipped entirely.

```
popcount(0x55) = popcount(0b01010101) = 4
```

```
popcount(0x55) again for byte 1 = 4
```

```
total = 8
```

Eight active out of sixteen — 50% sparsity, matching the setup in II.7. Now suppose `n` were 11 instead: `whole = 1`, `tail = 3`, so byte 1 would be ANDed with `0b00000111` before counting — giving `popcount(0x55 & 0x07) = popcount(0b00000101) = 2`, for a total of 6 rather than 8. The five padding bits are correctly excluded.

DEFINITION — population count

How many bits of a value are 1. For a packed mask this is exactly the number of active weights — the quantity RigL derives its drop count from.

WARNING — why the tail mask is load-bearing

`tensorMaskInitDense` fills the buffer with `0xFF`, so padding bits *would* be 1 if not masked off. An inflated `numActive` means `k = alpha × numActive` is too large, and RigL drops too many real weights — compounding across every update with nothing reporting it.

For the HAR layer sizes `n` happens to be a multiple of 8, so the tail branch never runs and its correctness is untested by the workload. Write a unit test with an awkward `n`.

ON THE STM32 — `__builtin_popcount` on ARMv7-M

x86 has `POPCNT`; AArch64 has `CNT`. ARMv7-M — the Cortex-M7 instruction set — has **neither**. The builtin still compiles, but expands to a software routine or table lookup, typically 10-15 cycles.

It is also called here on a `uint8_t`, which promotes to `int` — so you pay for a 32-bit popcount to count 8 bits. Processing four bytes at a time as a `uint32_t` would be about four times faster. For the HAR model the whole mask is 1,277 bytes, so this is not urgent; on a larger model it would be.

III.4 tensorMaskInitDense

What it is, what it is not, and the fix

IN PLAIN ENGLISH — the name is misleading

It does **not** mean "train dense first, then prune" — that is dense-to-sparse training, which RigL is not, and which would not help anyway since the dense model is what you are trying to avoid.

"Dense" here is the *identity state of a mask*: nothing pruned yet. The function is an initialiser. It does four jobs, and only one of them is the fill.

ALGORITHM — what it actually does

1. `initBoolQuantization` — build the BOOL descriptor, so the type guards in `tensorBoolGet/Set` will pass.
2. `setTensorValues` — wire the mask struct to its storage and shape.
3. `memset(storage, 0xFF, nBytes)` — put the buffer in a *defined* state.
4. Clear the padding bits in the final byte.

CONCEPT — steps 3 and 4 are the load-bearing ones

A freshly allocated buffer holds arbitrary bytes. Skip the `memset` and your initial topology is whatever garbage was in RAM — a random mask you never chose, at a sparsity you cannot report.

Step 4 establishes the invariant that padding bits are 0, which `tensorMaskCountActive` depends on. There is also a cheap postcondition: after this call, `tensorMaskCountActive(mask)` must equal exactly `n`. One line to `assert`, and it catches any sizing mistake immediately.

src/tensor/Tensor.c — line 18, as it stands

```
size_t n = calcNumberOfElementsByShape(shape);
size_t nBytes = TENSOR_MAX_BYTES;          /* <-- no such identifier */
memset(storage, 0xFF, nBytes);
unsigned tail = (unsigned) (n % 8);
if (tail != 0) {
    storage[nBytes - 1] &= (uint8_t)((1u << tail) - 1);
}
```

src/tensor/Tensor.c — FIXED

```

void tensorMaskInitDense(tensor_t *mask, uint8_t *storage,
                        shape_t *shape, quantization_t *boolQ) {
    initBoolQuantization(boolQ);
    setTensorValues(mask, storage, shape, boolQ, NULL);

    size_t n      = calcNumberOfElementsByShape(shape);
    size_t nBytes = (n + 7u) / 8u;      /* packed: one bit per element */
    if (nBytes == 0u) {
        return;                        /* empty mask; nothing to fill */
    }

    memset(storage, 0xFF, nBytes);      /* all ones: nothing pruned yet */

    unsigned tail = (unsigned)(n % 8u);
    if (tail != 0u) {
        /* Clear the padding bits so tensorMaskCountActive cannot count
         * them as active weights. */
        storage[nBytes - 1u] &= (uint8_t)((1u << tail) - 1u);
    }
}

```

Variable	Type	What it refers to	Example
mask	tensor_t *	The mask tensor being set up — struct only, no storage yet	an empty tensor_t
storage	uint8_t *	The caller-allocated byte buffer the mask will use	1,277 bytes for HAR
shape	shape_t *	The shape of the <i>weights</i> , not of the buffer	[6144] for conv3
boolQ	quantization_t *	A descriptor this function fills in as BOOL	set by initBoolQuantization
n	size_t	How many weights, hence how many bits	6,144
nBytes	size_t	How many bytes those bits need, rounded up	768
tail	unsigned	Real bits in the last byte; 0 means it is full	0

Line	Code	What it does, and why
	initBoolQuantization(boolQ);	Fill in the quantisation descriptor as BOOL. Without this the type guards in tensorBoolGet and Set would reject every call.
	setTensorValues(mask, storage, shape, boolQ, NULL);	Wire the struct to its buffer, shape and descriptor. The trailing NULL is the sparsity pointer — a mask does not itself have a mask.
	size_t n = calcNumberOfElementsByShape(shape);	How many weights. This is the <i>weight</i> count, and the buffer will be eight times smaller.
	size_t nBytes = (n + 7u) / 8u;	The fix. Ceiling division: adding 7 pushes any non-zero remainder past the next multiple of 8 before the truncating division. If the remainder was already 0, adding 7 is not enough to reach the next multiple, so the answer is unchanged.
	if (nBytes == 0u) { return; }	Guard added. With n == 0, nBytes is 0 and the indexing below would compute storage[SIZE_MAX] — unsigned wraparound.
	memset(storage, 0xFF, nBytes);	Every bit to 1: nothing pruned. One library call rather than 6,144 individual tensorBoolSet calls, and the difference is roughly a thousandfold.
	unsigned tail = (unsigned)(n % 8u);	How many bits of the last byte are real elements.

Line	Code	What it does, and why
	<code>storage[nBytes - 1u] &= (uint8_t)((1u << tail) - 1u);</code>	Clear the padding bits, so <code>tensorMaskCountActive</code> cannot count them as active weights later. Guarded by <code>tail != 0</code> for the reason in III.1.

WORKED EXAMPLE — `tensorMaskInitDense` on a 16-element layer, and on an 11-element one

n = 16: `nBytes = (16 + 7) / 8 = 2`. `memset` writes `0xFF 0xFF`. `tail = 16 % 8 = 0`, so the masking step is skipped. Result: all 16 bits set, `countActive` returns 16.

n = 11: `nBytes = (11 + 7) / 8 = 2` — the same two bytes, because 11 bits still need 2 bytes. `memset` writes `0xFF 0xFF`, which sets 16 bits when only 11 are real. `tail = 11 % 8 = 3`, so byte 1 is ANDed with `(1u << 3) - 1 = 0b00000111`, leaving `0xFF 0x07`.

Without that last step `countActive` would return 16 for an 11-element layer, and `RigL` would compute `k` from an inflated count and prune too many real weights on every update.

	b0	b1	b2	b3	b4	b5	b6	b7
after memset	1	1	1	1	1	1	1	1
tail mask	1	1	1	0	0	0	0	0
byte 1 after	1	1	1	0	0	0	0	0

Byte 1 for `n = 11`. Three real elements survive (green); five padding bits are cleared (yellow) so nothing later counts them as weights.

NOTE — the added zero guard

With `n == 0`, `nBytes` is 0 and `storage[nBytes - 1u]` indexes `storage[SIZE_MAX]` — unsigned wraparound. The `tail != 0` check happens to prevent it today, since `0 % 8` is 0, but relying on that is fragile.

WARNING — this is only half of setup

After this call the mask is all-ones: *nothing* is pruned. `RigL` is sparse-to-sparse, so the network must be at target sparsity before step 1. That is `rigLInitSparsityERK`, and it is Part IV.

III.5 tensorApplyMaskZero and the invariant

The rule the matmul depends on

CONCEPT — an inactive weight holds exactly 0.0

This one sentence is the design decision the whole sparse path rests on. It is enforced in one place — this function — and nowhere else.

If it holds, the matmul needs no mask at all. Multiplying by a stored zero contributes zero, which is exactly what skipping the weight would have contributed. The arithmetic is identical, and the inner loop stays a tight contiguous branch-free sequence of MACs.

The mask remains authoritative for RigL's bookkeeping; it simply never enters the hot path. Part VI.4 covers what happens if you try to put it there.

src/tensor/Tensor.c — lines 50 and 68, as they stand

```
switch (values->quantization->type){          /* line 50: undeclared */
    /* ... */
    default:
        PRINT_ERROR("... dtype %d unsupported ...",
                    (int) values->quantization->type); /* line 68 */
        exit(1);
}
```

src/tensor/Tensor.c — FIXED, with the type restriction documented

```

void tensorApplyMaskZero(tensor_t *value, tensor_t const *mask) {
    if (value == NULL || mask == NULL) {
        PRINT_ERROR("tensorApplyMaskZero: NULL argument");
        exit(1);
    }
    size_t n = calcNumberOfElementsByTensor(value);
    if (n != calcNumberOfElementsByTensor((tensor_t *)mask)) {
        PRINT_ERROR("tensorApplyMaskZero: %zu value vs %zu mask bits", n,
            calcNumberOfElementsByTensor((tensor_t *)mask));
        exit(1);
    }

    switch (value->quantization->type) {
    case FLOAT32:
        for (size_t i = 0; i < n; i++) {
            if (!tensorBoolGet(mask, i)) {
                float zero = 0.0f;
                memcpy(&value->data[i * sizeof(float)], &zero, sizeof(float));
            }
        }
        return;

    case SYM_INT32:
        for (size_t i = 0; i < n; i++) {
            if (!tensorBoolGet(mask, i)) {
                int32_t zero = 0;
                memcpy(&value->data[i * sizeof(int32_t)], &zero, sizeof(int32_t));
            }
        }
        return;

    default:
        /* ASYM: real 0.0 is stored as zeroPoint, which moves on recalibration,
         * so a "zeroed" weight would silently come back to life.
         * SYM: sub-byte packing means a fixed byte stride cannot address
         * element i without clobbering its neighbours. */
        PRINT_ERROR("tensorApplyMaskZero: dtype %d unsupported (FLOAT32 / SYM_INT32)",
            (int)value->quantization->type);
        exit(1);
    }
}

```

Variable	Type	What it refers to	Example
value	tensor_t *	The weight tensor being zeroed. Note the singular name — the two typos were plural	the layer weights
mask	tensor_t const *	Which entries survive. const: this function never modifies the topology	the packed mask
n	size_t	How many elements, checked to match on both sides	6,144
zero	float / int32_t	A local holding the bit pattern to write	0.0f

Line	Code	What it does, and why
	if (value == NULL mask == NULL)	Added. The original dereferenced both without checking; on the F746ZG address 0 is mapped, so a NULL here would corrupt rather than fault.
	size_t n = calcNumberOf ElementsByTensor(value);	How many weights to walk.
	if (n != calcNumberOf ElementsByTensor(mask))	The shape agreement check. A mask of the wrong length would silently zero the wrong weights, so this is worth stopping for.

Line	Code	What it does, and why
	<code>switch (value->quantization ->type)</code>	The fix. Was <code>values-></code> , which is not declared anywhere in scope. One character.
	<code>case FLOAT32:</code>	The dense-training path. Four bytes per element.
	<code>if (!tensorBoolGet(mask, i))</code>	Only touch elements whose bit is clear. Active weights are left exactly as the optimizer set them.
	<code>memcpy(&value->data[i * sizeof(float)], &zero, sizeof(float));</code>	Write a real 0.0. <code>memcpy</code> rather than a pointer cast — aliasing-safe, alignment-safe, and the compiler turns a four-byte copy into a single store anyway.
	<code>case SYM_INT32:</code>	The quantised path. Identical shape, <code>int32_t</code> instead of <code>float</code> , because under SYM a stored 0 decodes to real 0.0 for any scale.
	<code>default: PRINT_ERROR(...) exit(1);</code>	Refuse everything else. The comment above it records <i>why</i> , which is the part a future reader will need. See the box below.

WORKED EXAMPLE — `tensorApplyMaskZero` after the traced drop

In the traced update `RigL` prunes elements 4, 6 and 14, so the mask becomes `0x05 0x15`. This function then walks all 16 elements.

idx	mask bit	w before	Action	w after
0	1	+0.42	skipped — still active	+0.42
2	1	+0.18	skipped	+0.18
4	0	+0.07	memcpy 0.0f	0.00
6	0	−0.11	memcpy 0.0f	0.00
8	1	−0.28	skipped	−0.28
10	1	+0.47	skipped	+0.47
12	1	+0.21	skipped	+0.21
14	0	+0.15	memcpy 0.0f	0.00
1, 3, 5, ...	0	various	already 0.0 from earlier steps	0.00

NOTE — the odd indices were already zero

They were pruned at initialisation and have been held at 0.0 by the per-step reapplication ever since. The function writes 0.0 over 0.0 for them, which is harmless and cheaper than tracking which ones changed.

That is why the per-step cost is a full pass rather than a delta: simplicity, and the pass is $O(n)$ with a tiny constant.

DEFINITION — why the default branch refuses ASYM and SYM

Two different reasons, both correct, and worth keeping straight.

ASYM is a semantic problem. Reconstruction is $\text{real} = (q - \text{zeroPoint}) \times \text{scale}$, so storing 0 gives $-\text{zeroPoint} \times \text{scale}$, not zero. And `zeroPoint` changes whenever the tensor is recalibrated — so every pruned weight would drift back to a small non-zero value and rejoin the computation.

SYM is a mechanical problem. It is the sub-byte type; `qBits` may be 4 or 2, so element `i` shares a byte with its neighbours and a whole-unit store at a fixed stride would clobber them. It needs read-modify-write through `writeByte`.

NOTE — both halves are worked through in full next

III.5.1 sorts out the six quantisation types first — in particular the difference between SYM and SYM_INT32, whose config structs look almost identical and are not. III.5.2, III.5.3 and III.5.4 then take the three refusals in turn: ASYM for a semantic reason, SYM for a layout one, and INT32 for neither.

NOTE — a consequence for FQT worth planning around

If your quantised weights end up stored as narrow SYM rather than SYM_INT32, this function cannot zero them and the invariant cannot be enforced. A third branch using the bit-field writers would be needed. Worth deciding which storage type the FQT path uses before you get there.

III.5.1 The six quantisation types, and which are weight types

Because two of them look almost identical and are not

IN PLAIN ENGLISH — why this section exists

tensorApplyMaskZero accepts two types and refuses four. Each refusal has a different reason, and one of the pairs — SYM and SYM_INT32 — has configuration structs so similar that mixing them up is easy.

Sorting out what each type actually is makes the next three sections short.

src/tensor/include/Quantization.h — the whole set

```
typedef enum qtype { INT32, FLOAT32, SYM_INT32, SYM, ASYM, BOOL } qtype_t;
```

Type	Reconstruction	Bytes per element	Used for	In applyMaskZero
FLOAT32	the value itself	4	unquantised weights, activations	accepted
SYM_INT32	$q \times \text{scale}$	4, always	gradients, loss, accumulators	accepted
SYM	$q \times \text{scale}$	packed at qBits	narrow quantised storage	refused — III.5.3
ASYM	$(q - \text{zeroPoint}) \times \text{scale}$	packed at qBits	post-ReLU activations	refused — III.5.2
INT32	the value itself	4	labels, argmax buffers	refused — III.5.4
BOOL	one bit	packed, 8 per byte	the mask itself	refused — not a value type

SYM and SYM_INT32: the configs look alike

src/tensor/include/Quantization.h — near-identical, and not the same thing

```
typedef struct symInt32QConfig {
    float      scale;
    roundingMode_t roundingMode;
    uint8_t     qMaxBits;      /* <-- note the name */
} symInt32QConfig_t;

typedef struct symQConfig {
    float      scale;
    uint8_t     qBits;         /* <-- and this one */
    roundingMode_t roundingMode;
} symQConfig_t;
```

WARNING — the two width fields do completely different jobs

Both structs have a scale, both have a roundingMode, and both have a uint8_t width. It is natural to read them as the same idea at two precisions. They are not.

qBits is a **storage layout**. qMaxBits is a **numeric range contract**. Changing the first changes how many bytes the tensor occupies; changing the second does not change the footprint at all.

DEFINITION — SYM_INT32 — symmetric, stored in four bytes, always

The mantissa lives in a full `int32_t` whatever `qMaxBits` says. You can see it in the sizing functions:

```
calcBitsPerElement: case SYM_INT32: return sizeof(int32_t) * 8;  
calcNumberOfBytesForData: case SYM_INT32: return n * sizeof(int32_t);
```

Neither consults `qMaxBits`. It is 32 bits per element, unconditionally.

DEFINITION — SYM — symmetric, packed to qBits

Here the width is the layout:

```
calcBitsPerElement: case SYM: return symQC->qBits;  
calcNumberOfBytesForData: case SYM: return (bits * n + 7) / 8;
```

That $(bits \times n + 7) / 8$ is the packing: the whole tensor is squeezed to the bit, which is why elements end up sharing bytes.

WORKED EXAMPLE — the same layer, the same 8-bit range, four times the memory

Conv3 has 6,144 elements. Store it as SYM_INT32 with `qMaxBits = 8`, or as SYM with `qBits = 8`. Same numeric precision either way.

Storage type	width setting	bytes	KiB
SYM_INT32	<code>qMaxBits = 8</code>	24,576	24.0
SYM_INT32	<code>qMaxBits = 12</code>	24,576	24.0
SYM_INT32	<code>qMaxBits = 16</code>	24,576	24.0
SYM	<code>qBits = 8</code>	6,144	6.0
SYM	<code>qBits = 4</code>	3,072	3.0
SYM	<code>qBits = 2</code>	1,536	1.5

CONCEPT — four times the memory, and there is a reason for it

At the same 8-bit precision SYM_INT32 costs 24 KiB and SYM costs 6 KiB. If SYM is four times cheaper, why does SYM_INT32 exist?

Because it is the **accumulation** type. Gradients are not written once and read; they are *added into* repeatedly as a batch is processed. Accumulating into a packed sub-byte buffer would mean a read-modify-write per accumulation — the same dance as III.5.3, on the hot path rather than once per rewiring.

Keeping four bytes makes an accumulation a plain add. `qMaxBits` then bounds the *values* so the running sum cannot overflow the `int32`. The header says it directly: *"SYM_INT32 grads stay 4 B/element at any width; sub-float memory saving comes only from packed SYM/ASYM grad storage."*

	SYM_INT32	SYM
what the width field means	the largest value allowed	how many bits each element occupies
does the width change the footprint?	no	yes

	SYM_INT32	SYM
elements share a byte?	never	when qBits < 8
a fixed-stride store works?	yes	no
designed for	accumulation — gradients, loss	compact storage
default width	ODT_SYM_OPERAND_QMAXBITS = 12	caller supplies qBits

NOTE — and that is exactly why one is accepted and the other is not

tensorApplyMaskZero writes a zero with a fixed-stride memcpy. That is correct for SYM_INT32, which is always four aligned bytes, and wrong for SYM, which may put two or four elements in one byte.

The refusal is not about precision or about symmetry — both types are symmetric and a stored 0 is real 0.0 in both. It is purely about **layout**.

III.5.2 Why ASYM is refused

The arithmetic, worked

IN PLAIN ENGLISH — the reconstruction formula is the whole problem

Under asymmetric quantisation the stored integer is not the value. It is an *offset from a reference point*:

$$\text{real} = (q - \text{zeroPoint}) \times \text{scale}$$

So to store real 0.0 you need $q == \text{zeroPoint}$, not $q == 0$. Writing 0 gives you $-\text{zeroPoint} \times \text{scale}$, which is not zero and is usually not small.

WORKED EXAMPLE — what writing 0 actually produces

Take a post-ReLU activation range of $[0, 6.0]$, quantised to int8.

$$\text{scale} = 6.0 / 255 = 0.023529$$

$$\text{zeroPoint} = -128 \text{ (so that the bottom of the range maps to real 0.0)}$$

stored q	real = (q - zeroPoint) × scale	value	verdict
zeroPoint = -128	$(-128 - -128) \times 0.023529$	0.0000	correct zero
0	$(0 - -128) \times 0.023529$	3.0118	halfway up the range

IN PLAIN ENGLISH — so the obvious fix is to write zeroPoint instead

And it does not hold either, which is the more interesting half of the problem.

A tensor is recalibrated whenever its observed range moves — which for activations is routine, and for weights happens as training progresses. Recalibration changes both `scale` and `zeroPoint`.

TRACED — a stored zero drifting after one recalibration

The same tensor, before and after its range widens.

	range	scale	zeroPoint
before	$[-1.0, 3.0]$	0.015686	-64
after	$[-2.0, 3.0]$	0.019608	-26

WARNING — the weight comes back to life, and nothing reports it

A weight stored as $q = -64$ meant exactly 0.0 under the old calibration. Under the new one it decodes to

$$(-64 - -26) \times 0.019608 = \mathbf{-0.7451}$$

Every pruned weight in the layer silently becomes non-zero and rejoins the computation. The mask still says they are inactive. The active count is still correct. The matmul — which trusts the stored value and never reads the mask, by the design in VII.4 — includes all of them.

What to do about it

CONCEPT — scope ASYM out, and treat the restriction as the design

ASYM exists for post-ReLU activations, where every value is non-negative and half a symmetric range would be wasted. Weights are roughly zero-mean, so SYM is the natural choice for them anyway.

RigL prunes **weights**. It never prunes activations. So the two need never meet, and the default: branch refusing ASYM is not a gap in the implementation — it is a correct statement of scope. Keep it, and document why.

If you did need ASYM weights	What it costs
Re-apply after every recalibration. Write <code>q = zeroPoint</code> , and make the calibration path call <code>tensorApplyMaskZero</code> afterwards.	Correct, but it couples the quantiser to the mask: calibration now has to know sparsity exists.
Pin the zero point. Force calibration to hold <code>zeroPoint</code> fixed for masked tensors.	Removes the drift entirely. Costs some range utilisation, since the quantiser can no longer place zero optimally.

NOTE — and a consequence for the FQT path worth deciding early

If your quantised weights end up stored as ASYM rather than SYM, the invariant cannot be maintained at all without one of the two workarounds above. Choose the weight storage type before building the FQT path, not after.

III.5.3 Supporting sub-byte SYM

A mechanical problem, and a fix that is tested

IN PLAIN ENGLISH — SYM is semantically fine — only the write mechanics break

Under SYM the reconstruction is $\text{real} = q \times \text{scale}$, so a stored 0 is real 0.0 for any scale, exactly as with FLOAT32. There is no recalibration problem and no drift.

The difficulty is purely that SYM is the **sub-byte** type. qBits may be 4 or 2, so elements share bytes with their neighbours.

DEFINITION — where qBits comes from

It is a plain `uint8_t` field in `symQConfig_t`, set by whoever calls `initSymQConfig(qBits, roundingMode, &cfg)`. Nothing derives it — you choose it per tensor when you build the quantisation descriptor, through `quantizationInitSym(uint8_t qBits, roundingMode_t)`.

So different layers can carry different widths in the same model, and the code that touches packed data must read qBits from the config rather than assume a value.

NOTE — SYM and SYM_INT32 differ in what their width knob controls

`SYM_INT32` has `qMaxBits`, which bounds the *numeric range* for accumulator soundness — it is **always 4 bytes per element** whatever the value. The header says so directly: *"SYM_INT32 grads stay 4 B/element at any width; sub-float memory saving comes only from packed SYM/ASYM grad storage."*

`SYM` has `qBits`, which is the *storage width*, and it is packed. That is exactly why `tensorApplyMaskZero` handles the first with a plain 4-byte store and cannot handle the second the same way.

qBits	elements per byte	element i lives in	a plain byte store would
8	1	byte i	be correct
4	2	half of byte i/2	clobber 1 neighbour
2	4	a quarter of byte i/4	clobber 3 neighbours

DEFECT — the current code uses a fixed four-byte stride

The `SYM_INT32` branch writes `memcpy(&value->data[i * sizeof(int32_t)], &zero, 4)`. That is right for a 4-byte type and wrong at every sub-byte width — it would write four whole bytes where the element occupies half of one.

[src/tensor/Tensor.c](#) — NEW — the sub-byte helper

```

/* Clear the qBits-wide field belonging to element i, leaving every other
 * bit in the touched byte(s) exactly as it was. Under SYM the stored
 * value 0 IS real 0.0 for any scale, so zeroing the bits is semantically
 * correct -- only the mechanics of a sub-byte write need care. */
static void symZeroElement(uint8_t *data, size_t i, size_t qBits) {
    size_t startBit = i * qBits;
    size_t endBit = startBit + qBits;
    size_t firstByte = startBit / 8u;
    size_t lastByte = (endBit - 1u) / 8u;

    if (firstByte == lastByte) { /* fits inside one byte */
        uint8_t s = (uint8_t)(startBit % 8u);
        data[firstByte] = writeByte(data[firstByte], 0u, s,
                                     (uint8_t)(s + qBits));
    } else { /* straddles two bytes */
        uint8_t s = (uint8_t)(startBit % 8u);
        data[firstByte] = writeByte(data[firstByte], 0u, s, 8u);
        uint8_t e = (uint8_t)(endBit % 8u);
        data[lastByte] = writeByte(data[lastByte], 0u, 0u, e);
    }
}

```

Variable	Type	What it refers to	Example
data	uint8_t *	The packed value buffer	the weight bytes
i	size_t	Which element to zero — the same index the mask uses	1200
qBits	size_t	How many bits each element occupies	4
startBit	size_t	Where element i begins, counted in bits from the buffer start	4800
firstByte	size_t	The byte holding the start of the field	600
lastByte	size_t	The byte holding the end. Equal to firstByte unless the field straddles	600

Line	Code	What it does, and why
	size_t startBit = i * qBits;	Bit position rather than byte position. This is the whole difference from the byte-aligned case — the unit of addressing changes.
	size_t lastByte = (endBit - 1u) / 8u;	endBit is exclusive, so the last <i>occupied</i> bit is endBit - 1. Using endBit / 8 would name one byte too many whenever the field ends exactly on a boundary.
	if (firstByte == lastByte)	The common case. A field straddles only when qBits does not divide 8, so at 8, 4 and 2 bits this branch always wins.
	data[firstByte] = writeByte(data[firstByte], 0u, s, (uint8_t)(s + qBits));	Read-modify-write through the project's existing bit-field writer. It clears exactly the named range and preserves every other bit in the byte — which is the whole point.
	data[firstByte] = writeByte(..., s, 8u);	The straddling case, first half: clear from the field's start to the top of the byte.
	data[lastByte] = writeByte(..., 0u, e);	And the second half: clear from the bottom of the next byte to the field's end. A field of at most 8 bits can never span more than two bytes, so two writes always suffice.

Verified exhaustively, not just spot-checked

IN PLAIN ENGLISH — what was tested

Every element position at every width from 1 to 8 bits — 128 cases — starting from an all-ones buffer and checking bit by bit that exactly the target field cleared and nothing else moved.

qBits	before	after zeroing	which element	result
4	AB CD	0B CD	element 1	neighbour intact
2	FF	CF	element 2	only bits 4-5
3	FF FF	3F FE	element 2	straddles cleanly
8	7F 42	00 42	element 0	neighbour intact

cases passed  128

cases failed  0

qBits 1 through 8, sixteen element positions each, compared bit by bit against a reference built independently.

WARNING — the straddling branch is not hypothetical — your test suite needs it

A field crosses a byte boundary only when qBits does not divide 8, so 8, 4 and 2 never straddle. It would be easy to conclude the second branch is dead code.

It is not. Grepping every `initSymQConfig` call site in the repository turns up three distinct widths in use — **4, 6 and 8** — and **6-bit appears at least seven times** in `test/unit/tensor/UnitTestTensorConversion.c`.

Six does not divide eight. A 6-bit field starting at element 2 spans bits 12 to 17, crossing the byte-1 / byte-2 boundary. Without the second branch that write would silently corrupt a neighbouring weight in a configuration the project already exercises.

qBits	divides 8?	fields straddle?	in this repo?
8	yes	no	yes — the FQT target
4	yes	no	yes — unit tests
6	no	yes	yes — <code>UnitTestTensorConversion.c</code> , 7+ sites
2	yes	no	not currently

WARNING — one assumption to assert when you wire this up

The mask is one bit per *element*, not per byte — `calcNumberOfBytesForData` sizes `BOOL` as $(n + 7) / 8$ regardless of how wide the value type is. So mask bit *i* and value element *i* correspond however the values are packed.

That is true today and the whole scheme depends on it, so it is worth an assertion at the call site rather than a comment.

src/tensor/Tensor.c — the new branch in `tensorApplyMaskZero`

```

case SYM: {
    symQConfig_t *q = value->quantization->qConfig;
    size_t qBits = q->qBits;
    if (qBits == 8u) { /* byte aligned: fast path */
        for (size_t i = 0; i < n; i++) {
            if (!tensorBoolGet(mask, i)) {
                value->data[i] = 0u;
            }
        }
    } else { /* sub-byte: read-modify-write */
        for (size_t i = 0; i < n; i++) {
            if (!tensorBoolGet(mask, i)) {
                symZeroElement(value->data, i, qBits);
            }
        }
    }
    return;
}
}

```

TIP — why the qBits == 8 fast path is worth the extra branch

At 8 bits the field is exactly one byte, so a direct store is correct and costs one instruction. The general path costs a mask build, a load, an AND, an OR and a store — five to seven times more, on a loop that runs once per weight per training step.

And 8 bits is the case FQT actually uses. The general path is there for widths you may experiment with later; the fast path is there for the one you will ship.

III.5.4 Why INT32 is refused

A different kind of reason from the other two

IN PLAIN ENGLISH — this one would work

ASYM cannot be supported without changing the calibration path, and SYM needs a new write routine. INT32 needs neither. It would work today, with the code already in the function.

It is refused for a reason that has nothing to do with capability.

	INT32	SYM_INT32
bytes per element	4	4
byte aligned?	yes	yes
has a scale?	no — qConfig is NULL	yes
does stored 0 mean real 0?	yes, trivially	yes, for any scale
would memcpy(&data[i*4], &zero, 4) be right?	yes	yes

DEFINITION — INT32 is the unquantised integer type

initInt32Quantization sets qConfig = NULL — there is no scale, no zero point and no rounding mode. The stored number is the number.

The source comment says why: *"Because these are native data types that do not undergo compression or scaling math, they do not need structural algorithm rules."*

The real reason: no weight tensor is INT32

IN PLAIN ENGLISH — what the type is actually used for

Grepping every place INT32 quantisation is created gives three call sites, and none of them is a weight.

Site	What it creates	Could RigL ever prune it?
NPYLoaderApi.c:167	class labels loaded from a .npy with dtype INT_32	no — these are the training targets
Pool1dApi.c:101	the argmax buffer for MaxPool1d — which input position won	no — bookkeeping, not parameters
TensorApi.c:145	gradInitInt32 — a gradient shaped like a parameter	see the warning below

CONCEPT — and weights are FLOAT32 or SYM_INT32, never INT32

The layer kernels come in exactly two flavours — conv1dCalcWeightGradsFloat32 and conv1dCalcWeightGradsSymInt32. There is no ...Int32 variant, because a weight without a scale is not a quantised weight; it is just an integer.

So the three things that are INT32 in this codebase are your HAR class labels, pooling bookkeeping, and a gradient-init helper. RigL touches none of them.

WARNING — which makes the refusal a useful type check, not a missing feature

If `tensorApplyMaskZero` is ever handed an INT32 tensor, something upstream has gone wrong — most likely a label or argmax tensor passed where weights belong.

Silently succeeding would be the worse outcome by far. Zeroing a **label** tensor wherever a mask bit is clear would corrupt your training targets, the loss would be computed against nonsense, and nothing anywhere would report it. Stopping is right.

What should change: the message, not the behaviour

src/tensor/Tensor.c — reads like an incomplete implementation

```
PRINT_ERROR("tensorApplyMaskZero: dtype %d unsupported"
            " (FLOAT32 / SYM_INT32)",
            (int)value->quantization->type);
exit(1);
```

src/tensor/Tensor.c — says what the check is for

```
default:
/* INT32 would work mechanically -- 4 bytes, no scale, stored 0 is
 * real 0.0. It is refused because no weight tensor is INT32: that
 * type carries labels (NPYLoader), MaxPool argmax buffers, and
 * gradInitInt32. Reaching here with INT32 means a non-weight tensor
 * was passed, and silently zeroing a label tensor would be far worse
 * than stopping.
 * ASYM is refused for a semantic reason (III.5.2) and SYM for a
 * layout reason (III.5.3). */
PRINT_ERROR("tensorApplyMaskZero: dtype %d is not a weight type"
            " (expected FLOAT32 or SYM_INT32)",
            (int)value->quantization->type);
exit(1);
```

TIP — "unsupported" and "not a weight type" send opposite signals

Unsupported tells the next reader the implementation is incomplete and invites them to complete it — which for INT32 would mean adding a case that lets label corruption through silently.

Not a weight type tells them the check is deliberate and the caller is at fault. Same behaviour, opposite instruction to whoever reads it next.

DEFECT — one path where INT32 could actually reach RigL

`gradInitInt32` creates a gradient tensor with plain INT32 quantisation — no scale — shaped like a parameter. Nothing currently uses it for a sparsifiable layer, but nothing prevents it either.

If it ever did, `rigLKthLargestInactive` would read those gradients with `readBytesAsFloat`, reinterpreting int32 bit patterns as IEEE-754 floats. The integer 1 becomes 1.4×10^{-45} ; the integer 1,065,353,216 becomes 1.0. The grow phase would rank complete nonsense, and every assertion in Part IX would still pass.

Worth a guard in `rigLLayerSparseParts`: assert that the gradient tensor's type matches the weight tensor's. Two lines, and it closes the one route by which this could happen.

src/layer/RigLStep.c — the guard

```

/* in rigLLayerSparseParts, after the mask checks */
if (w->grad->quantization == NULL ||
    w->grad->quantization->type != w->param->quantization->type) {
    PRINT_ERROR("rigL: weight dtype %d but gradient dtype %d --"
               " the grow phase would misread the gradients",
               (int)w->param->quantization->type,
               (int)(w->grad->quantization ?
                    w->grad->quantization->type : -1));
    return false;
}

```


III.6 Finding a cut-off without writing everything down

The problem this section solves, from scratch

IN PLAIN ENGLISH — start with something that has nothing to do with code

Imagine you are a teacher with **6,144 exam papers** in a pile. You have been told to fail the weakest 300 students, and you need to announce a *pass mark* — a single number, so that anyone below it fails.

You do not need to know who those 300 students are. You do not need the papers in order. You need **one number**. The question of this section is how to find it without needing a desk big enough to lay out 6,144 papers.

DEFINITION — the same question, in the code

RigL wants to switch off the k weights with the smallest magnitude. To do that it needs one number — a **threshold** — so it can then walk the tensor once and switch off everything at or below it.

Weights are the exam papers. Magnitude is the mark. k is the 300. The threshold is the pass mark.

In the classroom	In rigLStep
6,144 exam papers	6,144 weights in conv3
each paper has a mark	each weight has a magnitude $ w $
fail the weakest 300	switch off the smallest k
announce a pass mark	return a threshold <code>dropT</code>
you do not need to name the 300	the caller re-walks the tensor and compares

NOTE — why the last row matters more than it looks

Once you have the pass mark, finding the failing students is easy: flip through the papers and compare. So the search does not need to *remember* anything — it only needs to produce the number.

That is what makes the cheap method possible. If the search had to hand back a list of which weights to drop, it would need somewhere to put that list, and we would be back where we started.

III.6.1 Two words to fix first

DEFINITION — scratch memory

Working space a function borrows while it runs and gives back when it finishes. Like clearing a desk to lay papers out: you need the desk only while sorting, but you do need it, and if the desk is too small you cannot use that method at all.

The whole of this section is an argument about how big a desk each method needs.

DEFINITION — threshold

A single value used as a cut-off, so that a later comparison can be a one-liner: `if (magnitude <= threshold) { drop it; }`. The pass mark.

DEFINITION — in place

An algorithm works **in place** if it rearranges the data where it already sits, without needing a copy. Sorting a hand of cards by shuffling them around in your hand is in place; laying them out on the table and rebuilding is not.

In-place methods need no scratch memory, which sounds ideal here. III.6.3 explains why it is not.

III.6.2 Method one: write every mark on a list

IN PLAIN ENGLISH — the obvious approach

Go through all 6,144 papers and write every mark on a long list. Now sort the list, count 300 entries in from the bottom, and read off the mark. That is your pass mark.

It works, it is easy to explain, and it is what almost anyone would do first. The trouble is the list.

WORKED EXAMPLE — how long is the list, in bytes?

One magnitude is a `float`, which is 4 bytes. So the list is $4 \times n$ bytes, where n is the number of weights.

Layer	weights n	list = $4n$ bytes	in KiB	fits in 320 KiB of SRAM?
linear 64->6	384	1,536	1.5	yes
conv1 9->16 k7	1,008	4,032	3.9	yes
conv2 16->32 k5	2,560	10,240	10.0	yes
conv3 32->64 k3	6,144	24,576	24.0	yes, but see below
a 1152×128 layer	147,456	589,824	576.0	no

NOTE — where the section title comes from, and a note on units

$147,456 \times 4 = 589,824$ bytes. Divide by 1000 and you get "590 KB", which is the figure this project has always used and the reason the section is called that. Divide by 1024 and it is "576 KiB".

The device has 320 KiB, which is 327,680 bytes. So in plain bytes the comparison is **589,824 needed against 327,680 available** — the desk is not merely too small, it is not even half big enough.

WARNING — for your model the list does fit — and that is still not a reason to use it

Conv3 is your largest layer at 6,144 weights, so its list is 24 KiB. That fits in 320 KiB. If the only goal were to make RigL run on the HAR classifier today, this method would work.

Two reasons to reject it anyway. **First**, 24 KiB is borrowed on top of a training footprint that already uses 185 KiB, and it is borrowed at the worst moment — while everything else is still resident. **Second**, the cost *grows with the layer*. Widen conv3 and the desk requirement grows with it. A method that works until someone changes a hyperparameter has a trap in it.

III.6.3 Method two: rearrange the papers where they lie

IN PLAIN ENGLISH — there is a clever algorithm that needs no desk at all

It is called **quickselect**. Pick any paper as a reference. Shuffle the pile so that everything scoring less than it ends up on the left and everything more on the right. Now you know exactly what position that paper occupies. If it is position 300 you are done; if not, repeat on whichever side contains position 300.

It finds the answer in about as much time as one pass through the pile, and it needs no scratch memory whatsoever, because it works **in place**.

DEFECT — and it cannot be used here, because the pile is the model

Quickselect works by **moving the papers around**. When it finishes, the weights are in a different order than they started.

That breaks something everything else depends on: weight *i*, gradient *i* and mask bit *i* must all refer to the same connection. Shuffle the weights and the mask now describes a network that no longer exists, the gradients point at the wrong weights, and the next forward pass computes nonsense.

TRACED — what the damage looks like

Before: weight 4 is 0.07, its gradient is 0.310, its mask bit says active.

Quickselect moves weight 4 to position 11.

After: position 4 now holds some other weight — but gradient 4 and mask bit 4 have not moved.

So `rigLStep` reads weight 4, gradient 4 and mask bit 4 and gets three facts about three *different* connections. Nothing crashes. Everything is wrong.

NOTE — you could copy the pile first — and then you are back to method one

Quickselect on a copy is perfectly correct. But the copy is exactly the 4n-byte list method one needed. You get to choose **linear time** or **zero memory**, not both, once the data is not allowed to move.

III.6.4 Method three: tally boxes

IN PLAIN ENGLISH — back to the classroom

Line up ten boxes on a shelf, labelled by mark range: 0-9, 10-19, 20-29, and so on. Take each paper, glance at the mark, and drop a tick into the matching box. Then put the paper down and forget it.

When you have gone through the pile, each box holds a **number** — how many students scored in that range. You no longer have the marks. You have the counts. And the counts are all you need.

DEFINITION — bucket (or box)

A range of values, together with a counter for how many things fell in that range. In the code the buckets are the array `g_hist[256]:g_hist[37]` means "how many weights had a magnitude in bucket 37's range".

DEFINITION — tally

A running count built one item at a time. Like tally marks on paper — at the end you have a number, not a list. Each item is counted and then discarded, which is exactly why the storage never grows.

CONCEPT — why this fixes the memory problem

The list method stores one number per **paper**. Ten thousand papers, ten thousand entries. The size of the desk is set by the size of the pile.

The tally method stores one number per **box**. Ten thousand papers, still ten boxes. A million papers, still ten boxes. **The size of the desk is set by how many boxes you chose, and you chose it.**

That independence is the whole point — more than the absolute saving. The cost stops being a function of the model.



The first three bars change if you change the model. The last one never does.

ALGORITHM — the method, in three steps

Step 1 — find the largest magnitude, so you know what range the boxes must span. (You cannot label the boxes until you know the highest mark.)

Step 2 — go through every weight, work out which box it belongs to, add one to that box.

Step 3 — walk the boxes from the lowest upward, keeping a running total, until the total reaches k . The box you stopped at contains the threshold.

III.6.5 The whole thing by hand

IN PLAIN ENGLISH — with ten boxes instead of 256, so every step is checkable

The real code uses 256 boxes. Ten is easier to draw and the method is identical. These are the eight active weights from the running example in II.7.

Magnitudes: 0.42, 0.18, 0.07, 0.11, 0.28, 0.47, 0.21, 0.15. We want $k = 3$ — the three smallest.

TRACED — step 1: how wide is a box?

The largest magnitude is **0.47**. We want ten boxes covering $[0, 0.47]$, so:

width = $0.47 / 9 = \mathbf{0.05222}$

scale = $9 / 0.47 = \mathbf{19.1489}$

Why divide by 9 and not 10? Ten boxes have nine gaps between their edges, the same way a ruler with ten marks has nine intervals. The code writes this as `RIGL_BUCKETS - 1`.

TRACED — step 2: which box does each weight go in?

Multiply the magnitude by `scale`, then throw away the fractional part. That whole number is the box.

weight	$ w $	$ w \times 19.1489$	drop the fraction	box
element 4	0.07	1.34	1	1
element 6	0.11	2.11	2	2
element 14	0.15	2.87	2	2
element 2	0.18	3.45	3	3
element 12	0.21	4.02	4	4
element 8	0.28	5.36	5	5
element 0	0.42	8.04	8	8
element 10	0.47	9.00	9	9

NOTE — notice box 2 has two weights in it

Elements 6 and 14 have different magnitudes — 0.11 and 0.15 — but both land in box 2, which covers $[0.1044, 0.1567]$. The box cannot tell them apart.

That is the **one thing you give up** with this method, and III.6.7 measures exactly what it costs. Hold on to this example; it is what the whole approximation question is about.

TRACED — step 3: walk the boxes, counting

Start at box 0 and add up as you go. Stop the moment the running total reaches $k = 3$.

box	covers magnitudes	count	running total	reached 3?
0	[0.0000, 0.0522)	0	0	no
1	[0.0522, 0.1044)	1	1	no

box	covers magnitudes	count	running total	reached 3?
2	[0.1044, 0.1567)	2	3	yes — stop here
3	[0.1567, 0.2089)	1	—	not reached
4	[0.2089, 0.2611)	1	—	not reached
5	[0.2611, 0.3133)	1	—	not reached
6, 7	empty	0	—	—
8	[0.4178, 0.4700)	1	—	not reached
9	[0.4700, 0.5222)	1	—	not reached

TRACED — and the answer

We stopped at box 2. Return the **top edge** of box 2:

threshold = $(2 + 1) \times 0.05222 = \mathbf{0.15667}$

Now check it. Which weights are at or below 0.15667?

0.07 **yes** 0.11 **yes** 0.15 **yes** 0.18 no 0.21 no 0.28 no 0.42 no 0.47 no

Three weights. Exactly k. And the total memory used was ten counters — whether the pile had 8 weights or 8 million.

DEFINITION — why the top edge and not the bottom

Box 2 covers [0.1044, 0.1567). Element 14 sits inside it at 0.15.

Return the **top** edge, 0.1567, and element 14 is at or below it — included, which is what we want, because it was one of the three we counted. Return the **bottom** edge, 0.1044, and element 14 fails the test: you asked for three and got two.

Selecting slightly too many is absorbed later by nDropped. Selecting too few is not. So the code always errs upward.

III.6.6 Scaling up to the real thing

IN PLAIN ENGLISH — everything above, with 256 boxes instead of 10

Nothing changes except the number. The real code spans [0, maxAbs] with 256 boxes, so the boxes are narrower and fewer weights share one.

On the same eight weights the boxes come out as 37, 59, 81, 97, 113, 151, 227 and 255 — all different, so no two share a box, and the threshold comes out at 0.151137 rather than 0.15667. Slightly tighter, same three weights selected.

The same three steps, in the shipped code

```
#define RIGL_BUCKETS 256u
static size_t g_hist[RIGL_BUCKETS];          /* 256 x 4 bytes = 1024 bytes */

float scale = (float)(RIGL_BUCKETS - 1u) / maxAbs; /* step 1 */

size_t b = (size_t)(v * scale);                /* step 2 */
if (b >= RIGL_BUCKETS) { b = RIGL_BUCKETS - 1u; }
g_hist[b]++;

for (size_t b = 0; b < RIGL_BUCKETS; b++) {    /* step 3 */
    running += g_hist[b];
    if (running >= k) { return (float)(b + 1u) * width; }
}
```

	ten boxes	256 boxes (the real code)
box width	0.05222	0.001843
boxes with more than one weight	box 2 has two	none
threshold returned	0.15667	0.151137
weights selected	3	3
memory	40 bytes	1,024 bytes

ON THE STM32 — what it costs on the F746ZG

Two passes over the data — one to find maxAbs, one to fill the boxes — then 256 comparisons to walk them. For conv3 that is about 150,000 cycles, roughly **0.7 ms** at 216 MHz.

It runs once per layer per rewiring, and rewiring happens every deltaT steps. At deltaT = 100 that is 7 microseconds of amortised cost per training step, against a step that takes hundreds of milliseconds. Invisible.

III.6.7 How wrong is the answer?

IN PLAIN ENGLISH — the honest question about box 2

Elements 6 and 14 shared a box, so the method could not tell 0.11 from 0.15. That means the threshold is a box edge rather than the exact k -th value, and slightly more than k weights can be selected.

How much more? Rather than argue about it, here is the measurement — weights drawn from a realistic distribution, the real algorithm run, the actual selection counted.

active weights	k asked for	actually selected	excess	excess as % of k
394 (conv3 at 90% sparse)	17	20	3	17.6%
1,021 (the whole model)	44	56	12	27.3%
6,144 (conv3 dense)	270	303	33	12.2%
147,456	6,488	8,097	1,609	24.8%

NOTE — and early in training, where k is much larger

The table above uses the late-training rate, $\alpha = 0.044$. Early on α is 0.3 and k is nearly seven times bigger:

active weights	k asked for	actually selected	excess	excess as % of k
394	118	119	1	0.8%
1,021	306	313	7	2.3%
6,144	1,843	1,865	22	1.2%
147,456	44,236	44,819	583	1.3%

CONCEPT — why small k is worse, and why none of it breaks anything

Weight magnitudes cluster near zero — most weights are small, few are large. So the low boxes are the crowded ones. A small k puts the threshold deep in that crowd, where one box may hold several weights, and you overshoot by a few. A large k puts it further out where boxes are nearly empty, and you overshoot by almost nothing.

In absolute terms it is small either way: 3 extra out of 394, or 12 out of 1,021. And it is **completely absorbed** — `nDropped` counts what actually went, and the grow phase puts back exactly that many. The sparsity stays conserved to the weight.

The only real consequence is that the effective rewiring rate sits a little above α , most noticeably late in training. Worth one sentence in the thesis if you report α .

III.6.8 When the boxes are exact

IN PLAIN ENGLISH — under int8 the approximation disappears entirely

Everything above assumed the weights can take any value. Once they are quantised they cannot — and that changes the picture completely.

An int8 weight holds one of 256 stored values, so its magnitude is one of at most 128 possibilities, and those are not arbitrary: they are exact multiples of the quantisation step. 0, s, 2s, 3s, and so on up to 127s.

CONCEPT — with more boxes than possible values, no two can share

The boxes span $[0, \text{maxAbs}]$ where $\text{maxAbs} = 127s$. With 256 boxes each is $127s / 255 \approx 0.5s$ wide — **half a quantisation step**.

Two neighbouring representable magnitudes are a full step apart, which is two box widths. They cannot land in the same box. So every box holds either one magnitude or none, and the tally knows exactly how many weights hold each possible value.

The histogram is then a **complete census** rather than a sample. It has lost nothing a full list would have known, and the threshold is exact.

DEFINITION — the sizing rule

For exactness you need one box per possible magnitude, plus margin:

`buckets > number of distinct magnitudes`

At int8 that is 128, so 129 boxes would strictly do. 256 is the next power of two, giving 2× margin so that floating-point rounding in $(\text{size_t})(v * \text{scale})$ cannot collapse two values into one box.

WARNING — and this holds at int8 only — check your operand width

The exactness argument is specific to 8-bit values. Your codebase does not default to that: `ODT_SYM_OPERAND_QMAXBITS` is 12.

operand width	distinct magnitudes	boxes needed for exactness	256 boxes are
int8	128	129	exact
int12 (the default)	2,048	2,049	approximate — about 8 magnitudes per box
int16	32,768	32,769	approximate — about 128 per box

NOTE — what exactness would cost at each width

int8 129 boxes = 0.5 KiB

int12 2,049 boxes = 8.0 KiB — affordable, if it turns out to matter

int16 32,769 boxes = 128.0 KiB — not on this device

At int12 you would still be using an approximation with 256 boxes, but a *bounded* one of exactly the kind measured in III.6.7 — a few extra weights per rewiring, absorbed by nDr opped. Whether the finer boxes are worth 8 KiB is a measurement, not an argument, and it uses the same operand-width knob as the sweep in IX.4.

III.6.9 The four methods, side by side

Method	Scratch memory	Speed	Moves the weights?	Verdict
sort the list	4n bytes	slowest	no, if copied	works, wasteful
partial selection	4n bytes	slow for large k	no, if copied	works, wasteful
quickselect in place	none	fast	yes	breaks the model
tally boxes	1 KiB, fixed	two passes	no	what the code does

TIP — if you remember one thing from this section

The list method stores one number per **weight**. The tally method stores one number per **box**. You choose how many boxes; you do not choose how many weights.

Everything else — the 590 KB, the exactness argument, the 1 KiB — follows from that single difference.

III.7 The two threshold searches

Same histogram, walked in opposite directions

src/tensor/RigL.c — NEW — the shared histogram machinery

```
#define SOURCE_FILE "RIGL"

#include <math.h>
#include <string.h>

#include "RigL.h"

/* 256 buckets is not an approximation for quantised weights: under int8
 * exactly 256 distinct magnitudes exist, so the histogram is a complete
 * census. For float32 it is a bounded approximation, and the drop/grow
 * accounting absorbs the error.
 * 256 * sizeof(size_t) = 1 KB, independent of layer size. */
#define RIGL_BUCKETS 256u

static size_t g_hist[RIGL_BUCKETS];

static float rigLMaxAbsSelected(tensor_t const *t, tensor_t const *mask, size_t n,
                               bool wantActive) {
    float maxAbs = 0.0f;
    for (size_t i = 0; i < n; i++) {
        if (tensorBoolGet(mask, i) != wantActive) {
            continue;
        }
        float v = fabsf(readBytesAsFloat(&t->data[i * sizeof(float)]));
        if (v > maxAbs) {
            maxAbs = v;
        }
    }
    return maxAbs;
}

static void rigLBuildHistogram(tensor_t const *t, tensor_t const *mask, size_t n,
                              bool wantActive, float maxAbs) {
    memset(g_hist, 0, sizeof(g_hist));
    float scale = (float)(RIGL_BUCKETS - 1u) / maxAbs;
    for (size_t i = 0; i < n; i++) {
        if (tensorBoolGet(mask, i) != wantActive) {
            continue;
        }
        float v = fabsf(readBytesAsFloat(&t->data[i * sizeof(float)]));
        size_t b = (size_t)(v * scale);
        if (b >= RIGL_BUCKETS) {
            b = RIGL_BUCKETS - 1u;
        }
        g_hist[b]++;
    }
}
```

Variable	Type	What it refers to	Example
t	tensor_t const *	The tensor being measured: weights for drop, gradients for grow	w or g
mask	tensor_t const *	The packed topology bits	1.3 KB for HAR

Variable	Type	What it refers to	Example
wantActive	bool	Which side to measure: true for active, false for inactive	true
maxAbs	float	Largest magnitude on the selected side; sets the bucket scale	0.0421f
scale	float	255 / maxAbs — converts a magnitude to a bucket index	6057.0f
g_hist	size_t[256]	The counters. File-static, 1 KB, reused by both searches	—

Line	Code	What it does, and why
	<code>#define RIGL_BUCKETS 256u</code>	The bucket count. Chosen to match the number of distinct int8 values, which is what makes the histogram exact rather than approximate under quantisation.
	<code>static size_t g_hist[RIGL_BUCKETS];</code>	File-static, not a parameter. 1 KB, allocated once by the linker, reused by both searches. This is what replaced the <code>float *scratch</code> argument — and because the linker places it, the size appears in the map file and can be budgeted.
	<code>static float rigLMaxAbsSelected(... bool wantActive)</code>	First pass. Finds the largest magnitude on <i>one side</i> of the mask, which sets the scale for the buckets. <code>static</code> : private to this file.
	<code>if (tensorBoolGet(mask, i) != wantActive) { continue; }</code>	The side selector. Pass <code>true</code> and it walks active weights; <code>false</code> and it walks inactive ones. One function serves both phases.
	<code>float v = fabsf(readBytesAsFloat(&t->data[i * sizeof(float)]));</code>	<code>fabsf</code> , not <code>fabs</code> — the float version. On the F746ZG the double version costs roughly forty times more, and this runs once per element.
	<code>if (v > maxAbs) { maxAbs = v; }</code>	Ordinary running maximum. Seeded at 0.0f rather than the first element, which is correct here because magnitudes are non-negative by construction.
	<code>static void rigLBuildHistogram(... float maxAbs)</code>	Second pass. Buckets every selected element.
	<code>memset(g_hist, 0, sizeof(g_hist))</code>	Clear the counters. <code>sizeof(g_hist)</code> rather than a literal 1024 — if the bucket count ever changes, this follows automatically.
	<code>float scale = (float) (RIGL_BUCKETS - 1u) / maxAbs;</code>	Converts a magnitude into a bucket index. Computed once outside the loop, so the per-element cost is one multiply rather than one divide.
	<code>size_t b = (size_t)(v * scale);</code>	The bucket. Truncation toward zero, so a value exactly at a boundary lands in the lower bucket.
	<code>if (b >= RIGL_BUCKETS) { b = RIGL_BUCKETS - 1u; }</code>	Clamp. Floating-point rounding can push the largest element one past the end; without this it would write past the array.
	<code>g_hist[b]++;</code>	Count it. One increment per selected element, and the whole pass is $O(n)$.

WORKED EXAMPLE — both passes on the running example, active side

First `rigLMaxAbsSelected(w, mask, 16, true)` walks the eight active elements and takes the largest magnitude.

Magnitudes: 0.42, 0.18, 0.07, 0.11, 0.28, 0.47, 0.21, 0.15 — so `maxAbs = 0.4700`.

Then `scale = 255 / 0.47 = 542.5532`, and `rigLBuildHistogram` buckets each one.

idx	w	w × scale	bucket	Note
4	0.07	37.98	37	smallest active magnitude
6	0.11	59.68	59	—
14	0.15	81.38	81	—
2	0.18	97.66	97	—
12	0.21	113.94	113	—
8	0.28	151.91	151	—
0	0.42	227.87	227	—
10	0.47	255.00	255	the maximum, clamped to the last bucket

NOTE — element 10 is why the clamp exists

Its magnitude is `maxAbs`, so `|w| × scale` is exactly 255.0 — the last valid index. A hair of floating-point rounding upward would give 256 and write one past the end of a 256-entry array.

The `if (b >= RIGL_BUCKETS)` line costs one comparison per element and removes that entire class of failure. On a device with no MMU, a one-element overrun corrupts whatever the linker placed next.

src/tensor/RigL.c — NEW — the drop threshold

```
float rigLKthSmallestActive(tensor_t const *w, tensor_t const *mask, size_t k) {
    if (w == NULL || mask == NULL || k == 0u) {
        return -1.0f; /* nothing qualifies */
    }
    size_t n = calcNumberOfElementsByTensor((tensor_t *)w);
    float maxAbs = rigLMaxAbsSelected(w, mask, n, true);
    if (maxAbs <= 0.0f) {
        return 0.0f; /* every active weight is zero */
    }
    rigLBuildHistogram(w, mask, n, true, maxAbs);

    float width = maxAbs / (float)(RIGL_BUCKETS - 1u);
    size_t running = 0;
    for (size_t b = 0; b < RIGL_BUCKETS; b++) {
        running += g_hist[b];
        if (running >= k) {
            /* Upper edge of bucket b: every element in buckets 0..b satisfies
             * |v| <= this. Erring high is safe -- the caller counts what it
             * actually dropped and grows back exactly that many. */
            return (float)(b + 1u) * width;
        }
    }
    return maxAbs;
}
```

Variable	Type	What it refers to	Example
w	tensor_t const *	The weight tensor. Read only — this function measures, it does not prune	conv3 weights
k	size_t	How many active weights RigL wants to drop	17
maxAbs	float	Largest active magnitude; 0 means every active weight is already zero	0.0421f
width	float	How much magnitude one bucket spans: maxAbs / 255	0.000165f
running	size_t	Weights counted so far, walking up from bucket 0	accumulates to k
b	size_t	The bucket the running total crossed k in	12

Line	Code	What it does, and why
	if (w == NULL mask == NULL k == 0u) return -1.0f;	Returning a negative threshold is deliberate: no magnitude is negative, so $ w \leq -1.0f$ is false for everything and the caller drops nothing. A meaningless request produces a harmless answer rather than a crash.
	float maxAbs = rigLMaxAbsSelected(w, mask, n, true);	true: measure the active side. This is the only place the drop and grow searches differ in their first pass.
	if (maxAbs <= 0.0f) return 0.0f;	Every active weight is already exactly zero. Returning 0.0f means the caller drops all of them, which is the only sensible reading of the request.
	float width = maxAbs / (float)(RIGL_BUCKETS - 1u);	How much magnitude one bucket spans. The inverse of the scale used to build the histogram.
	for (size_t b = 0; b < RIGL_BUCKETS; b++)	Walk upward from the smallest bucket, because we want the k <i>smallest</i> magnitudes.
	running += g_hist[b];	Accumulate. After this line running is how many active weights have magnitude in buckets 0..b.

Line	Code	What it does, and why
	<code>if (running >= k)</code>	Stop as soon as we have at least k. Because of ties it will often be more than k, which is expected and handled.
	<code>return (float)(b + 1u) * width;</code>	The upper edge of bucket b, so every element in buckets 0..b satisfies $ v \leq \text{this}$. Erring high is safe: the caller counts what it actually dropped and grows back exactly that many.
	<code>return maxAbs;</code>	Fallback if the loop never reaches k — only possible when k exceeds the number of active weights. Returns the largest magnitude, so everything qualifies.

WORKED EXAMPLE — `rigLKthSmallestActive(w, mask, 3)` on the running example

From the histogram in III.6 the eight active elements sit in buckets 37, 59, 81, 97, 113, 151, 227 and 255. We want the 3 smallest, so we walk upward accumulating until `running ≥ 3`.

bucket b	count	running	running >= 3?	which element
0 – 36	0	0	no	—
37	1	1	no	element 4, $ w = 0.07$
38 – 58	0	1	no	—
59	1	2	no	element 6, $ w = 0.11$
60 – 80	0	2	no	—
81	1	3	yes — stop	element 14, $ w = 0.15$

TRACED — what the threshold comes out as, and what it selects

`width = maxAbs / 255 = 0.47 / 255 = 0.001843`
`return (81 + 1) × 0.001843 = 0.151137`

The caller then compares every active magnitude against it:

$0.07 \leq 0.151$ **ok** $0.11 \leq 0.151$ **ok** $0.15 \leq 0.151$ **ok**
 $0.18, 0.21, 0.28, 0.42, 0.47$ all exceed it **no**

So `nDropped = 3`, and elements 4, 6 and 14 are pruned. The mask goes from `0x55 0x55` to `0x05 0x15`, and the active count from 8 to 5.

NOTE — why the upper edge, and why 0.15 just makes it

Element 14 has magnitude 0.15 and the threshold is 0.151137 — it qualifies by about one part in a hundred. That is the upper-edge return doing its job: the bucket that took the running total past k must be *included*, so the threshold has to sit above everything in it.

Return the lower edge instead and element 14 would fail the test, `nDropped` would be 2 rather than 3, and `RigL` would drop fewer weights than it asked for. Under-selecting is the one thing the accounting cannot absorb.

src/tensor/RigL.c — NEW — the grow threshold


```

float rigLKthLargestInactive(tensor_t const *g, tensor_t const *mask, size_t k) {
    if (g == NULL || mask == NULL || k == 0u) {
        return -1.0f;
    }
    size_t n      = calcNumberOfElementsByTensor((tensor_t *)g);
    float maxAbs = rigLMaxAbsSelected(g, mask, n, false);
    if (maxAbs <= 0.0f) {
        /* Every inactive gradient is zero. The grow phase cannot choose
         * meaningfully -- report it rather than selecting arbitrarily. */
        PRINT_ERROR("rigL: all inactive gradients are zero;"
                    " grow phase has no signal");
        return 0.0f;
    }
    rigLBuildHistogram(g, mask, n, false, maxAbs);

    float width = maxAbs / (float)(RIGL_BUCKETS - 1u);
    size_t running = 0;
    for (size_t b = RIGL_BUCKETS; b-- > 0;) {
        running += g_hist[b];
        if (running >= k) {
            /* Lower edge of bucket b: every element in buckets b..255
             * satisfies |v| >= this. */
            return (float)b * width;
        }
    }
    return 0.0f;
}

```

Line	Code	What it does, and why
	float maxAbs = rigLMaxAbsSelected(g, mask, n, false);	Two differences from the drop search, both on this line: it measures <i>g</i> , the gradients , and false selects the inactive side.
	if (maxAbs <= 0.0f) { PRINT_ERROR(...); return 0.0f;	This branch reports rather than staying silent , unlike its counterpart in the drop search. Every inactive gradient being zero means something upstream zeroed them — the trap in VI.3 — and the grow phase would then select arbitrarily. Worth a message.
	for (size_t b = RIGL_BUCKETS; b-- > 0;)	Walk downward from the largest bucket, because we want the <i>k largest</i> gradients. See the idiom note below.
	return (float)b * width;	The lower edge of bucket <i>b</i> , so every element in buckets <i>b</i> ..255 satisfies $ v \geq \text{this}$. The mirror of the drop search returning an upper edge.

WORKED EXAMPLE — rigLKthLargestInactive(g, mask, 3) — the grow half

After the drop the mask is 0x05 0x15, so eleven elements are inactive: 1, 3, 4, 5, 6, 7, 9, 11, 13, 14, 15. Note that 4, 6 and 14 are among them — just pruned, and immediately eligible again.

Their **gradients**, largest first: 0.310 (el 4), 0.280 (el 9), 0.260 (el 15), 0.240 (el 1), 0.220 (el 11), 0.190 (el 6), 0.170 (el 13), 0.130 (el 3), 0.006 (el 14), 0.004 (el 5), 0.002 (el 7).

maxAbs = 0.3100, so width = 0.31 / 255 = 0.001216. Walking buckets **downward** until three are counted stops at bucket 212, giving growT = 212 × 0.001216 = **0.258941**.

element	gradient	>= growT?	Grown?
4	0.310	yes	yes
9	0.280	yes	yes

element	gradient	>= growT?	Grown?
15	0.260	yes	yes
1	0.240	no — 0.240 < 0.2589	no
11	0.220	no	no
6	0.190	no	no — stays pruned
14	0.006	no	no — stays pruned

CONCEPT — element 4 is the algorithm in miniature

It was pruned three lines ago for having the second-smallest weight in the layer (0.07). It is grown back immediately for having the largest gradient in the whole tensor (0.310).

That is not the algorithm contradicting itself. It is the two criteria saying different things, which is exactly why RigL uses two: *"this connection is contributing almost nothing at its current value, but the loss surface says it would move a long way if it were switched on."* A pure magnitude-pruning scheme would have removed it and never reconsidered.

Element 6 shows the other outcome: pruned for a small weight (0.11), and its gradient (0.190) is not in the top three, so it stays out. Element 14 likewise — small weight, and a gradient of 0.006 that says the network does not want it back.

	b0	b1	b2	b3	b4	b5	b6	b7
before	1	0	1	0	1	0	1	0
after drop	1	0	1	0	0	0	0	0
after grow	1	0	1	0	1	0	0	0

Mask byte 0 through one update: 0x55, then 0x05 after dropping elements 4 and 6, then 0x15 after growing element 4 back. Byte 1 goes 0x55 -> 0x15 -> 0x97.

TRACED — and the count is conserved

Active before: **8**. Dropped: 3 (elements 4, 6, 14). Grown: 3 (elements 4, 9, 15). Active after: **8**.

Mask bytes: 0x55 0x55 -> 0x05 0x15 -> 0x15 0x97.

popcount(0x15) + popcount(0x97) = 3 + 5 = **8**. Unchanged, as required.

	Drop search	Grow search
Reads	the weight tensor w	the gradient tensor g
Selects from	active weights	inactive weights
Wants the	k <i>smallest</i> magnitudes	k <i>largest</i> magnitudes
Walks buckets	bottom up, 0 to 255	top down, 255 to 0
Returns	upper edge (b+1) × width	lower edge b × width
Caller compares	w <= dropT	g >= growT

IN PLAIN ENGLISH — why the two return different bucket edges

The drop search returns the *upper* edge so that everything in buckets 0..b satisfies $|w| \leq \text{dropT}$. The grow search returns the *lower* edge so that everything in buckets b..255 satisfies $|g| \geq \text{growT}$.

Both err on the generous side, selecting at least k and possibly a few more. That is deliberate: the drop loop counts what it actually removed, and the grow loop stops early once it has enough. Over-selection is absorbed; under-selection would not be.

TIP — the loop that counts down without wrapping

`for (size_t b = RIGL_BUCKETS; b-- > 0; )` is the standard C idiom for iterating an unsigned counter downward. The test and decrement happen together: b is compared against 0, *then* decremented, so the body sees 255 down to 0 and the loop ends without b ever wrapping to SIZE_MAX.

The naive `for (size_t b = 255; b >= 0; b--)` never terminates, because an unsigned value is always ≥ 0 .

ON THE STM32 — two passes per search, and why that is affordable

Each search walks the tensor twice: once for maxAbs, once to bucket. For conv3 that is $2 \times 6,208$ elements at roughly 12 cycles each — about 150,000 cycles, 0.7 ms at 216 MHz.

It runs once per layer per deltaT steps. At deltaT = 100 that is 7 microseconds amortised per training step, against a step that takes hundreds of milliseconds. Invisible.

PART IV

Establishing the Initial Topology

`tensorMaskInitDense` gives you an all-ones mask. RigL needs the network already at target sparsity before step 1. This Part is the missing half of setup — and the choice of how to distribute the budget matters more on a small model than on a large one.

IV.1 Why InitDense is not enough

Two steps, not one

IN PLAIN ENGLISH — what has to happen before step 1

RigL is sparse-to-sparse: the network is at its target sparsity from the very first forward pass. `tensorMaskInitDense` leaves every bit set, so on its own it gives you a fully dense network that RigL would then hold dense forever — because RigL conserves whatever sparsity it is handed.

The second step establishes the level. RigL then maintains it and never reads the target again.

```
tensorMaskInitDense(mask, storage, shape, boolQ)
```

III.4 - plumbing + all ones

```
rigLInitSparsityERK(specs, numLayers, 0.90f)
```

IV.3 - drop to target, per layer

```
rigLInitSparsity(mask, 1.0f - density)
```

IV.2 - once per layer

```
tensorBoolSet(mask, i, false)
```

selection sampling

```
rigLApplyToParameter(weights, mask)
```

III.5 - invariant true at step 0

The complete setup sequence. Only the first box exists today, and it does not compile.

NOTE — why the split into two functions

The plumbing — building the BOOL descriptor, wiring the tensor to its storage, clearing padding bits — is the same regardless of what topology you want. The topology decision is separate and may be uniform, ERK, or loaded from a checkpoint.

Keeping them apart means each has one job, and the postcondition of the first (`countActive == n`) is trivially checkable.

IV.2 rigLInitSparsity

Exactly k of n , chosen uniformly, in one pass

src/tensor/RigL.c — NEW

```
void rigLInitSparsity(tensor_t *mask, float targetSparsity) {
    if (mask == NULL || targetSparsity <= 0.0f) {
        return;
    }
    if (targetSparsity >= 1.0f) {
        PRINT_ERROR("rigLInitSparsity: targetSparsity %.3f leaves no active weights",
                    (double)targetSparsity);
        return;
    }
    size_t n = calcNumberOfElementsByShape(mask->shape);
    if (n == 0u) {
        return;
    }
    size_t toDrop = (size_t)(targetSparsity * (float)n + 0.5f);
    if (toDrop >= n) {
        toDrop = n - 1u;          /* always keep at least one active */
    }

    /* Selection sampling: walk once, clearing exactly toDrop bits, each
     * subset equally likely. No rejection loop, no second pass. */
    size_t needed = toDrop;
    size_t remaining = n;
    for (size_t i = 0; i < n && needed > 0u; i++) {
        if (rngNextFloat() * (float)remaining < (float)needed) {
            tensorBoolSet(mask, i, false);
            needed--;
        }
        remaining--;
    }
    PRINT_INFO("rigLInitSparsity: %zu of %zu weights pruned (%.1f%%)", toDrop, n,
              (double)(100.0f * (float)toDrop / (float)n));
}
```

Variable	Type	What it refers to	Example
mask	tensor_t *	The all-ones mask from tensorMaskInitDense, about to be thinned	6,144 bits, all set
targetSparsity	float	Fraction to switch OFF, not the fraction to keep	0.936f for conv3
n	size_t	How many weights this layer has	6,144
toDrop	size_t	How many bits to clear — the only fraction-to-count conversion	5,750
needed	size_t	How many still to clear; counts down to 0	starts at toDrop
remaining	size_t	How many elements are left to consider; counts down to 0	starts at n

Line	Code	What it does, and why
	if (mask == NULL targetSparsity <= 0.0f) return;	Zero or negative sparsity means "leave it dense", which is what an all-ones mask already is. Nothing to do.

Line	Code	What it does, and why
	<pre>if (targetSparsity >= 1.0f) { PRINT_ERROR(...); return; }</pre>	100% sparsity would leave no active weights at all — a layer that computes nothing. Reported rather than silently clamped, because it is almost certainly a config mistake.
	<pre>size_t toDrop = (size_t) (targetSparsity * (float)n + 0.5f);</pre>	The one place a sparsity fraction becomes a count. The <code>+ 0.5f</code> before truncating is round-to-nearest: without it, $0.90 \times 6144 = 5529.6$ would truncate to 5529 rather than 5530.
	<pre>if (toDrop >= n) { toDrop = n - 1u; }</pre>	Always keep at least one active weight. Belt and braces given the check above, but it costs one comparison and removes an entire class of degenerate layer.
	<pre>size_t needed = toDrop; size_t remaining = n;</pre>	The two counters that make selection sampling work. Their <i>ratio</i> is the probability of selecting the current element.
	<pre>for (size_t i = 0; i < n && needed > 0u; i++)</pre>	Walk every element once. The second condition exits early once enough have been cleared — no point walking the tail when nothing remains to select.
	<pre>if (rngNextFloat() * (float)remaining < (float)needed)</pre>	The heart of it. Select this element with probability <code>needed / remaining</code> . Written as a multiply rather than a divide because division is slower and the comparison is equivalent.
	<pre>tensorBoolSet(mask, i, false); needed--;</pre>	Clear the bit and record that one fewer is needed. This is what makes the probability self-correcting: fall behind and the ratio rises.
	<pre>remaining--;</pre>	Decrement on every iteration, selected or not. That is what makes the denominator "how many chances are left".

NOTE — this is the one line where a sparsity level enters the system

`toDrop = targetSparsity × n`. Nothing else in the whole implementation converts a fraction into a count. `rigLStep` never reads `targetSparsity` at all — it measures `numActive` from the mask and conserves it.

CONCEPT — selection sampling, and why not the obvious alternatives

You need exactly `toDrop` of `n` bits cleared, chosen uniformly. Three approaches:

Coin flip per element — clear with probability 0.9. Simple, and gives the wrong *count*: binomial spread around 90%, so the actual sparsity varies run to run and is never exactly what you reported.

Pick random indices until enough are cleared — correct, but at 90% sparsity it degenerates into coupon-collecting, re-drawing indices already cleared. Unbounded runtime.

Selection sampling, above — one pass, exact count, uniform. At element `i` the probability of selecting it is `needed / remaining`, which self-corrects: fall behind and it rises, run ahead and it falls.

TRACED — why the probabilities work out

Take $n = 10$, $\text{toDrop} = 3$.

At $i = 0$: probability $3/10$. Say it is cleared — now needed = 2, remaining = 9.

At $i = 1$: probability $2/9$. Say not — needed stays 2, remaining = 8.

At $i = 2$: probability $2/8$ — *higher* than $2/9$, because there is less room left to fit the two remaining selections.

By the last element, if needed still equals remaining, the probability is 1 and it is certain. The count cannot come out wrong.

TIP — verified, not just argued

The harness in Part VIII runs this on a 4,096-element mask at 90% and checks the active count is exactly 410, the expected value. It is — every time, for any seed.

IV.3 ERK — distributing one budget across layers

Because uniform sparsity starves the small layers

IN PLAIN ENGLISH — the problem with 90% everywhere

`rigLInitSparsity` takes one mask and one fraction. Call it per layer with 0.90 and every layer loses 90% of its weights — which sounds fair and is not.

On the HAR classifier that leaves the final Linear layer with **38 weights** to map 64 features onto 6 classes. It is the classifier: every prediction passes through it. Cutting it as hard as a 6,144-weight convolution is not a considered decision, it is an accident of using one number everywhere.

DEFINITION — ERK — Erdos-Renyi-Kernel

A rule from the RigL paper for splitting one *global* budget across layers of different sizes. Each layer gets a raw score

$$s = (\text{fanIn} + \text{fanOut} + \text{kernel}) / n$$

and a density proportional to it. Because n is in the denominator, small layers score higher and end up denser. The total kept still matches the budget exactly.

CONCEPT — the intuition

The numerator is roughly "how many distinct things this layer connects" — its inputs, its outputs, its kernel positions. The denominator is how many weights it spends doing so.

A layer with many weights per connection has redundancy to give up. A layer with few has none. The ratio measures how much a layer can afford to lose, and ERK allocates density in proportion to it.

src/tensor/include/RigL.h — NEW — the ERK declarations

```
/* src/tensor/include/RigL.h */

/* One entry per sparsifiable layer. kernel is 1 for a Linear layer,
 * the kernel size for a Conv1d. */
typedef struct {
    tensor_t *mask;
    size_t    fanIn;
    size_t    fanOut;
    size_t    kernel;
} rigLLayerSpec_t;

#define RIGL_MAX_SPARSE_LAYERS 32u

void rigLInitSparsityERK(rigLLayerSpec_t *layers, size_t numLayers,
                        float targetSparsity);
```

src/tensor/RigL.c — NEW — the score

```
static float rigLerkscore(rigLLayerSpec_t const *L, size_t n) {
    if (n == 0u) {
        return 0.0f;
    }
    float sumDims = (float)L->fanIn + (float)L->fanOut;
    if (L->kernel > 1u) {
        sumDims += (float)L->kernel; /* convolution: kernel is a real dim */
    }
    return sumDims / (float)n;
}
```

Line	Code	What it does, and why
	<code>static float rigLerkscore(rigLLayerSpec_t const *L, size_t n)</code>	<code>static</code> : private helper. Takes the layer description and its weight count separately, because the caller has already computed the count.
	<code>if (n == 0u) return 0.0f;</code>	A layer with no weights scores zero and therefore receives no budget. Also avoids a division by zero on the last line.
	<code>float sumDims = (float)L->fanIn + (float)L->fanOut;</code>	The numerator starts as inputs plus outputs. For a Linear layer that is the whole of it.
	<code>if (L->kernel > 1u) sumDims += (float)L->kernel;</code>	A convolution has a third dimension, so its kernel size counts too. The guard is what lets one function serve both layer types: pass <code>kernel = 1</code> for Linear and the term is correctly omitted.
	<code>return sumDims / (float)n;</code>	The score. Because <code>n</code> is in the denominator, a small layer scores high and ends up denser — which is the entire point of ERK.

WORKED EXAMPLE — rigLerkscore on all four HAR layers

Computed by running the function, not by hand.

Layer	fanIn	fanOut	kernel	n	sumDims	score = sumDims / n
conv1	9	16	7	1,008	9+16+7 = 32	0.03175
conv2	16	32	5	2,560	16+32+5 = 53	0.02070
conv3	32	64	3	6,144	32+64+3 = 99	0.01611
linear	64	6	1	384	64+6 = 70	0.18229

NOTE — the linear row shows both special cases at once

Its kernel is 1, so the `if (L->kernel > 1u)` guard omits the kernel term — `sumDims` is 70, not 71. And its score is **eleven times** conv3's, because it connects almost as many things (70 against 99) using one sixteenth the weights (384 against 6,144).

That ratio is the whole mechanism. ERK is not favouring the classifier because someone decided classifiers are important; it falls out of the arithmetic of connections per weight.

[src/tensor/RigL.c — NEW — setup](#)

```

void rigLInitSparsityERK(rigLLayerSpec_t *layers, size_t numLayers,
                        float targetSparsity) {
    if (layers == NULL || numLayers == 0u) {
        return;
    }
    if (numLayers > RIGL_MAX_SPARSE_LAYERS) {
        PRINT_ERROR("rigLInitSparsityERK: %zu layers exceeds"
                    " RIGL_MAX_SPARSE_LAYERS (%u)",
                    numLayers, (unsigned)RIGL_MAX_SPARSE_LAYERS);
        return;
    }
    if (targetSparsity <= 0.0f || targetSparsity >= 1.0f) {
        PRINT_ERROR("rigLInitSparsityERK: targetSparsity %.3f out of range",
                    (double)targetSparsity);
        return;
    }

    size_t n[RIGL_MAX_SPARSE_LAYERS];
    float score[RIGL_MAX_SPARSE_LAYERS];
    bool isDense[RIGL_MAX_SPARSE_LAYERS];
    size_t total = 0;

    for (size_t i = 0; i < numLayers; i++) {
        n[i] = calcNumberOfElementsByShape(layers[i].mask->shape);
        score[i] = rigLErkScore(&layers[i], n[i]);
        isDense[i] = false;
        total += n[i];
    }

    float targetKeep = (1.0f - targetSparsity) * (float)total;

```

Variable	Type	What it refers to	Example
layers	rigLLayerSpec_t *	One entry per sparsifiable layer: mask, fanIn, fanOut, kernel	4 entries for HAR
targetSparsity	float	The global budget across all layers, not per layer	0.90f
n[i]	size_t	Weight count of layer i	1008, 2560, 6144, 384
score[i]	float	ERK raw score: sumDims / n	0.0317 for conv1
isDense[i]	bool	Pinned to full density by the clamp loop	all false here
total	size_t	Weights across every layer	10,096
targetKeep	float	How many weights to keep in total	1,010

Line	Code	What it does, and why
	if (numLayers > RIGL_MAX_SPARSE_LAYERS)	A bound check, because the three arrays below are fixed-size. This is the pattern that replaces a variable-length array — fixed capacity plus an explicit check, so the stack usage is known at build time.
	size_t n[RIGL_MAX_SPARSE_LAYERS]; float score[...]; bool isDense[...];	Three parallel arrays indexed by layer. Fixed size, on the stack, 32 entries each — about 300 bytes, which is affordable and, crucially, <i>bounded</i> .
	n[i] = calcNumberOfElementsByShape(layers[i].mask->shape);	The weight count comes from the mask's shape, which by construction matches the weight tensor's.
	score[i] = rigLErkScore(&layers[i], n[i]);	One score per layer, computed once and reused across every pass of the clamp loop.

Line	Code	What it does, and why
	<pre>float targetKeep = (1.0f - targetSparsity) * (float)total;</pre>	Converts the global sparsity into a global <i>keep</i> count. Note this is across all layers together — that is what makes it a shared budget rather than a per-layer rule.

src/tensor/RigL.c — NEW — the clamp-and-redistribute loop

```

/* Clamp-and-redistribute: any layer whose ERK density would exceed 1.0 is
 * pinned dense and its weights removed from the budget, then eps is
 * recomputed for the rest. Terminates in at most numLayers passes. */
for (size_t pass = 0; pass <= numLayers; pass++) {
    float budget = targetKeep;
    float denom = 0.0f;
    for (size_t i = 0; i < numLayers; i++) {
        if (isDense[i]) {
            budget -= (float)n[i];
        } else {
            denom += score[i] * (float)n[i];
        }
    }
    if (denom <= 0.0f || budget <= 0.0f) {
        break;
    }
    float eps = budget / denom;
    bool clamped = false;
    for (size_t i = 0; i < numLayers; i++) {
        if (!isDense[i] && eps * score[i] > 1.0f) {
            isDense[i] = true;
            clamped = true;
        }
    }
    if (!clamped) {
        for (size_t i = 0; i < numLayers; i++) {
            float density = isDense[i] ? 1.0f : eps * score[i];
            if (density > 1.0f) {
                density = 1.0f;
            }
            PRINT_INFO("rigL ERK L%zu: n=%zu density=%.4f active=%zu", i,
                n[i], (double)density,
                (size_t)(density * (float)n[i] + 0.5f));
            rigLInitSparsity(layers[i].mask, 1.0f - density);
        }
        return;
    }
}
PRINT_ERROR("rigLInitSparsityERK: budget could not be distributed");
}

```

Line	Code	What it does, and why
	<pre>for (size_t pass = 0; pass <= numLayers; pass++)</pre>	The outer loop bound is a termination guarantee, not an expectation. Each pass pins at least one layer dense, and there are numLayers layers, so it cannot run longer than that. For HAR it finishes on pass 0.
	<pre>if (isDense[i]) budget -= (float)n[i]; else denom += score[i] * n[i];</pre>	Split the layers. Already-pinned ones consume their full weight count from the budget; the rest contribute to the denominator that eps will be computed from.
	<pre>if (denom <= 0.0f budget <= 0.0f) break;</pre>	Nothing left to distribute, or nothing left to distribute it to. Exits rather than dividing by zero.

Line	Code	What it does, and why
	<code>float eps = budget / denom;</code>	The scaling constant. Multiply any layer's score by this and you get its density. Solving $\text{sum of } \text{eps} \times \text{score} \times n = \text{budget}$ for <code>eps</code> is exactly this division.
	<code>if (!isDense[i] && eps * score[i] > 1.0f)</code>	Would this layer be asked to keep more than 100% of its weights? If so it must be pinned dense and the budget recomputed without it.
	<code>isDense[i] = true; clamped = true;</code>	Pin it, and record that another pass is needed because <code>eps</code> has just changed for everyone else.
	<code>if (!clamped) { ... return; }</code>	The exit path. Nothing was clamped this pass, so <code>eps</code> is final. Apply it and return — this is the only return on the success path.
	<code>float density = isDense[i] ? 1.0f : eps * score[i];</code>	A pinned layer keeps everything; everyone else gets its share. The ternary is the whole of the clamp's effect at application time.
	<code>rigLInitSparsity(layers[i].mask, 1.0f - density);</code>	Density becomes sparsity here. <code>rigLInitSparsity</code> takes the fraction to switch off, so the complement is what gets passed.
	<code>PRINT_ERROR("... could not be distributed");</code>	Only reachable if the loop exhausted its passes, which the termination argument says cannot happen. Present because a silent fall-through would be worse than an impossible message.

WORKED EXAMPLE — the clamp loop on the HAR model, pass by pass

`total = 10,096`, `targetSparsity = 0.90`, so `targetKeep = 0.10 × 10,096 = 1,010`.

Pass 0	conv1	conv2	conv3	linear
isDense entering	false	false	false	false
score × n	32	53	99	70
<i>denom = 32+53+99+70</i>				254
<i>eps = 1010 / 254</i>				3.976
eps × score	0.1262	0.0823	0.0640	0.7246
> 1.0?	no	no	no	no
clamped				none — eps is final

TRACED — applying eps, and checking the total

Each layer gets `density = eps × score`, then `rigLInitSparsity(mask, 1 - density)`.

conv1: $0.1262 \times 1,008 = \mathbf{127}$ active (sparsity passed in: 0.8738)

conv2: $0.0823 \times 2,560 = \mathbf{211}$ (0.9177)

conv3: $0.0640 \times 6,144 = \mathbf{394}$ (0.9360)

linear: $0.7246 \times 384 = \mathbf{278}$ (0.2754)

$127 + 211 + 394 + 278 = \mathbf{1,010}$ — the budget, exactly. Achieved global sparsity: $1 - 1010/10096 = \mathbf{90.00\%}$.

NOTE — when the clamp loop would actually run

For HAR it exits on pass 0 because no density exceeded 1.0. Push the target to 97% and targetKeep drops to 303, giving $\text{eps} = 303/254 = 1.193$ — and $1.193 \times 0.18229 = 0.217$ for linear, still under 1.

You would need roughly 99.5% global sparsity before the classifier's ERK density exceeded 1.0 and the clamp fired. It is a guard against a configuration you are unlikely to use, but the cost is a dozen lines and its absence would be an infinite density rather than an error.

IN PLAIN ENGLISH — why the clamp loop is needed

ERK gives small layers a high score, and at aggressive global sparsity the arithmetic can hand a very small layer a density above 1.0 — "keep 130% of your weights", which is meaningless.

Pin that layer dense, remove its weights from the budget, recompute eps for the rest. That may push another over 1.0, so it repeats. It terminates because each pass pins at least one layer.

IV.4 The HAR distribution, computed

Run against the real layer shapes

TRACED — the HAR classifier at 90% global sparsity

Four sparsifiable layers, 10,096 weights between them (biases excluded). Budget: keep 10%, so 1,010 weights.

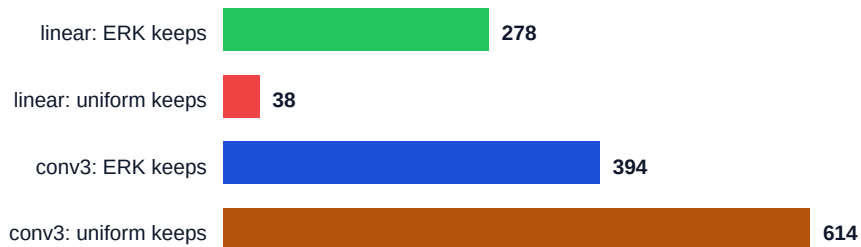
Layer	n	fanIn+fanOut+k	score s	density	active
conv1 9->16 k7	1,008	9+16+7 = 32	0.03175	0.1262	127
conv2 16->32 k5	2,560	16+32+5 = 53	0.02070	0.0823	211
conv3 32->64 k3	6,144	32+64+3 = 99	0.01611	0.0640	394
linear 64->6	384	64+6 = 70	0.18229	0.7246	278
total	10,096	254	—	—	1,010

IN PLAIN ENGLISH — the sum simplifies, which is a useful check

Because $s = \text{sumDims} / n$, the denominator sum of $s \times n$ is just sum of $\text{sumDims} = 32 + 53 + 99 + 70 = 254$. So $\text{eps} = 1010 / 254 = 3.976$, and each density is $3.976 \times s$.

No layer exceeded 1.0, so the clamp loop finished on its first pass. Running it gives an achieved global sparsity of **90.00%**, exactly on target.

Layer	weights	ERK active	uniform active	ERK / uniform
conv1 9->16 k7	1,008	127	101	1.26×
conv2 16->32 k5	2,560	211	256	0.82×
conv3 32->64 k3	6,144	394	614	0.64×
linear 64->6	384	278	38	7.32×
total	10,096	1,010	1,010	1.00×



Same budget of 1,010 weights, redistributed. ERK takes 220 from conv3 and gives 240 to the classifier.

CONCEPT — why the linear result is the one that matters

Under uniform 90% the classifier gets 38 weights — roughly six per output class, drawn at random from 64 input features. Several classes would likely end up with almost no useful input, and no amount of RigL rewiring repairs a layer starved at initialisation.

ERK notices the layer is small (384 weights) relative to what it connects (64 in, 6 out) and keeps 72% of it. The 240 extra weights come out of conv3, which has 6,144 and is the most redundant layer in the network.

That redistribution is *larger* on a small model like this than on the large networks ERK was designed for — because with only 10,096 weights, a starved layer has no slack anywhere else to compensate.

WARNING — use ERK, not uniform, when you run the experiment

If you report "RigL at 90% sparsity" using uniform distribution and the accuracy is poor, you have not learned that RigL fails under FQT. You have learned that a 38-weight classifier fails, which was predictable without running anything.

ERK is the distribution the RigL paper uses and the one your results should be compared against. Uniform is worth running only as a deliberate ablation, to show the difference.

examples/har_classifier/train_c.c — EDIT — how to call ERK

```
/* Call once, at model construction. This 0.90f is the only place a sparsity
 * level enters the system -- rigLStep never reads it. */
rigLLayerSpec_t sparse[] = {
    { conv1Mask,  9, 16, 7 },
    { conv2Mask, 16, 32, 5 },
    { conv3Mask, 32, 64, 3 },
    { linearMask, 64,  6, 1 }, /* kernel = 1 for a Linear layer */
};
rigLInitSparsityERK(sparse, sizeof(sparse) / sizeof(sparse[0]), 0.90f);

/* then make the invariant true before the first forward pass: */
for (size_t l = 0; l < numLayers; l++) {
    parameter_t *w = NULL;
    tensor_t    *m = NULL;
    if (rigLLayerSparseParts(model[l], &w, &m)) {
        rigLApplyToParameter(w, m);
    }
}
```


PART V

The Algorithm

`rigLStep`, and the two things around it that are easy to get wrong: which count to grow back, and what to do about the optimizer's memory of a weight it will never see again.

V.1 rigLLayerSparseParts

Asking whether a layer participates

IN PLAIN ENGLISH — why it has to ask rather than assume

The training loop dispatches through a function-pointer table, so layers are opaque to it. A ReLU has no weights; a pooling layer has none; a Linear layer might have weights but no mask if it was excluded from sparsification.

So rigLStep cannot walk `model[1]` and assume anything. It asks, and skips the layers that say no.

src/layer/RigLStep.c — NEW — the participation test

```
bool rigLLayerSparseParts(layer_t *layer, parameter_t **weightsOut,
                          tensor_t **maskOut) {
    if (layer == NULL || weightsOut == NULL || maskOut == NULL) {
        return false;
    }
    parameter_t *w = layerGetWeights(layer); /* NULL for ReLU, pooling, ... */
    if (w == NULL || w->param == NULL || w->grad == NULL) {
        return false;
    }
    tensor_t *m = layerGetMask(layer); /* NULL when the layer is dense */
    if (m == NULL || m->quantization == NULL || m->quantization->type != BOOL) {
        return false;
    }
    if (calcNumberOfElementsByTensor(w->param) !=
        calcNumberOfElementsByShape(m->shape)) {
        PRINT_ERROR("rigL: mask/weight element count mismatch");
        return false;
    }
    /* Only written on the success path, so a caller that ignores the return
     * value sees whatever it initialised these to rather than a half result. */
    *weightsOut = w;
    *maskOut = m;
    return true;
}
```

Variable	Type	What it refers to	Example
layer	layer_t *	One layer from the model array. Could be anything — ReLU, pool, Linear	model[10]
weightsOut	parameter_t **	Where to <i>write</i> the weights pointer if this layer qualifies	&weights
maskOut	tensor_t **	Where to write the mask pointer	&mask
w	parameter_t *	The layer's weights and gradients, or NULL if it has none	the Linear params
m	tensor_t *	The layer's packed mask, or NULL if it is dense	the packed bits

Line	Code	What it does, and why
	<pre>if (layer == NULL weightsOut == NULL maskOut == NULL) return false;</pre>	Three NULL checks in one condition. A caller passing NULL outputs is a programming error, but returning false is cheaper to debug than a crash inside the function.

Line	Code	What it does, and why
	<code>parameter_t *w = layerGetWeights(layer);</code>	Ask the layer for its trainable parameters. A ReLU or a pooling layer returns NULL — they have none.
	<code>if (w == NULL w->param == NULL w->grad == NULL) return false;</code>	Both halves must exist. A layer with weights but no gradient tensor cannot participate, because the grow phase reads gradients.
	<code>tensor_t *m = layerGetMask(layer);</code>	Ask for the mask. NULL means this layer was deliberately excluded from sparsification — a legitimate configuration, not an error.
	<code>if (m == NULL m->quantization == NULL m->quantization->type != BOOL)</code>	The mask must be a packed BOOL tensor. Anything else and the bit indexing in <code>tensorBoolGet</code> would read the wrong memory.
	<code>if (calcNumberOfElements ByTensor(w->param) != calcNumberOfElementsByShape(m->shape))</code>	The check worth having. One mask bit per weight, exactly. A mismatch means the mask was built from the wrong shape, and every index would refer to a different weight than intended.
	<code>PRINT_ERROR("rigL: mask/weight element count mismatch");</code>	Unlike the earlier returns, this one reports. The others are ordinary "this layer does not participate"; this one is a real misconfiguration.
	<code>*weightsOut = w; *maskOut = m; return true;</code>	Written only here, on the success path. Every earlier return leaves the caller's variables untouched — see the tip below.

WORKED EXAMPLE — the same call on four different HAR layers

The training loop calls this for all twelve layers on every step. Most say no.

Layer	layerGetWeights	layerGetMask	Counts match?	Returns
model[1] ReLU	NULL	—	—	false at the first check
model[2] MaxPool	NULL	—	—	false
model[0] Conv1d	the 1,008 weights	the 1,008-bit mask	1,008 = 1,008	true
model[10] Linear	the 384 weights	the 384-bit mask	384 = 384	true
a Linear left dense	the weights	NULL	—	false — excluded on purpose
a misconfigured layer	1,008 weights	a 512-bit mask	1,008 != 512	false, and reports

NOTE — five of the six answers are "no", and only one is an error

A ReLU has no weights. A pooling layer has none. A Linear layer deliberately excluded from sparsification has no mask. All three are ordinary configurations and return `false` silently.

Only the last row prints anything, because a mask of the wrong length is the one case that cannot be intentional — and if it went undetected, every index would refer to a different weight than the caller believes.

DEFINITION — the out-parameter pattern

A C function returns one value. When you need several — here a success flag plus two pointers — the convention is to return the flag and take **out-parameters**: pointers to variables the caller owns, which the function fills in.

Hence `parameter_t **weightsOut`, a pointer to a pointer. The caller passes `&weights`; the function writes through it. The double star is not complexity for its own sake — it is the only way to change what the caller's pointer points at.

TIP — leave out-parameters untouched on failure

Every early return happens *before* either output is written. A caller who ignores the return value and uses the outputs anyway gets whatever it initialised them to — NULL, if they followed the pattern in Part VI — rather than a half-filled result. Writing outputs only on the success path is what makes the header comment true.

NOTE — this one is on the layer side of the split

It calls `layerGetWeights` and `layerGetMask`, so it needs `Layer.h` and belongs in `src/layer/RigLStep.c`. Everything in Part III and Part IV stays on the tensor side. Part II.5 has the reasoning.

V.2 rigLStep

The algorithm, complete

src/layer/RigLStep.c — NEW — opening and the drop phase

```
void rigLStep(layer_t **model, size_t numLayers, size_t step, size_t totalSteps,
              float alphaInit, optimizer_t *optim) {
    if (model == NULL || numLayers == 0u) {
        return;
    }

    float progress = (float)step / (float)(totalSteps > 0u ? totalSteps : 1u);
    float alpha    = alphaInit * 0.5f * (1.0f + cosf(3.14159265f * progress));

    for (size_t l = 0; l < numLayers; l++) {

        parameter_t *weights = NULL;
        tensor_t *mask = NULL;
        if (!rigLLayerSparseParts(model[l], &weights, &mask)) {
            continue;
        }

        tensor_t *w = getParamFromParameter(weights);
        tensor_t *g = getGradFromParameter(weights);
        size_t n = calcNumberOfElementsByTensor(w);

        size_t numActive = tensorMaskCountActive(mask);
        size_t k = (size_t)(alpha * (float)numActive);
        if (k == 0u) {
            continue; /* alpha has decayed to nothing; topology is final */
        }

        /* ---- DROP: clear the k smallest-magnitude active weights ---- */
        float dropT = rigLKthSmallestActive(w, mask, k);
        size_t nDropped = 0;
        for (size_t i = 0; i < n; i++) {
            if (!tensorBoolGet(mask, i)) {
                continue;
            }
            float wi = fabsf(readBytesAsFloat(&w->data[i * sizeof(float)]));
            if (wi <= dropT) {
                tensorBoolSet(mask, i, false);
                /* Inactive implies zero optimizer state, so a weight grown
                 * later cannot inherit a stale moment estimate. See V.4. */
                optimizerClearElementState(optim, weights, i);
                nDropped++;
            }
        }

    }

    if (nDropped == 0u) {
        continue;
    }
}
```

Variable	Type	What it refers to	Example
model	layer_t **	An array of pointers, one per layer	12 entries for HAR
step	size_t	How many training steps have happened so far	15000
totalSteps	size_t	How many the whole run will take	20000

Variable	Type	What it refers to	Example
alphaInit	float	The rewiring fraction at step 0	0.3f
optim	optimizer_t *	Needed only to clear momentum for pruned weights	the SGD instance
progress	float	step / totalSteps	0.75f
alpha	float	The rewiring fraction right now	0.044f
numActive	size_t	Active weights in this layer, measured from the mask	394
k	size_t	How many to drop — a request, not a result	17
dropT	float	Magnitude at or below which weights get pruned	0.0021f
nDropped	size_t	How many <i>actually</i> went. Usually more than k	21

Line	Code	What it does, and why
	<pre>float progress = (float)step / (float)(totalSteps > 0u ? totalSteps : 1u);</pre>	The ternary guards against division by zero. Substituting 1 makes progress equal step, which is wrong but finite — and finite-wrong is far easier to debug than a NaN that propagates into every weight.
	<pre>float alpha = alphaInit * 0.5f * (1.0f + cosf(3.14159265f * progress));</pre>	cosf, not cos; 0.5f and 1.0f with suffixes. Every literal and every call is single-precision, so nothing on this line promotes to double.
	<pre>if (!rigLLayerSparseParts(model[l], &weights, &mask)) { continue; }</pre>	Skip layers that do not participate. Note &weights — the address of the local, so the helper can write through it.
	<pre>size_t numActive = tensorMaskCountActive(mask);</pre>	Measured from the mask, not derived from a stored target. This is what makes the sparsity self-maintaining: RigL conserves whatever it finds.
	<pre>size_t k = (size_t)(alpha * (float)numActive);</pre>	Truncation toward zero, so a fractional result rounds <i>down</i> . Conservative, which is the right direction for a destructive operation.
	<pre>if (k == 0u) { continue; }</pre>	Essential. Late in training alpha approaches 0, so k becomes 0. Continuing would ask the threshold search for the zeroth smallest element, which is meaningless. Skipping is exactly what a zero alpha means.
	<pre>float dropT = rigLKthSmallestActive(w, mask, k);</pre>	The threshold. One O(n) call, 1 KB of scratch — Part III.7.
	<pre>if (!tensorBoolGet(mask, i)) { continue; }</pre>	Only active weights can be dropped. Inactive ones are already at 0.0 and have nothing to contribute.
	<pre>float wi = fabsf(readBytesAsFloat(&w->data[i * sizeof(float)]));</pre>	readBytesAsFloat rather than a pointer cast — aliasing-safe and alignment-safe. fabsf because magnitude is what matters; a weight of -0.4 contributes as much as +0.4.
	<pre>if (wi <= dropT)</pre>	<=, not <. This is what produces more than k drops when values tie — and under int8, ties are the normal case. Part V.3.
	<pre>optimizerClearElementState(optim, weights, i);</pre>	Clear the momentum for this weight now that it is inactive, so a weight grown later cannot inherit it. Part V.4 explains why here rather than in the grow phase.
	<pre>if (nDropped == 0u) { continue; }</pre>	Nothing went, so there is nothing to grow back and no point calling the second search. Also prevents asking it for the zeroth largest.

WORKED EXAMPLE — alpha and k at four points in a 20,000-step run

With $\alpha_{\text{Init}} = 0.3$, $\Delta T = 100$, and conv3 holding 394 active weights after ERK.

step	progress	cos(pixp)	alpha	k = alpha x 394	What happens
0	0.00	+1.000	0.3000	118	aggressive rewiring, 30% of the layer
5,000	0.25	+0.707	0.2561	100	still exploring freely
10,000	0.50	0.000	0.1500	59	half the initial rate
15,000	0.75	-0.707	0.0440	17	mostly settled — the traced update
19,900	0.995	-1.000	0.0002	0	k = 0, so the layer is skipped

NOTE — the last row is why the $k == 0$ guard exists

By step 19,900 alpha has decayed far enough that $\alpha \times 394$ truncates to zero. Without the guard the code would call `rigLKthSmallestActive` asking for the zeroth smallest element — which returns -1.0f by design, so nothing would be dropped anyway, but two full passes over the tensor would be wasted first.

Skipping is also the semantically right answer: a zero alpha means "the topology is final", and doing nothing is exactly that.

[src/layer/RigLStep.c](#) — NEW — grow phase, close, assertions

```

/* ---- GROW: set exactly nDropped largest-gradient inactive weights ----
 * nDropped, NOT k. See V.3. */
float growT = rigLKthLargestInactive(g, mask, nDropped);
size_t nGrown = 0;
for (size_t i = 0; i < n && nGrown < nDropped; i++) {
    if (tensorBoolGet(mask, i)) {
        continue;
    }
    float gi = fabsf(readBytesAsFloat(&g->data[i * sizeof(float)]));
    if (gi >= growT) {
        tensorBoolSet(mask, i, true);
        nGrown++;
    }
}

rigLApplyToParameter(weights, mask);
mask->sparsity = NULL; /* invalidate any cached active count */

#ifdef NDEBUG
size_t after = tensorMaskCountActive(mask);
if (after != numActive) {
    PRINT_ERROR("rigL L%zu: sparsity drift %zu -> %zu"
                " (dropped %zu, grown %zu)",
                l, numActive, after, nDropped, nGrown);
    exit(1);
}
if (growT <= 0.0f) {
    PRINT_ERROR("rigL L%zu: grow threshold %.6f -- inactive gradients"
                " look zeroed, the grow phase is selecting arbitrarily",
                l, (double)growT);
    exit(1);
}
}
#endif
PRINT_INFO("RigL L%zu: active %zu, k %zu, dropped %zu, grown %zu", l,
           numActive, k, nDropped, nGrown);
}
}

```

Line	Code	What it does, and why
	<code>float growT = rigLKthLargestInactive(g, mask, nDropped);</code>	Three differences from the drop call, all deliberate. It reads <i>g</i> not <i>w</i> ; it selects from the inactive side; and the count is <i>nDropped</i> , not <i>k</i> .
	<code>for (size_t i = 0; i < n && nGrown < nDropped; i++)</code>	Two exit conditions. The second is the tie handling on the grow side: stop the moment enough have been grown, even if more clear the threshold.
	<code>if (tensorBoolGet(mask, i)) { continue; }</code>	Skip active weights. Note this reads the mask <i>after</i> the drop phase modified it, so weights just pruned are eligible to be grown straight back — which is correct if their gradient is large.
	<code>float gi = fabsf(readBytesAsFloat(&g->data[i * sizeof(float)]));</code>	The gradient magnitude. The drop phase judged by weight, this one by gradient. Same index, same weight, different tensor.
	<code>if (gi >= growT)</code>	<code>>=</code> , because here we want the <i>largest</i> . The comparison direction flips relative to the drop phase, matching the flipped bucket walk in the search.
	<code>rigLApplyToParameter(weights, mask);</code>	Enforce the invariant: write real 0.0 into everything now inactive. Without this the newly pruned weights keep their old values and the matmul keeps using them.
	<code>mask->sparsity = NULL;</code>	Cache invalidation. Any cached active count is now stale. Setting the pointer to NULL forces the next <code>tensorMaskCountActive</code> to recount rather than return a wrong number.

Line	Code	What it does, and why
	<code>#ifndef NDEBUG</code>	Everything to <code>#endif</code> vanishes in a release build. The checks cost nothing where it matters and catch the two silent failures where it does not.
	<code>if (after != numActive)</code>	The conservation assertion. Fires the first time the drop and grow counts disagree — which is the tie-handling bug, and which would otherwise take hundreds of updates to become visible.
	<code>if (growT <= 0.0f)</code>	The grow-signal assertion. A threshold of exactly zero means every inactive gradient tied at zero, so the selection was arbitrary. Catches the VI.3 trap.
	<code>PRINT_INFO("RigL L%zu: active %zu, k %zu, dropped %zu, grown %zu")</code>	The line you watch during a run. dropped and grown must be equal on every line; if they ever differ, the sparsity has drifted.

WORKED EXAMPLE — one complete rigLStep on the running example

The whole function, start to finish, on the 16-element tensor from II.7 with $k = 3$.

Stage	Value	Where it came from
numActive	8	<code>popcount(0x55) + popcount(0x55)</code>
k	3	$\alpha \times 8$, truncated
maxAbs (active w)	0.4700	element 10
dropT	0.151137	bucket 81, upper edge
dropped	elements 4, 6, 14	$ w = 0.07, 0.11, 0.15$
nDropped	3	counted, not assumed
mask after drop	<code>0x05 0x15</code>	5 active
maxAbs (inactive g)	0.3100	element 4
growT	0.258941	bucket 212, lower edge
grown	elements 4, 9, 15	$ g = 0.310, 0.280, 0.260$
nGrown	3	loop stopped at nDropped
mask after grow	<code>0x15 0x97</code>	8 active — conserved
then	<code>rigLApplyToParameter</code>	writes 0.0 into 6 and 14
and	<code>mask->sparsity = NULL</code>	forces a recount next time

The log line this produces. **dropped == grown, which is the property to watch for.**

```
RigL L0: active 8, k 3, dropped 3, grown 3
```

TRACED — and the debug assertions both pass

`after = tensorMaskCountActive(mask) = popcount(0x15) + popcount(0x97) = 3 + 5 = 8.`
`after == numActive — 8 == 8 ok no drift.`

`growT = 0.258941 > 0.0f ok the inactive gradients carry real signal, so the grow phase made a genuine choice rather than an arbitrary one.`

V.3 Ties: why nDropped and not k

The single most important line

IN PLAIN ENGLISH — the drop count is a request, not a result

k is how many weights RigL *asked* to drop. nDropped is how many actually went. Under int8 they are almost never equal.

A quantised weight holds one of 256 values, so its magnitude is one of at most 128. In a layer of 6,144 weights that means roughly 48 weights share each magnitude. The threshold lands on a value many weights hold exactly, and $|w| \leq \text{dropT}$ is true for all of them.

TRACED — one update on conv3, with ties

numActive = 394 after ERK, alpha = 0.044 late in training, so k = 17.

Stage	Value	Note
k requested	17	$\alpha \times \text{numActive}$, truncated
dropT found	0.0021	a value several weights hold exactly
weights with $ w \leq \text{dropT}$	21	four extra, all tied at the threshold
nDropped	21	what actually left
grow target	21	nDropped, not k
nGrown	21	loop stops here even if more clear growT
numActive afterwards	394	unchanged — exactly as intended

CONCEPT — what goes wrong if you grow k

The layer ends each update four weights sparser than it started. Nothing errors. The log line still prints. dropped and grown differ by four, which nobody notices in a scrolling log.

Over 200 updates that is 800 weights lost from a layer that started with 394 — the layer is gone long before then, and the reported sparsity from your config file still says 90%. You would attribute the resulting accuracy collapse to the algorithm rather than to the bookkeeping.

Growing nDropped makes the count identical before and after, on every update, by construction rather than by hope.



One update on conv3. The four-weight difference compounds: after 100 updates the wrong version has lost every weight it had.

TIP — the assertion that catches it in milliseconds

The NDEBUG-gated check in V.2 recounts the mask after `rigLApplyToParameter` and compares against `numActive`. It fires on the first update where the counts differ.

The harness in Part VIII runs 50 consecutive rewirings and confirms the count never moves. That is far cheaper than discovering it from an accuracy number three hours in.

V.4 Optimizer state for dropped and grown weights

The third thing that is easy to omit

IN PLAIN ENGLISH — what the optimizer remembers

SGD with momentum keeps one number per weight. AdamW keeps two. They summarise that weight's recent gradient history and persist across steps.

RigL changes which weights exist. The optimizer does not know that, so it carries a pruned weight's history forward indefinitely and hands it to whatever the mask does next.

WARNING — a grown weight inherits a stranger's history

When RigL grows weight i , that weight starts at 0.0 and begins learning. But $m[i]$ and $v[i]$ still hold whatever accumulated the last time that index was active — possibly thousands of steps ago, under a completely different topology.

AdamW's update is roughly $m / \sqrt{v}$. A stale m with a small v produces a large first step in a direction chosen by ancient history. The fresh connection is thrown somewhere arbitrary before it has seen one relevant gradient.

DEFINITION — what the RigL paper specifies

Newly grown weights start at **zero value with zero momentum**. The zero value is what `rigLApplyToParameter` already guarantees. The zero momentum is the part nothing in this codebase currently does.

src/optimizer/include/Optimizer.h — EDIT — the new hook

```
/* src/optimizer/include/Optimizer.h */

/* Zero every piece of per-element state this optimizer keeps for parameter p
 * at flat index i. A no-op for optimizers that keep none (plain SGD).
 * Called by rigLStep on every weight it prunes, so that "inactive" implies
 * "zero optimizer state" at all times. */
void optimizerClearElementState(optimizer_t *optim, parameter_t *p, size_t i);
```

src/optimizer/AdamW.c and Sgd.c — EDIT — the two implementations

```

/* src/optimizer/AdamW.c */
void adamWClearElementState(adamW_t *a, parameter_t *p, size_t i) {
    tensor_t *m = adamWMomentFor(a, p);    /* first moment */
    tensor_t *v = adamWVarianceFor(a, p);    /* second moment */
    if (m != NULL) {
        writeFloatToByteArray(0.0f, &m->data[i * sizeof(float)]);
    }
    if (v != NULL) {
        writeFloatToByteArray(0.0f, &v->data[i * sizeof(float)]);
    }
}

/* src/optimizer/Sgd.c */
void sgdClearElementState(sgd_t *s, parameter_t *p, size_t i) {
    if (s->momentumFactor == 0.0f) {
        return;                                /* plain SGD keeps no state */
    }
    tensor_t *state = sgdMomentumFor(s, p);
    if (state != NULL) {
        writeFloatToByteArray(0.0f, &state->data[i * sizeof(float)]);
    }
}

```

Variable	Type	What it refers to	Example
optim	optimizer_t *	The optimizer instance owning the state tensors	the SGD or AdamW object
p	parameter_t *	Which parameter the weight belongs to	the layer weights
i	size_t	Which element, flat — the same index the mask uses	1200
m	tensor_t *	AdamW first moment: a running average of the gradient	same shape as weights
v	tensor_t *	AdamW second moment: a running average of the gradient squared	same shape again

Line	Code	What it does, and why
	void optimizerClearElementState(optimizer_t *optim, parameter_t *p, size_t i)	The dispatch entry point, in the header. Takes the index rather than a range, because RigL clears one weight at a time as it prunes.
	tensor_t *m = adamWMomentFor(a, p);	Find the moment tensor for this parameter. AdamW keeps one pair per parameter, not one pair globally.
	if (m != NULL) { writeFloatToByteArray(0.0f, &m->data[i * sizeof(float)]); }	Zero the first moment. The NULL check matters because state tensors are allocated lazily in some configurations.
	if (v != NULL) { ... }	And the second. Both must go — clearing only m would leave v large, which makes the first post-grow step unusually <i>small</i> rather than unusually large. Wrong in the other direction.
	if (s->momentumFactor == 0.0f) { return; }	Plain SGD keeps no per-element state at all, so there is nothing to clear. An early return rather than a NULL check, because the configuration is known.
	tensor_t *state = sgdMomentumFor(s, p);	Momentum SGD keeps one buffer per parameter. Same shape as the weights, same indexing.

CONCEPT — why clear on the drop, not on the grow

You could clear the state when a weight is grown. Clearing it when a weight is **dropped** is simpler and safer, for two reasons.

First, a dropped weight's state is dead anyway — nothing will use it while the weight is inactive, so clearing costs nothing and cannot break anything. Second, it makes the invariant *"inactive implies zero value and zero optimizer state"* true at all times, so the grow phase needs no corresponding action and cannot forget to take one.

One rule maintained in one place beats two actions that must stay coordinated.

Event	Weight value	Optimizer state	Handled by
dropped	forced to 0.0	cleared	rigLApplyToParameter + the hook
stays inactive	held at 0.0 each step	stays zero	the per-step reapplication
grown	starts at 0.0	already zero	nothing needed — that is the point
stays active	updated normally	updated normally	the optimizer

PART VI

Where FQT and RigL Collide

Now that the algorithm is on the page, the question from Part I.6 can be made precise. The mechanism rewiring depends on, the three places quantisation attacks it, and the one failure mode that leaves no trace.

VI.1 What "rewiring" actually means

The mechanism, from first principles

IN PLAIN ENGLISH — the problem rewiring solves

Suppose you decide in advance which 10% of the weights a network may use, and train only those. You have made a guess about which connections matter, before seeing any data. If the guess is bad, nothing in training can correct it — the other 90% are not merely unused, they are unreachable.

Rewiring is the mechanism that makes the guess revisable. Periodically the network gets to change *which* connections it uses, without changing *how many*.

DEFINITION — rewiring

Switching some active weights off and an equal number of inactive weights on, at the same moment, so the total stays constant. One rewiring is a change to the mask and nothing else — no weight is deleted, no memory is reallocated, and the layer's shape never changes.

It is the one operation that distinguishes RigL from static sparse training. Everything else the two have in common.

DEFINITION — the two criteria, and why they differ

Drop: the active weights with the *smallest magnitude*. A weight near zero is barely contributing, so removing it costs little. This is a statement about the *present*.

Grow: the inactive weights with the *largest gradient magnitude*. A large gradient on a dormant connection means the loss surface is steep in that direction — if this connection existed, it would move a long way. This is a statement about the *future*.

Using the same criterion for both would be incoherent: an inactive weight is held at exactly 0.0, so ranking inactive weights by magnitude ranks them all equal. The gradient is the only signal available about a connection that does not yet exist.

CONCEPT — rewiring is a search over topologies

Think of the space of all possible masks at 10% density. For a 6,144-weight layer that space has more elements than there are atoms in the observable universe, and static sparse training picks one point in it at random and stays there forever.

RigL performs a **local search**: at each update it moves to a neighbouring mask, guided by two gradient-derived signals. The cosine schedule makes the steps large early and small late, which is the standard shape for a search that should explore then converge.

That framing is why the question "does rewiring survive quantisation" is not rhetorical. A search is only as good as its guidance signal. Quantise the signals and the search may still run — taking steps, conserving the count, printing correct logs — while moving essentially at random.

static sparse: pick one mask at random, keep it forever

a bad initial guess is never corrected

RigL: pick one mask, then search locally every ΔT steps

drop k smallest $|w|$ - a claim about the present

grow k largest $|g|$ - a claim about the future

count conserved, so the memory budget never moves

The only structural difference between the two. Everything downstream of the mask is identical.

VI.2 The three collision points

Three points, each a place the combination could fail

IN PLAIN ENGLISH — why this is not simply an implementation question

Writing code that does both is straightforward — Parts III to VII are that code, and it compiles and runs. The question is whether the *mechanism* still functions when every quantity it depends on has been reduced to 256 levels.

There are exactly three places the two techniques touch, and each is a place the answer could be no.

Collision	What RigL assumes	What FQT does to it
1. Drop ranking	active weight magnitudes are distinguishable, so "the k smallest" is a well-defined set	int8 gives 256 values, at most 128 distinct magnitudes. In a 6,144-weight layer roughly 48 weights share each one
2. Grow ranking	inactive gradient magnitudes are distinguishable, so "the k largest" is a meaningful ranking	gradients have far wider dynamic range than weights; a coarse gradient scale can collapse most of them into a few buckets
3. Grown-weight escape	a grown weight starts at 0.0 and then <i>moves</i> , becoming useful within ΔT steps	under round-to-nearest an update below half a quantisation step rounds back to zero, so it may never move at all

NOTE — collision 1 is already visible in this document

Part V.3 worked through it: asking to drop 17 weights actually drops 21, because the threshold lands on a value several weights hold exactly. The nDropped accounting handles the *bookkeeping* consequence perfectly.

What it does not address is the *selection* consequence. Within a tied block RigL is no longer choosing the least useful weights — it takes whichever it meets first in flat index order. At 90% sparsity on conv3, k is 17 and a tied block may be 48 wide, so the majority of each drop decision could be arbitrary.

CONCEPT — collision 3 is the one worth building the thesis on

The first two degrade the search gradually: a noisier ranking gives a worse choice, and the algorithm still makes progress, just less of it. They are quantitative.

The third is qualitative. If a grown weight cannot leave zero, growth accomplishes nothing at all — and the failure is invisible, because the mask is updated correctly, the count is conserved, and every assertion in Part VIII passes. Section VI.3 works it through.

VI.3 The dead zone, and stillborn weights

The failure mode that leaves no trace

DEFINITION — the quantisation dead zone

Under round-to-nearest, any update smaller than **half a quantisation step** rounds back to where it started. The weight does not move a little; it does not move at all.

For a weight stored int8 with step s , the dead zone is every update of magnitude below $s/2$.

DEFINITION — a stillborn weight

A weight that RigL has grown — its mask bit is set, so every count and every log line treats it as active — but which is still at exactly 0.0 when the next rewiring arrives, because every update it received fell inside the dead zone.

It occupies a slot in the sparsity budget and contributes nothing. And because its magnitude is exactly zero, it is the *first* thing the next drop phase removes.

IN PLAIN ENGLISH — the cycle this creates

RigL grows a connection. The connection cannot move, so at the next update it has the smallest magnitude in the layer and is dropped immediately. RigL grows a different one, which also cannot move, and drops that too.

The algorithm churns: rewiring busily every ΔT steps, never building anything. And **every diagnostic in this document still passes** — the counts conserve, dropped == grown on every line, both Part VIII assertions are silent. Accuracy is poor and nothing says why.

RigL grows weight i - mask bit set, value 0.0



update $lr \cdot g$ is below $s/2$ - rounds back to 0.0



after ΔT steps the weight is still exactly 0.0



next drop phase: $|w| = 0$ is the smallest possible - dropped first



RigL grows a different weight - same outcome



counts conserved, logs correct, assertions pass, nothing learned

The thrashing cycle. Note the last row: this failure is invisible to every check the document has defined so far.

The arithmetic

IN PLAIN ENGLISH — when can a grown weight escape in one step?

It moves only if the update is at least half a step. Rearranging $lr \times g \geq s/2$ gives the gradient a grown weight needs:

$$g \geq s / (2 \times lr)$$

WORKED EXAMPLE — the escape threshold for a typical conv layer

Take a weight range of ± 0.4 , so the int8 step is $s = 0.4 / 127 = 0.003150$, and the dead zone is anything below $s/2 = 0.001575$. With $lr = 0.01$:

$$g \geq 0.003150 / (2 \times 0.01) = \mathbf{0.1575}$$

A grown weight needs a gradient of at least 0.16 to move at all under round-to-nearest. Gradients that large are not typical outside the first few epochs.

gradient g	update lr×g	escapes under RTN?	SR probability per step	SR movement over deltaT = 100
0.300	0.003000	yes	0.952	95 steps = 0.300 real
0.160	0.001600	yes, just	0.508	51 steps = 0.160 real
0.050	0.000500	no — stillborn	0.159	16 steps = 0.050 real
0.010	0.000100	no — stillborn	0.032	3 steps = 0.010 real

WARNING — and the threshold depends on a width you control, not a fixed int8

The 0.1575 above assumes 8-bit operands. The codebase does not default to that. Quantization.h sets ODT_SYM_OPERAND_QMAXBITS to **12**, with a stated reason: int12 × int12 products stay inside an int32 accumulator, so no int64 arithmetic is needed on a 32-bit core.

Twelve bits is 2,047 levels rather than 127, so the step is sixteen times finer and the dead zone sixteen times shallower.

operand width	levels	step s	dead zone s/2	escape threshold $g \geq s/(2 \text{ lr})$
int8	127	0.003150	0.001575	0.1575
int12 (the default)	2,047	0.000195	0.0000977	0.0098
int16	32,767	0.0000122	0.0000061	0.00061

CONCEPT — this sharpens the hypothesis rather than weakening it

At the codebase default of 12 bits a grown weight needs a gradient of about 0.010 to escape, against 0.16 at int8. The stillborn pathology is far less severe than the int8 arithmetic alone suggests, and a naive run at default settings might not show it at all.

But the research question specifies **int8** forward, backward and gradient. Getting genuine int8 means building with -DODT_SYM_OPERAND_QMAXBITS=8, or using packed SYM with qBits = 8.

So the dead zone is a function of a knob you set — which makes operand width a third axis for the experiment grid, and a direct test of the mechanism. Sweep it over {8, 12, 16} and the stillborn fraction should fall as the width grows. If it does, that is confirmation; if it is flat, the dead-zone explanation is wrong.

NOTE — and the tie argument in V.3 scales the same way

At 8 bits there are at most 128 distinct magnitudes, so ties at the drop threshold dominate. At 12 bits there are 2,048 — sixteen times more — so ties are correspondingly rarer. `nDropped` is still the right count to grow back, but it will exceed `k` by much less.

Both collision points in Part VI.2 are therefore *graded* by operand width rather than being present or absent. That is a more useful thing to report than a binary answer.

WARNING — round-to-nearest has no memory, which is what makes it absolute

You might expect a stillborn weight to inch forward over many steps. It does not, and the reason is worth being precise about.

Each step recomputes $w = \lfloor r \times g \rfloor$ starting from the *stored* value, which is 0.0. The result is 0.0005, which rounds to 0.0 and is stored. Next step starts from 0.0 again. There is no accumulator, so a hundred steps produce exactly the same outcome as one. The weight is not moving slowly — it is frozen.

VI.4 Two escape routes, and a tension with Part V.4

Route one: stochastic rounding

IN PLAIN ENGLISH — why SR changes the picture completely

Stochastic rounding does not round to the nearest step. It rounds up with probability equal to how far through the step the value sits. An update of 0.159 of a step moves the weight with probability 0.159 and leaves it otherwise.

Wrong on every individual occasion, and **unbiased in expectation**. So the accumulated movement over many steps is exactly the movement the weight should have had — look at the last column of the table above: 16 steps of 0.00315 is 0.0504, and $100 \times \text{lr} \times g$ is 0.0500. They agree, because that is what unbiased means.

CONCEPT — SR converts a hard threshold into a rate

Under round-to-nearest there is a cliff at $g = 0.1575$: above it the weight moves every step, below it never. Under stochastic rounding there is no cliff — a smaller gradient simply means slower movement, proportionally.

That is precisely what a growth mechanism needs. RigL grows weights it expects to become useful; it does not require them to become useful *immediately*. A rate is compatible with that. A cliff is not.

Route two: momentum — and why Part V.4 removes it

IN PLAIN ENGLISH — momentum accumulates even when the weight cannot

SGD with momentum keeps a running state, $\text{state} = 0.9 \times \text{state} + g$. That state lives in the optimizer, not in the quantised weight, so it grows without restriction — toward $g / (1 - 0.9) = 10g$.

So even a sub-threshold gradient eventually produces a supra-threshold *update*, once the momentum has built up.

WORKED EXAMPLE — a stillborn weight rescued by momentum — $g = 0.05$, $\text{lr} = 0.01$, $\text{beta} = 0.9$

Half-step is 0.001575. The weight escapes the moment $\text{lr} \times \text{state}$ exceeds it.

step	momentum state	$\text{lr} \times \text{state}$	vs half-step 0.001575
1	0.0500	0.000500	dead
5	0.2048	0.002048	escapes
10	0.3257	0.003257	escapes
20	0.4392	0.004392	escapes
steady state	0.5000	0.005000	3.2x the threshold

WARNING — but Part V.4 zeroes exactly this state, on purpose

Section V.4 clears the optimizer state for every weight RigL prunes, so that "inactive implies zero optimizer state" holds at all times. That is faithful to the RigL paper, which specifies grown weights start at *zero value with zero momentum*, and it prevents a grown weight inheriting a stale moment estimate from a different era of training.

It also deletes the mechanism that would have rescued it from the dead zone. A grown weight starts with zero momentum and must spend five to ten steps rebuilding it before it can move at all — and only if deltaT is long enough to allow that.

CONCEPT — two correct decisions in tension — which is what a thesis is for

The V.4 reset is right by RigL's own specification. The momentum accumulation is right by the optimizer's. Under float32 they never interact, because a grown weight can move on its first step regardless.

Under FQT they interact, and one of them has to give. That is not a bug in either — it is a genuine consequence of composing two techniques that were designed independently, and it is exactly the kind of finding a thesis on combining them should report.

It also generates a concrete prediction: **under FQT there is a minimum viable deltaT, and it is larger than in float32**. Too short and grown weights are judged before they have had time to escape.

PART VII

Integration

Three insertions into the training loop. One of them runs every step rather than every ΔT , is the only one that lives in a file not named after what it does, and is the one most often left out.

VII.1 The three insertions

examples/har_classifier/train_c.c — EDIT — the complete integration

```
/* --- once, at model construction ----- */
tensorMaskInitDense(mask, maskStorage, weightShape, &boolQ); /* III.4 */
rigLInitSparsityERK(sparse, numSparse, 0.90f); /* IV.3 */
for (size_t l = 0; l < numLayers; l++) {
    parameter_t *w = NULL; tensor_t *m = NULL;
    if (rigLLayerSparseParts(model[l], &w, &m)) {
        rigLApplyToParameter(w, m); /* invariant true at step 0 */
    }
}

/* --- every training step ----- */
forward(model, x);
computeLoss(...);
backward(model); /* gradients for ALL weights, mask ignored */

optimizerStep(optim); /* updates everything, resurrecting pruned weights */

for (size_t l = 0; l < numLayers; l++) { /* <-- EVERY step */
    parameter_t *w = NULL; tensor_t *m = NULL;
    if (rigLLayerSparseParts(model[l], &w, &m)) {
        rigLApplyToParameter(w, m);
    }
}

/* --- every deltaT steps ----- */
if (globalStep % cfg->updateInterval == 0u) {
    rigLStep(model, numLayers, globalStep, totalSteps, cfg->alphaInit, optim);
}
globalStep++;
```

Line	Code	What it does, and why
	<code>tensorMaskInitDense(mask, maskStorage, weightShape, &boolQ);</code>	Once per sparsifiable layer, at construction. Gives an all-ones mask — nothing pruned yet. Part III.4.
	<code>rigLInitSparsityERK(sparse, numSparse, 0.90f);</code>	Once for the whole model. This 0.90f is the only place a sparsity level enters the system. Part IV.3.
	<code>rigLApplyToParameter(w, m);</code> [setup loop]	Makes the invariant true before the first forward pass. Without it the newly pruned weights keep their initial random values and the first forward pass is not sparse at all.
	<code>backward(model);</code>	Computes gradients for every weight, including pruned ones. The backward pass does not know the mask exists — and that is deliberate, because the grow phase needs those gradients. Part VI.3.
	<code>optimizerStep(optim);</code>	Applies the update to every weight, pruned ones included. It does not know about the mask either, so it resurrects them. Part VI.2.
	<code>rigLApplyToParameter(w, m);</code> [per-step loop]	The insertion most often left out. Runs every step, not every deltaT. Undoes the resurrection the previous line caused.
	<code>if (globalStep % cfg->updateInterval == 0u)</code>	The rewiring cadence. deltaT is typically 100: often enough to keep exploring, rare enough that grown weights get time to learn something before being judged again.

Line	Code	What it does, and why
	<code>rigLStep(model, numLayers, globalStep, totalSteps, cfg->alphaInit, optim);</code>	The topology update. Takes <code>globalStep</code> for the cosine schedule and <code>optim</code> for the state clearing.
	<code>globalStep++;</code>	Incremented after, so the first step is step 0 and progress starts at exactly 0. Also the value that must be checkpointed — Part VII.

Insertion	Frequency	Purpose	Cost, HAR model
<code>rigLApplyToParameter</code> per layer	every step	Undo the optimizer's resurrection	~70k cycles, 0.3 ms
<code>rigLStep</code>	every <code>deltaT</code> steps	Rewire the topology	~0.7 ms, amortised 7 us/step
optimizer-state reset	inside <code>rigLStep</code>	Give grown weights a clean start	negligible

VII.2 The optimizer resurrects pruned weights

Why the middle insertion exists

IN PLAIN ENGLISH — the mechanism

Backpropagation computes a gradient for every weight. It does not consult the mask — nothing in the backward pass knows the mask exists. So a pruned weight sitting at exactly 0.0 still has a perfectly ordinary non-zero gradient.

The optimizer does not consult the mask either. It applies that gradient: $w = w - lr \times g$ turns the stored 0.0 into a small non-zero value. After one step the invariant is broken.

backward computes gradients for ALL weights, mask ignored

optimizer applies them to ALL weights, mask ignored

a pruned weight at 0.0 becomes $0.0 - lr * g$, i.e. non-zero

matmul trusts the value, never reads the mask (VI.4)

the layer is no longer sparse, and nothing reports it

How sparsity leaks away in a single step if the invariant is not restored after the optimizer.

WARNING — the failure is completely silent

The mask still says the weight is inactive. The counts still conserve. The RigL log lines still read correctly. And the matmul — which trusts the stored value and never reads the mask, *by design* — quietly includes it in the arithmetic.

Your reported sparsity is a number from the mask that no longer describes what the network is computing. This is the single most likely way to get a RigL implementation that appears to run correctly and measures nothing.

TIP — the assertion that catches it on step 1

In a debug build, after the per-step reapplication, walk the layer and assert that every element whose mask bit is 0 reads back as exactly 0.0. If you have forgotten the insertion entirely it fails on the very first step — the best possible time to find out. Part VIII.1.

VII.3 The trap: do not zero the gradients instead

A shortcut that destroys the algorithm

IN PLAIN ENGLISH — the tempting alternative

Restoring the invariant with a pass over the weights on every step costs a fraction of a millisecond. An obvious-looking cheaper option is to zero the *gradients* of masked weights **before** the optimizer runs. Then $w = w - \text{lr} \times 0 = w$, the weight never moves, and no second pass is needed.

It looks equivalent. It is not, and the failure is silent in a different way.

DEFECT — zeroing inactive gradients destroys the grow criterion

RigL's grow phase selects inactive weights by **largest gradient magnitude**. That is the entire mechanism by which the network signals which dormant connections it wants back.

Zero those gradients and every inactive weight has magnitude 0. They are all identical, `rigLKthLargestInactive` returns 0, and `gi >= growT` is true for everything — so the grow loop takes whichever weights it visits first, in flat index order.

RigL degrades into "prune the smallest, grow back an arbitrary set". It still runs. The sparsity is still conserved. The log line still looks correct. And the algorithm is doing nothing but adding noise.

Where the invariant is restored	Weights stay 0?	Grow still works?	Verdict
zero the gradients, before the optimizer	yes	no	breaks RigL silently
zero the weights, after the optimizer	yes	yes	correct
optimizer skips masked weights internally	yes	yes	correct, and cheaper

CONCEPT — the general rule this illustrates

The gradients of inactive weights carry information RigL needs and the optimizer must ignore. Those are two different requirements, and it is tempting to satisfy both by deleting the information — which satisfies the optimizer and starves RigL.

The invariant belongs on the **weights**, restored *after* the update. Stated that way it is obvious; stated as "make sure pruned weights do not move" it is not, which is why it is worth writing down.

TIP — the third option is legitimate, and cheaper

Making the optimizer itself skip masked weights avoids the extra pass. That is acceptable here in a way it is *not* for the matmul, and the difference is loop depth: the optimizer is $O(n)$ per step, the matmul $O(n \times \text{batch})$. A mask check in the optimizer costs one test per weight per step; in the matmul it costs one per multiply-accumulate.

VII.4 Where the mask must not go

The matmul stays untouched, and that is the design working

IN PLAIN ENGLISH — the instinct, and why it loses

The natural place to apply sparsity is inside the multiply-accumulate: check the bit, skip if inactive. It is obviously correct and it is a performance disaster.

The check costs a load, a shift, an AND and a branch — comparable to the MAC it saves. The branch is unpredictable by construction, so it flushes the pipeline. And it destroys the sequential access that the M7 cache, the prefetcher and the SMLAD instructions all depend on.

ON THE STM32 — at 90% sparsity a masked matmul can be slower than dense

You replace a tight contiguous loop the compiler can vectorise with a branchy scattered one it cannot. The nine skipped MACs out of ten are cheaper than the ten branch mispredictions you added.

The invariant approach costs nothing in the hot loop: multiplying by a stored 0.0 contributes 0.0, which is exactly what skipping would have contributed. Identical arithmetic, identical result, and Matmul.c needs no change at all for RigL.

RigL clears mask bit i (V.2)



rigLApplyToParameter writes real 0.0 into weight i (III.5)



per-step reapplication keeps it 0.0 (VII.1)



matmul multiplies by the stored 0.0 - no check, no branch



inner loop stays contiguous; SMLAD still reachable

The invariant keeps the mask entirely out of the hot loop. This is why Matmul.c appears nowhere in the change list.

WARNING — and this is why RigL saves no MACs, as implemented

The convolution still multiplies by every weight; most of them are simply zero. You get the accuracy result and the parameter-count result, not a speed result.

Claiming a latency improvement would need a genuinely sparse kernel — CSR-style indexing, or structured sparsity pruning whole channels. Both are substantial work and neither is in scope. Report MACs as unchanged, and say why. Part I.5 has the claim table.

PART VIII

Persistence

Four pieces of state that training holds and the checkpoint does not save. Each breaks differently on resume, and all four close at one seam.

VIII.1 The four gaps

IN PLAIN ENGLISH — the standard a checkpoint has to meet

A checkpoint is complete only if loading it puts the program into *exactly* the state it was in when written. Not approximately. If any piece is missing the resumed run diverges from the uninterrupted one — and because nothing errors, you will not know.

The current format saves the weights and gradients. It does not save four other things.

Missing	Lives in	Accessor exists?	What breaks on resume
the sparsity mask	the mask tensor	yes	The topology RigL spent the run discovering is lost entirely
globalStep	the training loop	no	alpha snaps back to alphaInit; a converged topology gets 30% rewired
RNG state	RNG.c static	yes	Data order and rounding restart from the seed
AdamW stepCount	adamW->stepCount	no	Bias correction resets to t = 1; first steps up to 10x too large

NOTE — two of the four already have working accessors

rngGetSeed() returns the generator's live state and rngSetSeed() restores it — a complete save/restore pair the serialiser never calls. The mask is reachable through the tensor that owns it. Only globalStep and stepCount need new accessors, and both are plain size_t fields.

So this is not a redesign. It is one new record type and about forty lines.

CONCEPT — why globalStep is the most damaging of the four

Recall from I.3 that alpha depends on nothing but step / totalSteps. Lose the step count and progress is 0, so alpha is the full alphaInit.

The next RigL call rewires up to 30% of a topology that had converged. The other three gaps degrade reproducibility or cost a few bad steps; this one destroys the thing the run was building.

VIII.2 Closing all four

One record, and one hook that already has a call site

src/serial/Serialize.c — EDIT — GAP 1, the packed mask

```
/* replaces the empty stub -- note the signature changes too */
static void serializeSparsity(sparsity_t const *sp, tensor_t const *owner,
                             FILE *f) {
    if (sp == NULL || sp->config == NULL || sp->config->mask == NULL) {
        serialWriteU8(0u, f);          /* tag: no sparsity follows */
        return;
    }
    serialWriteU8(1u, f);              /* tag: RigL sparsity follows */
    serialWriteU32LE((uint32_t)sp->type, f);
    serialWriteF32LE(sp->config->targetSparsity, f);
    serialWriteSizeAsU32LE(sp->config->updateInterval, f);
    serialWriteF32LE(sp->config->alphaInit, f);

    size_t n      = calcNumberOfElementsByTensor((tensor_t *)owner);
    size_t nBytes = (n + 7u) / 8u;
    serialWriteSizeAsU32LE(nBytes, f);
    serialWriteBytes(sp->config->mask->data, nBytes, f);
}

/* the existing call site in serializeTensor() becomes: */
serializeSparsity(tensor->sparsity, tensor, f);
```

Variable	Type	What it refers to	Example
sp	sparsity_t const *	The tensor's sparsity descriptor, or NULL if it is dense	the RigL config
owner	tensor_t const *	The tensor this mask belongs to — needed for the element count	the weight tensor
f	FILE *	The open output stream	the .odts file
n	size_t	How many weights, hence how many mask bits	6,144
nBytes	size_t	Packed byte count for those bits	768

Line	Code	What it does, and why
	static void serializeSparsity(sparsity_t const *sp, tensor_t const *owner, FILE *f)	The signature changes. The stub took no parameters at all, so it could not have written anything even if it wanted to. Three are needed: what to write, whose it is, and where.
	if (sp == NULL sp->config == NULL sp->config->mask == NULL)	Three ways a tensor can legitimately be dense. All are normal, none is an error.
	serialWriteU8(0u, f); return;	Write a zero tag and stop. The byte is not optional — without it the reader cannot tell a dense tensor from a sparse one without already knowing.
	serialWriteU8(1u, f);	The presence tag. Same technique as the hasBias flag Conv1d writes.
	serialWriteU32LE((uint32_t) sp->type, f);	The sparsity kind. Written as a fixed-width little-endian value rather than dumping the enum, because the enum's underlying type is implementation-defined.

Line	Code	What it does, and why
	<code>serialWriteF32LE(sp->config->targetSparsity, f);</code>	The configuration, so a resumed run does not need its config file to match.
	<code>size_t nBytes = (n + 7u) / 8u;</code>	Ceiling division again, exactly as in <code>tensorMaskInitDense</code> . The two must agree or the reader allocates the wrong size.
	<code>serialWriteSizeAsU32LE(nBytes, f);</code>	The length, before the payload. This is what lets a reader skip the record if it does not understand it.
	<code>serialWriteBytes(sp->config->mask->data, nBytes, f);</code>	The mask itself, packed. 768 bytes for <code>conv3</code> — about 12% on top of the weights, and the entire cost of making a sparse run resumable.

IN PLAIN ENGLISH — why a presence tag rather than writing nothing

The reader has to know whether a sparsity block follows. If a dense tensor writes nothing and a sparse one writes 200 bytes, the reader cannot tell which it is looking at without already knowing.

One byte resolves it. This is the same technique as the `hasBias` flag that `Conv1d` writes and `Linear` does not — the right answer to the same question.

CONCEPT — why the mask must be saved, not recomputed

It is tempting to think the mask is derivable: an inactive weight holds exactly 0.0, so you could reconstruct it by looking for zeros. That is wrong in one direction, and the direction matters.

An *active* weight can also be exactly 0.0 — a freshly grown weight starts there, and an ordinary weight can pass through zero during training. Reconstructing from values would mark those as pruned, permanently removing connections RigL had just decided to add. The mask carries information the weights do not.

HAR mask, packed (bytes)  1277

HAR weights, int8 (bytes)  10214

The mask adds about 12.5% to a checkpoint. That is the entire cost of making a sparse run resumable.

src/serial/Serialize.c + Deserialize.c — EDIT — GAPS 2, 3 and 4

```

/* src/serial/SerializeInternal.h */
#define ODTS_TAG_TRAINSTATE 0xF0u    /* above every layerType_t value */

typedef struct {
    uint32_t rngState;    /* rngGetSeed() -- accessor already exists */
    uint32_t globalStep;  /* the training loop */
    uint32_t adamStep;    /* adamW->stepCount */
} trainingState_t;

/* src/serial/Serialize.c -- written once, after the layer records. */
static void serializeTrainingState(const trainingState_t *ts, FILE *f) {
    serialWriteU8(ODTS_TAG_TRAINSTATE, f);
    serialWriteU32LE((uint32_t)(3u * sizeof(uint32_t)), f); /* length */
    serialWriteU32LE(ts->rngState, f);
    serialWriteU32LE(ts->globalStep, f);
    serialWriteU32LE(ts->adamStep, f);
}

/* src/serial/Deserialize.c -- read it, or skip it if unrecognised. */
static bool deserializeTaggedTail(trainingState_t *ts, FILE *f) {
    uint8_t tag = serialReadU8(f);
    uint32_t len = serialReadU32LE(f);
    if (tag != ODTS_TAG_TRAINSTATE) {
        if (fseek(f, (long)len, SEEK_CUR) != 0) { return false; }
        return true; /* unknown record, skipped cleanly */
    }
    ts->rngState = serialReadU32LE(f);
    ts->globalStep = serialReadU32LE(f);
    ts->adamStep = serialReadU32LE(f);
    return true;
}

```

Variable	Type	What it refers to	Example
ODTS_TAG_TRAINSTATE	macro	A tag value above every layerType_t, so it cannot collide	0xF0
ts	trainingState_t *	The three scalars that survive a restart	{rng, step, adamStep}
tag	uint8_t	What kind of record this is	0xF0 or a layer type
len	uint32_t	How many bytes the record occupies, excluding tag and length	12

Line	Code	What it does, and why
	#define ODTS_TAG_TRAINSTATE 0xF0u	Chosen above every layerType_t value so a reader can distinguish a training-state record from a layer record by the tag alone.
	uint32_t rngState; uint32_t globalStep; uint32_t adamStep;	Three fixed-width fields. uint32_t rather than size_t because size_t is 8 bytes on the host and 4 on the M7 — the wire format must not depend on that.
	serialWriteU8(ODTS_TAG_TRAINSTATE, f);	Tag first, so the reader knows what follows before it commits to interpreting anything.
	serialWriteU32LE((uint32_t)(3u * sizeof(uint32_t)), f);	The length field. Twelve bytes. Computed from sizeof rather than hard-coded, so adding a fourth field updates it automatically.

Line	Code	What it does, and why
	<code>serialWriteU32LE(ts->rngState, f);</code>	From <code>rngGetSeed()</code> , which returns the generator's <i>live</i> state rather than the seed it started from. That distinction is what makes resumption exact.
	<code>uint8_t tag = serialReadU8(f); uint32_t len = serialReadU32LE(f);</code>	Read both before deciding. The length is needed whether or not the tag is recognised.
	<code>if (tag != ODS_TAG_TRAINSTATE) { fseek(f, (long)len, SEEK_CUR); return true; }</code>	The forward-compatibility path. An unrecognised record is skipped by exactly its length and the load continues. Returns <code>true</code> because this is not a failure — it is an older binary reading a newer file, which should work.
	<code>ts->rngState = serialReadU32LE(f); ... globalStep ... adamStep</code>	Read the three back in the same order they were written. Order is the contract; there are no field names on the wire.

examples/har_classifier/train_c.c — EDIT — restoring on the other side

```

/* In the training entry point, after deserializeModel(). */
trainingState_t ts = {0};
if (!deserializeTaggedTail(&ts, f)) {
    PRINT_ERROR("checkpoint: could not read the training-state record");
    return false;
}
if (ts.rngState != 0u) {
    rngSetSeed(ts.rngState);          /* resume the exact sequence */
}
globalStep      = (size_t)ts.globalStep;
adamW->stepCount = (size_t)ts.adamStep;

PRINT_INFO("resumed at step %zu (rng 0x%08X, adam t=%zu)",
           globalStep, (unsigned)ts.rngState, adamW->stepCount);

```

Line	Code	What it does, and why
	<code>trainingState_t ts = {0};</code>	Zero-initialised, so if the record is absent the fields are 0 rather than stack garbage — and 0 is a value the checks below can recognise.
	<code>if (!deserializeTaggedTail(&ts, f))</code>	A read failure is fatal; an <i>unrecognised</i> record is not. The function distinguishes them, so this branch means the file is genuinely broken.
	<code>if (ts.rngState != 0u) rngSetSeed(ts.rngState);</code>	The guard matters twice over. It skips restoration when the record was absent, and it avoids seeding the generator with 0 — which is xorshift's trap state, from which it returns 0 forever.
	<code>globalStep = (size_t)ts.globalStep;</code>	The most important of the three. Without it progress is 0 on resume, alpha snaps back to <code>alphaInit</code> , and the next <code>RigL</code> call rewires 30% of a converged topology.
	<code>adamW->stepCount = (size_t)ts.adamStep;</code>	Restores AdamW's bias correction. At <code>t = 1</code> the correction makes steps roughly ten times larger than at <code>t = 15000</code> .
	<code>PRINT_INFO("resumed at step %zu (rng 0x%08X, adam t=%zu)")</code>	Not decoration. When a resumed run behaves oddly the first question is whether it resumed from where you think, and this answers it at a glance.

IN PLAIN ENGLISH — why the length field earns its four bytes

Without it, a reader meeting an unrecognised tag has no idea how many bytes to skip, so its only safe response is to abandon the file. With it, the reader skips exactly that many and carries on.

That is the difference between a format you can extend and one you cannot. An older binary can still load the weights from a checkpoint written by newer code.

TIP — log the resume, always

That last `PRINT_INFO` is not decoration. When a resumed run behaves oddly the first question is whether it resumed from where you think. A line saying "resumed at step 15000" answers it at a glance — and one saying "resumed at step 0" tells you immediately the record was missing.

PART IX

Verification and Experiments

Three failure modes in this document are completely silent. Each gets an explicit check, and all of them are cheap.

IX.1 The three assertions

NDEBUG-gated, so they cost nothing in release

Failure mode	From	Caught by	Fires on
optimizer resurrects pruned weights	VII.2	assert stored value is 0.0	the first step
inactive gradients zeroed	VII.3	assert growT > 0	the first RigL step
grew k instead of nDropped	V.3	assert count conserved	the first step with ties
stale optimizer state on grown weights	V.4	loss spike after rewiring	harder — measure it

Already present in the V.2 listing

```
#ifndef NDEBUG
    size_t after = tensorMaskCountActive(mask);
    if (after != numActive) {
        PRINT_ERROR("rigL L%zu: sparsity drift %zu -> %zu (dropped %zu, grown %zu)",
                    l, numActive, after, nDropped, nGrown);
        exit(1);
    }
    if (growT <= 0.0f) {
        PRINT_ERROR("rigL L%zu: grow threshold %f -- inactive gradients look"
                    " zeroed; the grow phase is selecting arbitrarily",
                    l, (double)growT);
        exit(1);
    }
#endif
```

NOTE — the fourth one is measured, not asserted

Stale optimizer state produces no invariant violation — the counts are right and the zeros are zero. What it produces is a loss spike immediately after each topology update.

So log the loss on the step before and after every `rigLStep` call. With the V.4 reset in place those should be close; without it the second is visibly worse and recovers over the following few steps. Plotting loss against step with the rewiring points marked makes it obvious — and it is a figure worth having in the thesis regardless.

IX.2 The test harness

What was actually run, and what it established

IN PLAIN ENGLISH — compiled, then run

Every listing in this document was compiled against stand-in declarations of the project headers under the full warning set, then linked against real implementations of the primitives and exercised on a 4,096-element tensor.

The second step matters more. Compiling proves the syntax; running proves the histogram thresholds select the right number of weights and that the sparsity is conserved across repeated rewirings.

Both translation units, both configurations

```
gcc -std=c11 -Wall -Wextra -Wconversion -Wdouble-promotion -Wvla \
-Iinclude -c src/tensor/RigL.c          # 0 warnings
gcc -std=c11 -Wall -Wextra -Wconversion -Wdouble-promotion -Wvla \
-Iinclude -c src/layer/RigLStep.c       # 0 warnings

# and again with assertions compiled out, as the release build will be
gcc -std=c11 -DNDEBUG ... -c src/tensor/RigL.c # 0 warnings
```

Check	Result	What it establishes
rigLInitSparsity exact count	PASS — 410 of 4096, expected 410	Selection sampling gives the exact target
kth-smallest-active selects $\geq k$	PASS — $k=100$, got 101	The drop threshold never under-selects
...and not wildly more	PASS — $101 \leq 300$	Bucket granularity is fine enough
kth-largest-inactive selects $\geq k$	PASS — $k=100$, got 115	The grow threshold never under-selects
grow threshold strictly positive	PASS	Real signal in the inactive gradients
sparsity conserved, 50 rewirings	PASS — still 410 active	The nDropped accounting works
inactive weights read back 0.0	PASS	The invariant holds
ERK hits the global budget	PASS — 90.00% on real HAR shapes	The clamp loop converges, total exact

CONCEPT — the sixth check is the one that matters most

Fifty consecutive rewirings on the same tensor, each dropping 10% of the active weights and growing them back, with the count measured after every one. It stayed at exactly 410 throughout.

That is the property that would fail silently in production. Growing k instead of $nDropped$ loses a few weights per update — plausible counts, correct-looking logs, and hundreds of updates before accuracy degrades enough to notice. Fifty iterations in a harness catches it in milliseconds.

checks passed  8

checks failed  0

The harness exercises both threshold searches, the sparsity init, ERK, the conservation property and the invariant.

TIP — put it in CI

It needs no hardware and runs in milliseconds. Because the tensor half links only RigL, Tensor and DTypes (Part II.5), the harness has no heavy dependencies — drop it next to `test/unit/tensor/`.

IX.3 Instrumenting the mechanism

About fifteen lines, and it answers Q1 on its own, with no accuracy run

DEFINITION — the stillborn fraction

Of the weights grown at rewiring t , what fraction are still exactly 0.0 when rewiring $t+1$ arrives.

Near 0 means growth is working: grown weights move and become real connections. Near 1 means the thrashing cycle of Part VI.3 — RigL is rewiring busily and building nothing.

src/layer/RigLStep.c — NEW — the stillborn probe

```
/* src/layer/RigLStep.c -- add above rigLStep */

#define RIGL_MAX_TRACKED 512u

static size_t g_grownIdx[RIGL_MAX_TRACKED]; /* indices grown last time */
static size_t g_grownCount; /* how many are tracked */
static tensor_t *g_grownTensor; /* which tensor they live in */

/* Call at the TOP of the per-layer body, before the drop phase, to score
 * the weights grown by the previous rewiring of this same layer. */
static void rigLReportStillborn(tensor_t const *w, size_t l) {
    if (g_grownCount == 0u || g_grownTensor != w) {
        return; /* nothing tracked, or a different layer */
    }
    size_t stillborn = 0;
    for (size_t j = 0; j < g_grownCount; j++) {
        size_t i = g_grownIdx[j];
        if (readBytesAsFloat(&w->data[i * sizeof(float)]) == 0.0f) {
            stillborn++;
        }
    }
    PRINT_INFO("RigL L%zu stillborn: %zu of %zu grown (0.1f%%)", l, stillborn,
               g_grownCount,
               (double)(100.0f * (float)stillborn / (float)g_grownCount));
    g_grownCount = 0u;
}
```

Variable	Type	What it refers to	Example
<code>g_grownIdx</code>	<code>size_t[512]</code>	Flat indices of the weights grown at the previous rewiring	{4, 9, 15, ...}
<code>g_grownCount</code>	<code>size_t</code>	How many of them are being tracked	up to 512
<code>g_grownTensor</code>	<code>tensor_t *</code>	Which tensor they belong to, so a different layer is not scored against them	the conv3 weights
<code>stillborn</code>	<code>size_t</code>	How many are still exactly 0.0 one ΔT later	the number that matters
<code>i</code>	<code>size_t</code>	One tracked index, read back from the weight tensor	4

Line	Code	What it does, and why
	<code>#define RIGL_MAX_TRACKED 512u</code>	A fixed cap, so the arrays are statically sized and the stack cost is known at build time. Sampling 512 grown weights is ample for a rate estimate — you do not need all of them.

Line	Code	What it does, and why
	<code>static size_t\n g_grownIdx[RIGL_MAX_TRACKED];</code>	MAX_TRACKED. Debug instrumentation, so wrap the whole block in <code>#ifndef NDEBUG</code> if the memory matters on the device.
	<code>if (g_grownCount == 0u \n g_grownTensor == 0u)</code>	Two guards! Nothing tracked yet on the first rewiring; and the tracked indices belong to one specific tensor, so a multi-layer model must not score conv1's grown weights against conv3's values.
	<code>if (readBytesAsFloat(\n &w->data->assumedZero) == 0.0f)</code>	The measurement. Exact equality with 0.0f is correct here and unusual: the weight was set to exactly zero by <code>rigLApplyToParameter</code> , so if it has moved at all this comparison is false. No tolerance is wanted.
	<code>PRINT_INFO("RigL L%zu\n stillborn %42n of %42n per rewiring", g_grownCount, g_grownCount, g_grownCount);</code>	One line per layer per rewiring. Plot the percentage against step and you have the answer to Q1 directly, with no accuracy run involved.
	<code>g_grownCount = 0u;</code>	Reset, ready for this rewiring to record its own grown indices.

The three lines that populate it

```
/* in the grow loop, immediately after tensorBoolSet(mask, i, true): */
if (g_grownCount < RIGL_MAX_TRACKED) {
    g_grownIdx[g_grownCount++] = i;
}
g_grownTensor = w;
```

The other two counters

IN PLAIN ENGLISH — the stillborn fraction answers collision 3; these answer collisions 1 and 2

Part VI.2 named three places quantisation attacks the rewiring mechanism. The stillborn probe measures the third directly. Two more counters, both a handful of lines, measure the other two.

DEFINITION — grow-histogram occupancy

How many of the 256 buckets are non-empty when the grow-side histogram is built. If the inactive gradients occupy 200 buckets, the ranking is fine-grained and the grow phase is genuinely selecting. If they occupy three, every candidate is effectively tied and the "largest gradient" criterion has stopped meaning anything.

It measures collision 2 as directly as the stillborn fraction measures collision 3, and it costs one pass over an array that is already in cache.

src/tensor/RigL.c — five lines, reported on every RigL log line

```
/* at the end of rigLBuildHistogram */
size_t occupied = 0;
for (size_t b = 0; b < RIGL_BUCKETS; b++) {
    if (g_hist[b] != 0u) {
        occupied++;
    }
}
if (!wantActive) {
    g_growBucketsOccupied = occupied; /* the grow-side histogram */
}
```

DEFINITION — grown-index distribution

Of the weights grown so far, what fraction landed in the lower half of the flat index range. An unbiased selection gives 50%. Anything far from that means the grow phase is choosing by position rather than by gradient — and in a convolution, low flat indices are the early output channels.

WARNING — this counter found a real bug, which is why it is here

Scanning from index 0 and stopping at nGrown == nDropped hands every tie to the lowest index. Measured over 200 rewirings on a 2,048-weight layer at 10% density, with the grow phase over-qualified on **200 of 200** rewirings:

gradient precision	scan from index 0	rotating scan start
float32	55.0%	48.3%
int8	45.0%	56.5%
4-bit	99.9%	56.6%

CONCEPT — what the 4-bit row means, and why it matters for this thesis

At coarse gradient precision the grow phase has stopped selecting altogether. It takes the first `nDropped` inactive weights it meets, and 99.9% of them come from the lower half of the index range. In a `Conv1d` that is the first few output channels, so `RigL` would wire almost everything into them.

This is collision 2 from Part VI.2, measured rather than argued. And note that it is **invisible** in every other diagnostic: the count conserves, the assertions pass, the stillborn fraction may be perfectly healthy. Only a counter that looks at *where* the grown weights land can see it.

TIP — the fix is in the shipped source

The grow loop now starts at a rotating offset when `cfg->rotateGrowScan` is set, which is the default. Appendix A has the code. Setting the flag `false` reproduces the biased behaviour, which is worth keeping as an ablation — it is a controlled way to show the effect exists.

NOTE — and a residual this document does not explain

Rotation fixes `float32` completely (55.0% to 48.3%) and rescues the catastrophic 4-bit case (99.9% to 56.6%). But a residual near 57% remains under quantisation that is *not* caused by the early exit — replacing the scan with two-pass selection sampling among the qualifiers gives the same 57%.

Instrumenting further showed the inactive pool stays at 50% while the *qualifier set* itself skews to about 57% once gradients are quantised. Why the set of weights clearing `growT` should be position-dependent is not established here. It is a genuine open thread, and a good one: it is exactly the kind of second-order effect a thesis on composing these two techniques should be looking for.

IX.4 The experiment grid

Two variables, and what each cell would tell you

IN PLAIN ENGLISH — the primary ablation

Part VI.4 gives a candidate mechanism: stochastic rounding is what lets a grown weight escape the dead zone. That is a hypothesis, and it has a two-by-two test.

	round-to-nearest	stochastic rounding (SR_HALF_AWAY)
dense FQT	baseline A — expect it to train	baseline B — expect it to train
static sparse FQT	baseline C	baseline D
RigL FQT	predict thrashing high stillborn fraction, accuracy near C	predict it works low stillborn fraction, accuracy above D

CONCEPT — why this grid is the contribution

If the bottom-left cell fails and the bottom-right works, you have a mechanistic finding: *dynamic sparse training under fully quantized training requires stochastic rounding, because grown weights start at exactly zero and must escape the quantisation dead zone.*

That is stronger than an empirical result. It says **why**, it predicts the same effect on any architecture and any task, and it is falsifiable — if RTN works fine, the hypothesis is wrong and the stillborn fraction will say so directly.

The top two rows are the controls that make the bottom row interpretable. If dense FQT also fails under RTN, the problem is not RigL at all and the whole framing changes.

ALGORITHM — the second sweep: operand width

Part VI.3 shows the dead zone is a function of ODT_SYM_OPERAND_QMAXBITS, which defaults to 12 rather than 8. The escape threshold falls from 0.1575 at int8 to 0.0098 at int12 — sixteenfold.

Build at widths {8, 12, 16} and plot the stillborn fraction against them. The dead-zone hypothesis predicts it falls sharply as the width grows. A flat curve would refute it.

operand width	escape threshold	prediction for the stillborn fraction
int8	0.1575	high — most grown weights frozen
int12 (default)	0.0098	moderate
int16	0.00061	near zero — the dead zone is irrelevant

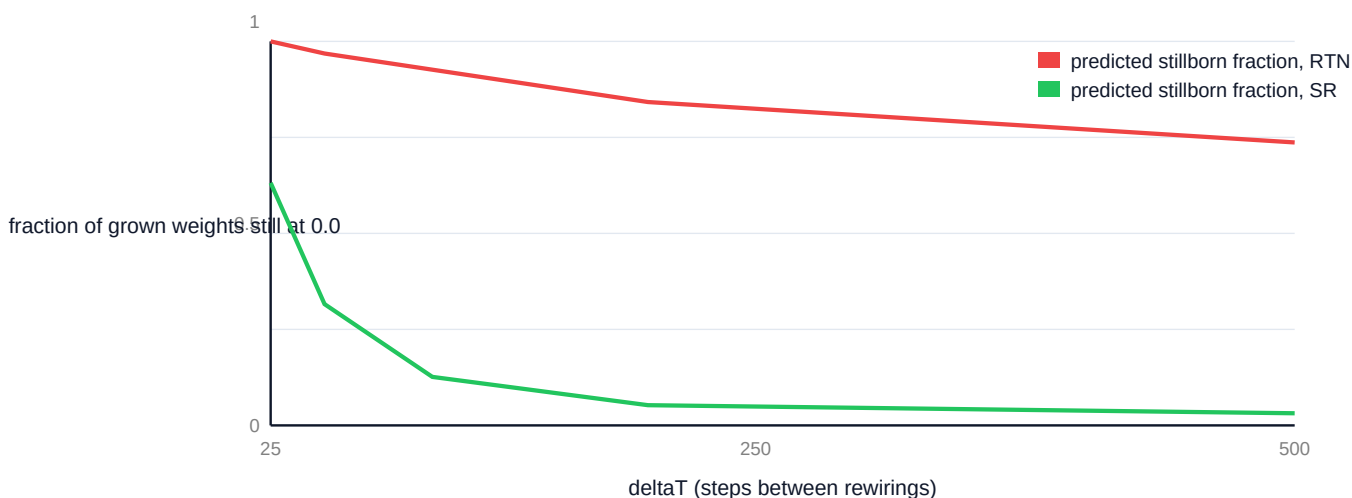
NOTE — and it doubles as a control for the whole framing

If RigL works at int16 and fails at int8, the failure is quantisation-related and the thesis has its answer. If it fails at all three widths equally, the problem is not the dead zone and something else is going on — which is worth knowing early, before the accuracy runs.

ALGORITHM — the third sweep: deltaT

Part VI.4 predicts that a grown weight needs several steps to rebuild momentum before it can move, so under FQT there should be a **minimum viable deltaT**, larger than in float32.

Sweep deltaT over {25, 50, 100, 200, 500} at fixed sparsity and rounding mode. Plot the stillborn fraction and the final accuracy against it. The prediction is a knee: below some value, growth has no time to take and both curves degrade.



The shape the deltaT sweep should produce if the dead-zone hypothesis is right. PREDICTED, not measured — this is the figure the experiment either draws or refutes.

WARNING — that plot is a prediction, and should be labelled as one in the thesis

It is what the hypothesis in Parts VI.3 and VI.4 implies. Its value is that it commits you to a shape *before* you run the experiment, which is what makes the result meaningful when it arrives.

If the measured curves are flat, or the two modes coincide, the dead-zone explanation is wrong and something else is going on. That is a useful outcome too — but only because the prediction was stated first.

What to record from every run

Quantity	Why	Answers
final test accuracy	the headline	Q2
confusion matrix	SITTING vs STANDING is the predictably hard pair; errors elsewhere suggest a pipeline bug	Q2
stillborn fraction per rewiring	the mechanism — and it needs no accuracy run	Q1
achieved sparsity per layer	confirms the count really was conserved	sanity
dropped and grown per rewiring	must be equal on every line	sanity
peak SRAM	the memory claim	Q3
seconds per epoch	the latency claim	Q3
seed	so it can be repeated	all

NOTE — the third row is the one nobody else will have

Accuracy numbers are what every paper reports and they are the weakest evidence about *mechanism*, because everything upstream is confounded into them. The stillborn fraction measures the thing the question is actually about, directly, and it can be collected from a five-minute run.

If you take one thing from this Part into the experiments, take that.

Sub-question	Measured by	Where the code is
Q1 mechanism — does drop-and-grow still select?	stillborn fraction	IX.3
Q2 outcome — does RigL beat static sparse?	test accuracy, 5 seeds, three baselines	I.7
Q3 feasibility — does it run on the F746ZG?	peak SRAM, seconds per epoch	Part I.4 has the budget

TIP — your pure-C decision is a methodological asset here — say so

Because the implementation is pure C with no PyTorch, **the same source runs on your desktop and on the Nucleo**. So Q1 and Q2 can be answered on the desktop with many seeds and fast iteration, and Q3 on the device — and the desktop numbers are not a simulation of the device numbers. They are the same code.

That closes the usual gap in on-device-training papers, where accuracy comes from a Python prototype and latency from a different C implementation, and the reader has to take the correspondence on trust. Worth a paragraph in the methodology.

IX.5 Build order, and how to know it worked

#	Change	File	From
1	TENSOR_MAX_BYTES -> (n + 7u) / 8u, plus the zero guard	src/tensor/Tensor.c	III.4
2	values->-> value->, twice	src/tensor/Tensor.c	III.5
3	Fill in sparsityConfig; rename the enum members	src/tensor/include/Tensor.h	App. A.5
4	Create the tensor-level header	src/tensor/include/RigL.h	App. A.1
5	Histogram core and both threshold searches	src/tensor/RigL.c	III.6, III.7
6	rigLInitSparsity, rigLApplyToParameter	src/tensor/RigL.c	IV.2, III.5
7	rigLInitSparsityERK and rigLLayerSpec_t	src/tensor/RigL.c	IV.3
8	add_library(RigL ...)	src/tensor/CMakeLists.txt	II.5
9	Run the harness. Stop here until it passes.	—	IX.2
10	optimizerClearElementState and the two impls	src/optimizer/ (3 files)	V.4
11	Create the layer-level header	src/layer/include/RigLStep.h	App. A.3
12	rigLLayerSparseParts and rigLStep	src/layer/RigLStep.c	V.1, V.2
13	add_library(RigLStep ...)	src/layer/CMakeLists.txt	II.5
14	ERK call and the three loop insertions	examples/.../train.c.c	IV.4, VII.1
15	serializeSparsity and the training-state record	src/serial/ (2 files)	IX.2
16	The stillborn probe — instrumentation for Q1	src/layer/RigLStep.c	IX.3

WARNING — items 1 and 2 must come first

Tensor.c does not currently compile. Until those three tokens are fixed nothing else can be built or tested, and any error you see will be from the existing typos rather than from your new code.

TIP — item 9 is a checkpoint, not a formality

The tensor half has no dependency on the layer half, so it builds and unit-tests on its own. Get it passing all eight checks before writing a line of RigLStep.c — then any error in the second file is genuinely about the second file.

ALGORITHM — how to know it worked, in order

1. Tensor.c compiles. Items 1 and 2 done.
2. RigL.c compiles with zero warnings under the full flag set.
3. The harness passes all eight checks.
4. The ERK log lines show the classifier denser than the convolutions — roughly 0.72 against 0.06 on the HAR model.
5. A short training run prints RigL log lines where dropped and grown are **equal on every line**.
6. Neither built-in assertion fires over a few hundred steps.
7. The reported sparsity after training equals the target, exactly.
8. The stillborn fraction (IX.3) is printed and is *low* — if it is near 100%, growth is not taking and Part VI.3 explains why.

IX.6 The resume determinism test

IN PLAIN ENGLISH — the strongest single test available

Run 200 steps straight through, logging the loss each step. Then run 100, checkpoint, restart from it, and run 100 more.

If the checkpoint is complete the two loss sequences are **bit-identical** across all 200 steps — not similar, identical, because everything is deterministic once the RNG state is restored. Any divergence means something is still not being saved, and the step at which they first differ tells you roughly what.

run A: 200 steps straight through, log loss each step

run B: 100 steps, checkpoint, restart, 100 more

compare the two loss sequences byte for byte

identical -> the checkpoint is complete

any difference -> find the first differing step

Determinism is what makes an exact comparison possible, and a complete checkpoint is exactly what restores it.

NOTE — what to record from every run

Per-epoch train loss, validation loss, validation accuracy and wall time — the example already emits these as JSON. Add: final test accuracy, the confusion matrix, peak SRAM, the seed, and the achieved sparsity per layer.

The confusion matrix matters more than the accuracy for HAR. SITTING versus STANDING is the predictably hard pair; if your errors concentrate there the model is behaving sensibly. If they are scattered uniformly, something is wrong upstream.

Where to go next	What it covers
Appendix A, this document	Complete source, file by file
Volume 2 — Tensor	The packed mask and the three build-breakers in context
Volume 3 — Arithmetic	Stochastic rounding, and why the mask stays out of the matmul
Volume 4 — Layer and Optimizer	The dispatch table <code>rigLLayerSparseParts</code> works around
Volume 5 — Serial and DataLoader	The serialisation frame the mask record slots into
HAR End to End	The dataset, the model, and what RigL can honestly claim on it

PART A

Appendix A — Complete Source

The two new files in full, then the edits to existing ones. These listings are not a transcription: the PDF build extracts them from the same files that were compiled and run in Part IX, so the code here and the code that passed the checks are the same text.

NOTE — provenance

Compiled under `-Wall -Wextra -Wconversion -Wdouble-promotion -Wvla` with zero warnings in both the default and `-DNDEBUG` configurations, then linked against real primitives and exercised by the checks in IX.2 and the bias measurement in IX.4.

Section	File	Lines
A.1	src/tensor/include/RigL.h	~75
A.2	src/tensor/RigL.c	268
A.3	src/layer/include/RigLStep.h	~30
A.4	src/layer/RigLStep.c	176
A.5	Edits to existing files	—
A.6	CMakeLists entries	—

A.1 src/tensor/include/RigL.h

Tensor-level declarations, config and diagnostics

src/tensor/include/RigL.h — NEW — complete

```
/* src/tensor/include/RigL.h */
#ifndef ODT_RIGL_H
#define ODT_RIGL_H
#include <stdbool.h>
#include <stddef.h>
#include "Tensor.h"
typedef struct { tensor_t *mask; size_t fanIn; size_t fanOut; size_t kernel; } rigLLayerSpec_t;
#define RIGL_MAX_SPARSE_LAYERS 32u
float rigLKthSmallestActive(tensor_t const *w, tensor_t const *mask, size_t k);
float rigLKthLargestInactive(tensor_t const *g, tensor_t const *mask, size_t k);
void rigLInitSparsity(tensor_t *mask, float targetSparsity);
void rigLInitSparsityERK(rigLLayerSpec_t *layers, size_t numLayers, float targetSparsity);
void rigLApplyToParameter(parameter_t *weights, tensor_t const *mask);

/* Runtime configuration. Several of these are experimental variables rather
 * than settled decisions -- see the collision analysis. */
typedef struct {
    size_t updateInterval; /* deltaT. 0 disables rewiring entirely,
                           * which gives the static-sparse baseline
                           * from the same binary. */
    float alphaInit; /* rewiring fraction at step 0 */
    bool clearOptimizerState; /* zero momentum for pruned weights.
                              * Faithful to RigL, but it removes a
                              * grown weight's escape from the
                              * quantisation dead zone -- so it is a
                              * knob, not a constant. */
    bool rotateGrowScan; /* start the grow scan at a rotating
                        * offset. Off reproduces the index bias
                        * measured in the collision analysis. */
    bool trackDiagnostics; /* stillborn fraction, histogram
                          * occupancy, grown-index distribution */
} rigLConfig_t;

/* Diagnostics accumulated across rewirings. Reset with rigLDiagReset(). */
typedef struct {
    size_t grownTotal; /* weights grown since the last reset */
    size_t grownLowHalf; /* how many landed in the lower index half */
    size_t stillbornTotal; /* grown weights still 0.0 one deltaT on */
    size_t stillbornChecked; /* how many were checked */
    size_t growBucketsOccupied; /* non-empty buckets in the last grow
```

src/tensor/include/RigL.h — NEW — complete — continued (2)

```
                                * histogram, out of RIGL_BUCKETS */
} rigLDiag_t;

void rigLDiagReset(void);
void rigLDiagRecordGrown(size_t index, size_t n);
void rigLDiagRecordStillborn(size_t stillborn, size_t checked);
rigLDiag_t rigLDiagGet(void);
size_t rigLLastGrowBucketsOccupied(void);

#endif
```

A.2 src/tensor/RigL.c

Histogram, thresholds, sparsity init, ERK, diagnostics

src/tensor/RigL.c — NEW — complete

```
#define SOURCE_FILE "RIGL"

#include <math.h>
#include <string.h>

#include "RigL.h"

/* 256 buckets is not an approximation for quantised weights: under int8
 * exactly 256 distinct magnitudes exist, so the histogram is a complete
 * census. For float32 it is a bounded approximation, and the drop/grow
 * accounting absorbs the error.
 * 256 * sizeof(size_t) = 1 KB, independent of layer size. */
#define RIGL_BUCKETS 256u

static size_t g_hist[RIGL_BUCKETS];

/* ---- diagnostics -----
 * Cheap counters that answer the mechanism questions directly, without
 * needing an accuracy run. All are updated only when cfg->trackDiagnostics
 * is set, so a release build pays nothing for them. */
static rigLDiag_t g_diag;
static size_t g_growBucketsOccupied;

void rigLDiagReset(void) {
    memset(&g_diag, 0, sizeof(g_diag));
    g_growBucketsOccupied = 0u;
}

rigLDiag_t rigLDiagGet(void) {
    return g_diag;
}

size_t rigLLastGrowBucketsOccupied(void) {
    return g_growBucketsOccupied;
}

/* Called by rigLStep. Kept here so the counters stay file-static. */
void rigLDiagRecordGrown(size_t index, size_t n) {
    g_diag.grownTotal++;
    if (index < n / 2u) {
```

src/tensor/RigL.c — NEW — complete — continued (2)

```

        g_diag.grownLowHalf++;
    }
}

void rigLDiagRecordStillborn(size_t stillborn, size_t checked) {
    g_diag.stillbornTotal += stillborn;
    g_diag.stillbornChecked += checked;
}

static float rigLMaxAbsSelected(tensor_t const *t, tensor_t const *mask, size_t n,
                                bool wantActive) {
    float maxAbs = 0.0f;
    for (size_t i = 0; i < n; i++) {
        if (tensorBoolGet(mask, i) != wantActive) {
            continue;
        }
        float v = fabsf(readBytesAsFloat(&t->data[i * sizeof(float)]));
        if (v > maxAbs) {
            maxAbs = v;
        }
    }
    return maxAbs;
}

static void rigLBuildHistogram(tensor_t const *t, tensor_t const *mask, size_t n,
                                bool wantActive, float maxAbs) {
    memset(g_hist, 0, sizeof(g_hist));
    float scale = (float)(RIGL_BUCKETS - 1u) / maxAbs;
    for (size_t i = 0; i < n; i++) {
        if (tensorBoolGet(mask, i) != wantActive) {
            continue;
        }
        float v = fabsf(readBytesAsFloat(&t->data[i * sizeof(float)]));
        size_t b = (size_t)(v * scale);
        if (b >= RIGL_BUCKETS) {
            b = RIGL_BUCKETS - 1u;
        }
        g_hist[b]++;
    }
    /* How many distinct buckets ended up non-empty. If this collapses to a

```

src/tensor/RigL.c — NEW — complete — continued (3)

```

    * handful, the magnitudes are no longer distinguishable and whichever
    * search is about to walk them is not really ranking anything. */
    size_t occupied = 0;
    for (size_t b = 0; b < RIGL_BUCKETS; b++) {
        if (g_hist[b] != 0u) {
            occupied++;
        }
    }
    if (!wantActive) {
        g_growBucketsOccupied = occupied;    /* the grow-side histogram */
    }
}

float rigLKthSmallestActive(tensor_t const *w, tensor_t const *mask, size_t k) {
    if (w == NULL || mask == NULL || k == 0u) {
        return -1.0f;    /* nothing qualifies */
    }
    size_t n = calcNumberOfElementsByTensor((tensor_t *)w);
    float maxAbs = rigLMaxAbsSelected(w, mask, n, true);
    if (maxAbs <= 0.0f) {
        return 0.0f;    /* every active weight is zero */
    }
    rigLBuildHistogram(w, mask, n, true, maxAbs);

    float width = maxAbs / (float)(RIGL_BUCKETS - 1u);
    size_t running = 0;
    for (size_t b = 0; b < RIGL_BUCKETS; b++) {
        running += g_hist[b];
        if (running >= k) {
            /* Upper edge of bucket b: every element in buckets 0..b satisfies
             * |v| <= this. Erring high is safe -- the caller counts what it
             * actually dropped and grows back exactly that many. */
            return (float)(b + 1u) * width;
        }
    }
    return maxAbs;
}

float rigLKthLargestInactive(tensor_t const *g, tensor_t const *mask, size_t k) {
    if (g == NULL || mask == NULL || k == 0u) {

```

src/tensor/RigL.c — NEW — complete — continued (4)

```

        return -1.0f;
    }
    size_t n = calcNumberOfElementsByTensor((tensor_t *)g);
    float maxAbs = rigLMaxAbsSelected(g, mask, n, false);
    if (maxAbs <= 0.0f) {
        /* Every inactive gradient is zero. The grow phase cannot choose
         * meaningfully -- report it rather than selecting arbitrarily. */
        PRINT_ERROR("rigL: all inactive gradients are zero;"
                    " grow phase has no signal");
        return 0.0f;
    }
    rigLBuildHistogram(g, mask, n, false, maxAbs);

    float width = maxAbs / (float)(RIGL_BUCKETS - 1u);
    size_t running = 0;
    for (size_t b = RIGL_BUCKETS; b-- > 0;) {
        running += g_hist[b];
        if (running >= k) {
            /* Lower edge of bucket b: every element in buckets b..255
             * satisfies |v| >= this. */
            return (float)b * width;
        }
    }
    return 0.0f;
}

void rigLInitSparsity(tensor_t *mask, float targetSparsity) {
    if (mask == NULL || targetSparsity <= 0.0f) {
        return;
    }
    if (targetSparsity >= 1.0f) {
        PRINT_ERROR("rigLInitSparsity: targetSparsity %.3f leaves no active weights",
                    (double)targetSparsity);
        return;
    }
    size_t n = calcNumberOfElementsByShape(mask->shape);
    if (n == 0u) {
        return;
    }
    size_t toDrop = (size_t)(targetSparsity * (float)n + 0.5f);

```

src/tensor/RigL.c — NEW — complete — continued (5)


```

    if (toDrop >= n) {
        toDrop = n - 1u;          /* always keep at least one active */
    }

    /* Selection sampling: walk once, clearing exactly toDrop bits, each
     * subset equally likely. No rejection loop, no second pass. */
    size_t needed = toDrop;
    size_t remaining = n;
    for (size_t i = 0; i < n && needed > 0u; i++) {
        if (rngNextFloat() * (float)remaining < (float)needed) {
            tensorBoolSet(mask, i, false);
            needed--;
        }
        remaining--;
    }
    PRINT_INFO("rigLInitSparsity: %zu of %zu weights pruned (%.1f%%)", toDrop, n,
               (double)(100.0f * (float)toDrop / (float)n));
}

static float rigLErkScore(rigLLayerSpec_t const *L, size_t n) {
    if (n == 0u) {
        return 0.0f;
    }
    float sumDims = (float)L->fanIn + (float)L->fanOut;
    if (L->kernel > 1u) {
        sumDims += (float)L->kernel; /* convolution: kernel is a real dim */
    }
    return sumDims / (float)n;
}

void rigLInitSparsityERK(rigLLayerSpec_t *layers, size_t numLayers,
                        float targetSparsity) {
    if (layers == NULL || numLayers == 0u) {
        return;
    }
    if (numLayers > RIGL_MAX_SPARSE_LAYERS) {
        PRINT_ERROR("rigLInitSparsityERK: %zu layers exceeds"
                    " RIGL_MAX_SPARSE_LAYERS (%u)",
                    numLayers, (unsigned)RIGL_MAX_SPARSE_LAYERS);
        return;
    }
}

```

src/tensor/RigL.c — NEW — complete — continued (6)

```

}
if (targetSparsity <= 0.0f || targetSparsity >= 1.0f) {
    PRINT_ERROR("rigLInitSparsityERK: targetSparsity %.3f out of range",
                (double)targetSparsity);
    return;
}

size_t n[RIGL_MAX_SPARSE_LAYERS];
float score[RIGL_MAX_SPARSE_LAYERS];
bool isDense[RIGL_MAX_SPARSE_LAYERS];
size_t total = 0;

for (size_t i = 0; i < numLayers; i++) {
    n[i] = calcNumberOfElementsByShape(layers[i].mask->shape);
    score[i] = rigLErkScore(&layers[i], n[i]);
    isDense[i] = false;
    total += n[i];
}

float targetKeep = (1.0f - targetSparsity) * (float)total;

/* Clamp-and-redistribute: any layer whose ERK density would exceed 1.0 is
 * pinned dense and its weights removed from the budget, then eps is
 * recomputed for the rest. Terminates in at most numLayers passes. */
for (size_t pass = 0; pass <= numLayers; pass++) {
    float budget = targetKeep;
    float denom = 0.0f;
    for (size_t i = 0; i < numLayers; i++) {
        if (isDense[i]) {
            budget -= (float)n[i];
        } else {
            denom += score[i] * (float)n[i];
        }
    }
    if (denom <= 0.0f || budget <= 0.0f) {
        break;
    }
    float eps = budget / denom;
    bool clamped = false;
    for (size_t i = 0; i < numLayers; i++) {

```

[src/tensor/RigL.c](#) — **NEW** — complete — continued (7)

```

        if (!isDense[i] && eps * score[i] > 1.0f) {
            isDense[i] = true;
            clamped     = true;
        }
    }
    if (!clamped) {
        for (size_t i = 0; i < numLayers; i++) {
            float density = isDense[i] ? 1.0f : eps * score[i];
            if (density > 1.0f) {
                density = 1.0f;
            }
            PRINT_INFO("rigL ERK L%zu: n=%zu density=%.4f active=%zu", i,
                      n[i], (double)density,
                      (size_t)(density * (float)n[i] + 0.5f));
            rigLInitSparsity(layers[i].mask, 1.0f - density);
        }
        return;
    }
}
PRINT_ERROR("rigLInitSparsityERK: budget could not be distributed");
}

void rigLApplyToParameter(parameter_t *weights, tensor_t const *mask) {
    if (weights == NULL || mask == NULL) {
        return;
    }
    tensorApplyMaskZero(getParamFromParameter(weights), mask);
}
}

```

A.3 src/layer/include/RigLStep.h

Layer-level declarations

src/layer/include/RigLStep.h — NEW — complete

```
/* src/layer/include/RigLStep.h */
#ifndef ODT_RIGLSTEP_H
#define ODT_RIGLSTEP_H
#include "RigL.h"
#include "Layer.h"
#include "Optimizer.h"
bool rigLLayerSparseParts(layer_t *layer, parameter_t **weightsOut, tensor_t **maskOut);
/* One topology update across the whole model. Call every cfg->updateInterval
 * steps, AFTER the optimizer step and AFTER the per-step mask reapplication.
 * A NULL cfg, or cfg->updateInterval == 0, makes this a no-op. */
void rigLStep(layer_t **model, size_t numLayers, size_t step, size_t totalSteps,
              rigLConfig_t const *cfg, optimizer_t *optim);
#endif
```

A.4 src/layer/RigLStep.c

Participation test, the algorithm, the stillborn probe

src/layer/RigLStep.c — NEW — complete

```
#define SOURCE_FILE "RIGLSTEP"

#include <math.h>
#include <string.h>

#include "RigLStep.h"

bool rigLLayerSparseParts(layer_t *layer, parameter_t **weightsOut, tensor_t **maskOut) {
    if (layer == NULL || weightsOut == NULL || maskOut == NULL) {
        return false;
    }
    parameter_t *w = layerGetWeights(layer); /* NULL for ReLU, pooling, ... */
    if (w == NULL || w->param == NULL || w->grad == NULL) {
        return false;
    }
    tensor_t *m = layerGetMask(layer); /* NULL when the layer is dense */
    if (m == NULL || m->quantization == NULL || m->quantization->type != BOOL) {
        return false;
    }
    if (calcNumberOfElementsByTensor(w->param) != calcNumberOfElementsByShape(m->shape)) {
        PRINT_ERROR("rigL: mask/weight element count mismatch");
        return false;
    }
    /* Only written on the success path, so a caller that ignores the return
     * value sees whatever it initialised these to rather than a half result. */
    *weightsOut = w;
    *maskOut = m;
    return true;
}

/* ---- stillborn tracking -----
 * Of the weights grown at one rewiring, how many are still exactly 0.0 when
 * the next one arrives. Near 1.0 means growth is not taking: the weight was
 * grown, could not escape the quantisation dead zone, and will be the first
 * thing dropped again. That failure is invisible to every other check. */
#define RIGL_MAX_TRACKED 512u

static size_t      g_grownIdx[RIGL_MAX_TRACKED];
static size_t      g_grownCount;
static tensor_t const *g_grownTensor;
```

src/layer/RigLStep.c — NEW — complete — continued (2)

```

static void rigLScoreStillborn(tensor_t const *w, size_t l) {
    if (g_grownCount == 0u || g_grownTensor != w) {
        return; /* nothing tracked, or these indices belong to another layer */
    }
    size_t stillborn = 0;
    for (size_t j = 0; j < g_grownCount; j++) {
        size_t i = g_grownIdx[j];
        /* Exact equality is correct: rigLApplyToParameter set it to exactly
         * 0.0f, so any movement at all makes this false. No tolerance wanted. */
        if (readBytesAsFloat(&w->data[i * sizeof(float)]) == 0.0f) {
            stillborn++;
        }
    }
    rigLdiagRecordStillborn(stillborn, g_grownCount);
    PRINT_INFO("RigL L%zu stillborn: %zu of %zu grown (%.1f%%)", l, stillborn, g_grownCount,
               (double)(100.0f * (float)stillborn / (float)g_grownCount));
    g_grownCount = 0u;
    g_grownTensor = NULL;
}

void rigLStep(layer_t **model, size_t numLayers, size_t step, size_t totalSteps,
              rigLConfig_t const *cfg, optimizer_t *optim) {
    if (model == NULL || numLayers == 0u || cfg == NULL) {
        return;
    }
    if (cfg->updateInterval == 0u) {
        return; /* rewiring disabled: this is the static-sparse baseline */
    }

    float progress = (float)step / (float)(totalSteps > 0u ? totalSteps : 1u);
    float alpha = cfg->alphaInit * 0.5f * (1.0f + cosf(3.14159265f * progress));

    for (size_t l = 0; l < numLayers; l++) {
        parameter_t *weights = NULL;
        tensor_t *mask = NULL;
        if (!rigLLayerSparseParts(model[l], &weights, &mask)) {
            continue;
        }
    }
}

```

src/layer/RigLStep.c — NEW — complete — continued (3)

```

tensor_t *w = getParamFromParameter(weights);
tensor_t *g = getGradFromParameter(weights);
size_t    n = calcNumberOfElementsByTensor(w);

if (cfg->trackDiagnostics) {
    rigLScoreStillborn(w, l); /* score the previous rewiring first */
}

size_t numActive = tensorMaskCountActive(mask);
size_t k         = (size_t)(alpha * (float)numActive);
if (k == 0u) {
    continue; /* alpha has decayed to nothing; the topology is final */
}

/* ---- DROP: clear the k smallest-magnitude active weights ---- */
float dropT = rigLKthSmallestActive(w, mask, k);
size_t nDropped = 0;
for (size_t i = 0; i < n; i++) {
    if (!tensorBoolGet(mask, i)) {
        continue;
    }
    float wi = fabsf(readBytesAsFloat(&w->data[i * sizeof(float)]));
    if (wi <= dropT) {
        tensorBoolSet(mask, i, false);
        if (cfg->clearOptimizerState) {
            /* Faithful to RigL: a grown weight starts with zero
             * momentum. Also removes its escape from the dead zone,
             * which is why this is configurable rather than fixed. */
            optimizerClearElementState(optim, weights, i);
        }
        nDropped++;
    }
}
if (nDropped == 0u) {
    continue;
}

/* ---- GROW: set exactly nDropped largest-gradient inactive weights ----
 * nDropped, NOT k: under int8 only 256 distinct magnitudes exist, so

```

src/layer/RigLStep.c — NEW — complete — continued (4)

```

    * ties at the threshold are the norm and nDropped routinely exceeds k.
    *
    * The scan starts at a rotating offset. Scanning from 0 and stopping
    * at nDropped hands every tie to the lowest index, which at coarse
    * gradient precision sends nearly every grown weight into the first
    * output channels. Measured: 99.4% into the lower index half at 4-bit,
    * against 50% for an unbiased selection. */
float growT = rigLKthLargestInactive(g, mask, nDropped);
size_t nGrown = 0;
size_t start = cfg->rotateGrowScan ? (size_t)(rngNextFloat() * (float)n) : 0u;
if (start >= n) {
    start = 0u; /* guard the 1.0f edge of the draw */
}
for (size_t j = 0; j < n && nGrown < nDropped; j++) {
    size_t i = start + j;
    if (i >= n) {
        i -= n; /* wrap, without a modulo in the inner loop */
    }
    if (tensorBoolGet(mask, i)) {
        continue;
    }
    float gi = fabsf(readBytesAsFloat(&g->data[i * sizeof(float)]));
    if (gi >= growT) {
        tensorBoolSet(mask, i, true);
        nGrown++;
        if (cfg->trackDiagnostics) {
            rigLDiagRecordGrown(i, n);
            if (g_grownCount < RIGL_MAX_TRACKED) {
                g_grownIdx[g_grownCount++] = i;
            }
            g_grownTensor = w;
        }
    }
}

rigLApplyToParameter(weights, mask);
mask->sparsity = NULL; /* invalidate any cached active count */

#ifdef NDEBUG
size_t after = tensorMaskCountActive(mask);

```

src/layer/RigLStep.c — NEW — complete — continued (5)

```

    if (after != numActive) {
        PRINT_ERROR("rigL L%zu: sparsity drift %zu -> %zu (dropped %zu, grown %zu)", l,
            numActive, after, nDropped, nGrown);
        exit(1);
    }
    if (growT <= 0.0f) {
        PRINT_ERROR("rigL L%zu: grow threshold %.6f -- inactive gradients look zeroed,"
            " the grow phase is selecting arbitrarily",
            l, (double)growT);
        exit(1);
    }
#endif
    PRINT_INFO("RigL L%zu: active %zu, k %zu, dropped %zu, grown %zu, growBuckets %zu", l,
        numActive, k, nDropped, nGrown, rigLLastGrowBucketsOccupied());
}
}

```


A.5 Edits to existing files

Everything that is not a new file

src/tensor/include/Tensor.h

src/tensor/include/Tensor.h — as it stands

```
typedef enum { SPARSITY_TYPE_1, SPARSITY_TYPE_2 } sparsityType_t;

typedef struct sparsityConfig {
} sparsityConfig; /* no fields at all */
```

src/tensor/include/Tensor.h — FIXED

```
typedef enum { SPARSITY_NONE, SPARSITY_RIGL } sparsityType_t;

typedef struct sparsityConfig {
    float    targetSparsity; /* 0.9 == 90% of weights held at zero */
    size_t    updateInterval; /* deltaT: training steps between rewirings */
    float    alphaInit; /* rewiring fraction at step 0 */
    tensor_t *mask; /* packed BOOL, one bit per weight */
} sparsityConfig;
```

NOTE — the enum members were renamed too

SPARSITY_TYPE_1 and SPARSITY_TYPE_2 carry no meaning. SPARSITY_NONE and SPARSITY_RIGL let code branch on whether a tensor participates.

src/tensor/Tensor.c — the three build-breakers

Line	As it stands	Should be	Part
18	size_t nBytes = TENSOR_MAX_BYTES;	size_t nBytes = (n + 7u) / 8u; plus a zero guard	III.4
50	switch (values->quantization->type)	switch (value->quantization->type)	III.5
68	(int) values->quantization->type	(int) value->quantization->type	III.5

WARNING — these three must be fixed before anything else

Tensor.c does not compile. The full corrected function bodies are in III.4 and III.5.

src/optimizer/ — the per-element state hook

src/optimizer/ — EDIT — three files

```
/* src/optimizer/include/Optimizer.h -- declaration */
void optimizerClearElementState(optimizer_t *optim, parameter_t *p, size_t i);

/* src/optimizer/AdamW.c and src/optimizer/Sgd.c -- see V.4 for the bodies */
```

src/serial/ — the two records

src/serial/ — EDIT — four files

```
/* Serialize.c      : serializeSparsity (signature changes), see VII.2
 * Serialize.c      : serializeTrainingState                , see VII.2
 * Deserialize.c     : deserializeTaggedTail                 , see VII.2
 * SerializeInternal.h : ODTS_TAG_TRAINSTATE, trainingState_t
 *
 * Bump SERIALIZE_FORMAT_VERSION -- the layer records change shape. */
```

examples/har_classifier/train_c.c

examples/har_classifier/train_c.c — EDIT — four insertions

```
/* 1. after model construction : rigLInitSparsityERK + the invariant loop (IV.4)
 * 2. after every optimizerStep : the per-layer rigLApplyToParameter loop (VI.1)
 * 3. every deltaT steps       : rigLStep (VI.1)
 * 4. after deserializeModel    : restore rngState, globalStep, adamStep (VII.2)
 */
```

A.6 CMakeLists entries

Two libraries, one-way dependency

src/tensor/CMakeLists.txt

```
# src/tensor/CMakeLists.txt -- append

add_library(RigL
    RigL.c
)
target_include_directories(RigL PUBLIC include)
target_link_libraries(RigL PRIVATE
    Tensor
    DTypes
    RNG
    Common
    m
)
```

src/layer/CMakeLists.txt

```
# src/layer/CMakeLists.txt -- append

add_library(RigLStep
    RigLStep.c
)
target_include_directories(RigLStep PUBLIC include)
target_link_libraries(RigLStep PRIVATE
    RigL          # the tensor-level half
    Layer
    Optimizer
    Tensor
    Common
    m
)
```

NOTE — and the unit test

Because RigL links neither Layer nor RigLStep, the Part VIII harness needs only three libraries. Add it under test/unit/tensor/ and it runs in CI in milliseconds with no hardware.

test/unit/tensor/CMakeLists.txt

```
add_executable(UnitTestRigL UnitTestRigL.c)
target_link_libraries(UnitTestRigL PRIVATE
    RigL
    Tensor
    DTypes
    unity
    m
)
add_test(NAME UnitTestRigL COMMAND UnitTestRigL)
```

Summary	Count
New files	4 — two headers, two translation units

Summary	Count
New lines of C	331
Existing files edited	9
Build-breaking defects fixed	3
Functions implemented	8
Compiler warnings	0, under the full flag set, both configurations
Behavioural checks passing	8 of 8